

AMAZON ENGINEERING PREPARATION

SDE-1 MASTER INTERVIEW HANDBOOK

FAANG Engineering Portfolio · Bar Raiser Preparation

CANDIDATE	Sunil Gupta
ROLE	Software Development Engineer — I
PROJECT	Carolina Furniture Concepts — MERN Operations Platform
STACK	React · Redux · Node · Express · MySQL · MSSQL · Puppeteer
COMPILED	May 25, 2026

7

PARTS

28

SECTIONS

12

STAR STORIES

11LEADERSHIP
PRINCIPLES

Confidential preparation material · Not for distribution

"Walk in calm. Lead with numbers. End on hooks."

EXEC

EXECUTIVE SUMMARY

A single, code-grounded preparation handbook for the Amazon SDE-1 loop. Every story, every primitive, every tradeoff in this document is sourced from actual implementation in `cfc/` (React + Vite + Redux frontend) and `backend/` (Node + Express + MySQL + MSSQL) — the two-showroom retail operations platform built end-to-end for Carolina Furniture Concepts.

90-SECOND ELEVATOR PITCH

"I own a MERN platform for Carolina Furniture Concepts — a two-showroom retailer in North Carolina. React + Vite + Redux Toolkit on top of Node, Express, MySQL for operational data and a read-only MSSQL legacy ERP for inventory. Three workflows: a real-time customer-opportunity queue called the UPS Board with a fairness algorithm, digital sales invoices with a returns-to-stock path, and an automated 7 PM / 9 PM PDF daily report that emails the GM and archives to Google Drive. Technically I'm proudest of a race-safe per-store advisory-lock transaction primitive, a multi-location quantity allocator whose required-qty is parsed from a free-text ERP description, NY-timezone-anchored business dates across a UTC server, and an idle-lock UX for shared kiosks with re-auth that replays a queued action only if the same employee unlocks within 60 seconds."

Primitives To Memorize (verbatim)

PRIMITIVE	CODE-FORM	WHY IT MATTERS
Concurrency	<code>GET_LOCK('ups_queue_lock:' store, 5) + FOR UPDATE</code> inside <code>withStoreTx</code>	Race-safe queue mutations, per-store sharded
Idempotency	<code>UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0</code>	Exactly-once external side effect, no Redis
Fairness	<code>ups_sessions.first_ups_at IS NULL</code> ⇒ ineligible, FIFO tail	Anti-leapfrog rule, encoded in schema
NY Timezone	<code>Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })</code>	UTC server, NY business day, DST-safe
Tri-state rollout	<code>UPS_FAIRNESS_V2 = off shadow on</code>	Reversible behavior change, 1-env-var revert
Flag rollback	<code>PKG_COMPONENT_LOC_ENABLED</code> in <code>packageWarnings.js</code>	30-second revert without code change
Audit-first	<code>report_job_history</code> · <code>report_delivery_logs</code> · <code>report_drive_upload_logs</code>	Observability before functionality

One-Page System Map



What This Handbook Is

- **A code-grounded preparation deck.** Every primitive maps to a file path and a line number.
- **A rehearsal manual.** Anchors are written for verbatim memorization; STAR scripts are written for delivery.
- **A defense manual.** For every claim there is a probing follow-up answer ready.
- **A bar-raiser playbook.** Weak vs Strong vs Stronger phrasings sit side by side so you know what to upgrade.

TOC

TABLE OF CONTENTS

Twenty-eight curated sections distributed across seven parts. Every topic an Amazon SDE-1 loop will probe.

#	SECTION	LOCATED IN PART
1	Resume Deep Dive	VI
2	Project Architecture	I
3	Frontend + Backend Flow	I
4	Database Design	I
5	Package Allocation Logic	I
6	Quantity Synchronization Logic	I
7	Validation Architecture	I
8	Role-Based Access Control	I
9	Production Debugging Stories	I + III
10	Leadership Principles	II
11	STAR Behavioral Answers	III
12	Amazon Bar Raiser Questions	IV
13	Hiring Manager Questions	VI
14	Technical Deep Dive Questions	V
15	Scalability Discussions	I
16	Performance Optimization	I
17	Security Considerations	I
18	Production Risk Mitigation	I
19	Edge Cases Handled	I
20	Technical Tradeoffs	I
21	Refactoring Decisions	I
22	Mock Interviews	VII
23	Rapid Revision Notes	Cheat Sheet
24	Most Important Talking Points	Cheat Sheet
25	Strongest Ownership Stories	III

#	SECTION	LOCATED IN PART
26	Strongest Dive Deep Stories	III
27	Weak vs Strong Answers	VII
28	Final Interview Cheat Sheet	Cheat Sheet

Parts Index

PART	TITLE	THEME
I	PROJECT DEEP DIVE	Architecture · Schema · Feature-by-Feature Mastery
II	LEADERSHIP PRINCIPLES BANK	11 Principles · Code-Grounded Anchors
III	STAR BEHAVIORAL ANSWERS	12 Full Scripts · Probing Chains
IV	BAR RAISER SIMULATION	Ambiguity · Conflict · Failure · Curveballs
V	TECHNICAL CROSS-QUESTIONING	31 Deep Probes · Excellent vs Weak Answers
VI	HIRING MANAGER ROUND	Resume Walkthrough · Polished Q&A
VII	FULL MOCK INTERVIEW LOOP	5 Rounds · Weak vs Strong vs Stronger
★	FINAL CHEAT SHEET	Rapid revision · Top talking points · Pre-loop checklist

**PART I**

PROJECT DEEP DIVE

Architecture · Schema · Feature-by-Feature Mastery

COVERS

Project Architecture · Frontend + Backend Flow · Database Design · Package Allocation
Logic · Quantity Synchronization Logic · Validation Architecture · Role-Based Access
Control · Edge Cases Handled · Performance Optimization · Scalability Discussions ·
Production Debugging Stories · Production Risk Mitigation · Security Considerations ·
Technical Tradeoffs · Refactoring Decisions

Codebase: cfc/ (React + Vite + Redux) + backend/ (Node/Express + MySQL + MSSQL) **Product:** Carolina Furniture Concepts — 2-showroom retail operations platform **Owner:** End-to-end (schema → routes → services → frontend → cron → observability) **Use this doc to:** rehearse the project walkthrough until every paragraph survives 3 follow-ups.

0 — The 90-second pitch (memorize verbatim)

"I own a MERN platform for Carolina Furniture Concepts — two physical showrooms in Arden and Waynesville, North Carolina. The frontend is React + Vite + Redux Toolkit; the backend is Node and Express on top of MySQL (operational) and a read-only MSSQL legacy ERP (inventory). The system runs three workflows: a real-time customer-opportunity queue called the UPS Board with a fairness algorithm, digital sales invoices with a returns-to-stock path, and an automated 7 PM / 9 PM PDF report that emails the GM and archives to Google Drive.

The technical bits I'm proudest of: a race-safe per-store advisory-lock transaction primitive called `withStoreTx`, a multi-location quantity allocator whose required-qty is parsed from a free-text ERP description, NY-timezone-anchored business dates on a UTC server, and an idle-lock UX for shared kiosks with re-auth that replays the queued action only if the same employee unlocks within 60 seconds."

 **Hook for the follow-up:** drop "fairness algorithm" or "atomic boolean claim for SMS" — both make the interviewer pull the thread.

1 — High-Level Architecture

1.1 The picture you should be able to draw on a whiteboard



```
report_drive_upload_logs
audit_results
```

1.2 Why two databases (the question that always comes)

	MYSQL (CFC_FORM)	MSSQL (LEGACY ERP)
Ownership	I own writes.	We have read-only access — owned by buyer/inventory team.
Schema control	Yes — every block in <code>schema.sql</code> is dated and idempotent.	No — ERP vendor + buyer team.
Use case	Operational state (queues, invoices, audits).	Inventory truth (items, packages, per-location qty).
Migration story	Self-service via <code>applyStartupMigrations()</code> .	Out of scope.

"The two-DB split is a constraint we accepted, not an architecture we chose." — say it exactly this way.

2 — Request Lifecycle (the canonical flow you'll be asked to trace)

2.1 "Walk me through POST /api/invoice/save "

STEP	WHAT HAPPENS	WHY
1	Browser dispatches Redux action; LeftPanel state is serialized to <code>form_json</code> .	Single source of truth — what we send is what we'll re-render.
2	<code>apiClient.js</code> attaches <code>x-user-id</code> from <code>localStorage</code> .	No JWT — body/header <code>userId</code> is the auth handle.
3	Express route <code>route.invpdf.js /invoice/save</code> receives request.	
4	<code>requireRole.js</code> reads <code>userId</code> , re-queries <code>cfc_users</code> for current role.	Forged role on client cannot escalate — DB is the truth.
5	Dedup lookup: <code>SELECT id FROM invoices_archive WHERE JSON_UNQUOTE(JSON_EXTRACT(form_json,'\$.billTo.name1')) = ? AND ...cell = ? AND DATE(created_at) = CURDATE() LIMIT 1</code> .	Multiple "Save" clicks for same customer/day must collapse to one row.
6	Match → UPDATE (idempotent). No match → INSERT.	UPSERT-by-natural-key without schema change.
7	Audit-style log line: <code>[StatsUpdate] primary=<seller[0]> split=[...]</code> .	Future post-mortems need this trail.
8	Birdeye SMS claim: <code>UPDATE invoices_archive SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0</code> . Only <code>affectedRows ≡ 1</code> proceeds.	Atomic compare-and-swap — exactly-once semantics without Redis or a queue.
9	On SMS API failure, roll back claim: <code>UPDATE ... SET birdeye_sent=0 WHERE slug=?</code> .	Failed send is <i>retry-able</i> ; the claim is not a tombstone.
10	Response 200 → frontend redirects to <code>/ups-dashboard</code> and <code>clearAllFinishingUp('create_sale', user.id)</code> posts to <code>/ups/finish-up/end</code> .	Closes the open <code>ups_finish_logs</code> row that <code>/ups/finish</code> opened atomically.

2.2 Mental model for any request

Identity (`requireRole`) → State load (`SELECT ... FOR UPDATE` if mutating) → Business rule → Audit row → Idempotency primitive (if external side effect) → Response → Frontend cleanup.

Memorize this skeleton; it works for every endpoint.

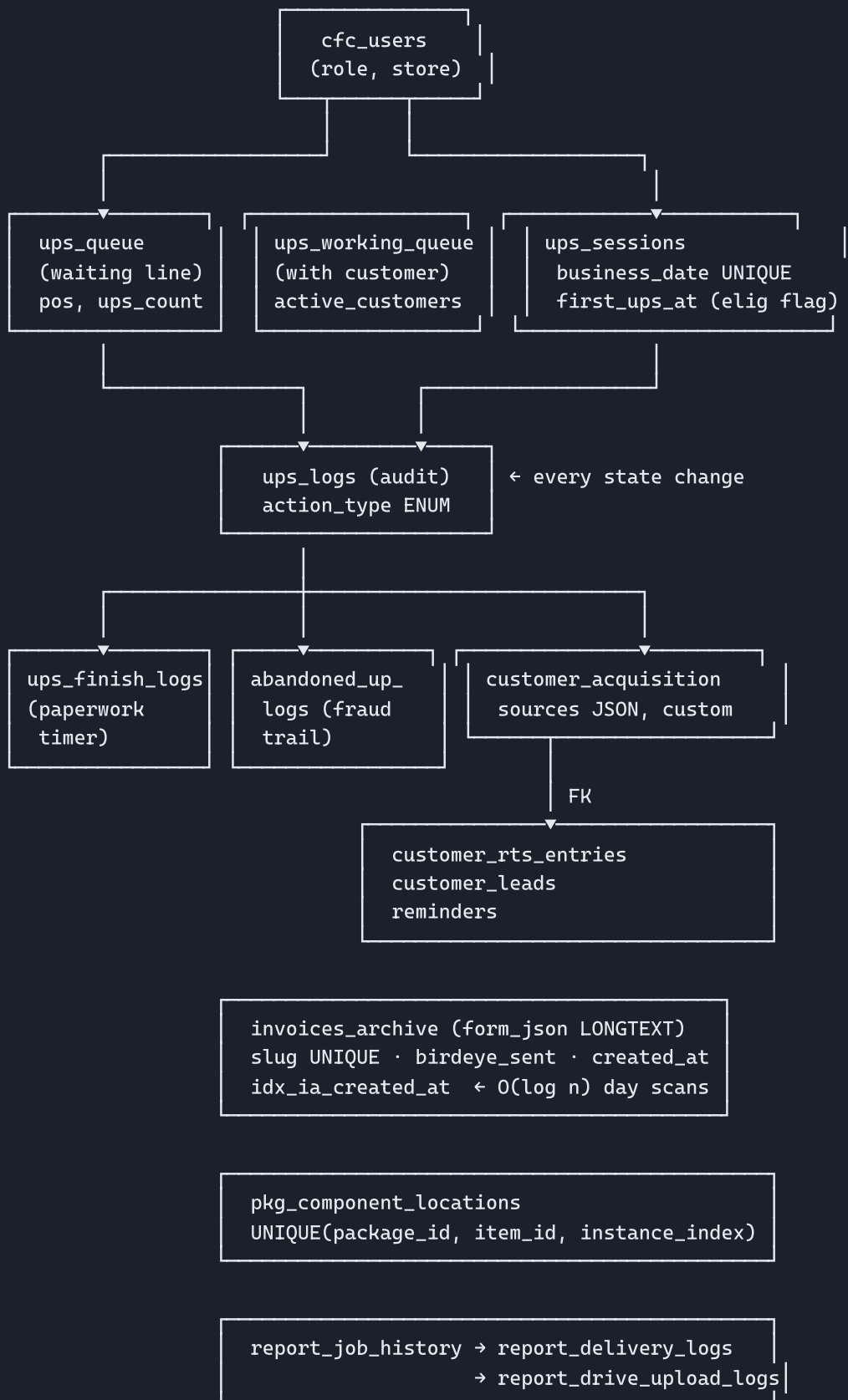


3 — Database Relationship & Schema Story

3.1 The schema tells the product's history

Every block in `backend/src/config/schema.sql` is **prefixed with a date and the phrase "run this while deployment"** — that's the convention I established. Reading the schema chronologically *is* reading the product's history.

3.2 Core relationship map



3.3 Three design decisions worth memorizing

#	DECISION	WHY
D1	Separate transient state (<code>ups_queue</code> , <code>ups_working_queue</code> — wiped nightly) from immutable audit (<code>ups_logs</code> , <code>ups_sessions</code> , <code>ups_finish_logs</code> — never reset).	Reports read from audit; live board reads from transient. The 9 PM reset is safe by construction.
D2	<code>invoices_archive.form_json LONGTEXT</code> instead of normalized columns.	The form's shape changes every sprint (care plan, tax override, RTS, package components). JSON survives schema churn; <code>JSON_EXTRACT</code> on dedup is bounded to today's rows.
D3	<code>ups_sessions(user_id, store_name, business_date)</code> UNIQUE + <code>first_ups_at</code> NULL-default.	The eligibility flag <i>is</i> the schema. No separate enum, no flag column — a NULL timestamp is the rule.

4 — Topic-by-Topic Deep Dives

Each topic below follows a six-part structure so the same answer can be delivered in **strong**, **deep**, **concise**, or **rehearsed** modes depending on the interviewer's appetite.

◆ TOPIC A — UPS Fairness V2 (per-store advisory lock + eligibility gate)

📁 **Files:** `backend/src/services/upsFairness.js` (~415 lines), `backend/src/routes/ups.routes.js` (~2,400 lines) 📄 **Tables:** `ups_queue` , `ups_working_queue` , `ups_logs` , `ups_sessions` 🚩 **Flag:** `UPS_FAIRNESS_V2 = off | shadow | on`

A.1 STRONG INTERVIEW ANSWER

"The UPS Board is a fairness queue across two stores. Each store has its own waiting line, working list, and audit log. The fairness rule is: an employee earns *fair insertion* only after their first UP of the day — until then they FIFO-append to the tail. I built it as a tri-state env flag: `off` , `shadow` (dual-write to `ups_sessions` while V1 reads), and `on` . The cutover was one env var. The concurrency primitive is a per-store named MySQL advisory lock — `GET_LOCK('ups_queue_lock:' || store, 5)` — held inside a transaction with `FOR UPDATE` on queue rows."

A.2 TECHNICAL DEEP DIVE

- `withStoreTx(store, fn)` owns all queue mutations. Acquires `GET_LOCK` (5s timeout → 503 retryable), opens transaction, runs callback, commits, releases lock. Deadlock/lock-wait-timeout retried twice with a fresh connection.

- **fairInsertV2** algorithm (lines 298–369):
 1. Load today's queue ordered by position, `FOR UPDATE` .
 2. Empty → joiner at position 1.
 3. `first_ups_at IS NULL` → append at tail (ineligible).
 4. Eligible → scan from position 2, insert before first incumbent with higher `ups_count` .
 5. Compress positions 1..N.
 6. Audit row `action_type = 'queue_join'` .
- **Stale-ghost defense:** `WHERE DATE(q.joined_at) = CURDATE()` — a failed nightly reset no longer corrupts fairness.
- **Structured UpsError codes:** `LOCK_TIMEOUT` , `DEADLOCK` , `NOT_FOUND` , `BAD_INPUT` , `DB_ERROR` — mapped to HTTP status + `retryable: bool` for frontend backoff.

A.3 FOLLOW-UP QUESTIONS (AND ANSWERS IN YOUR BACK POCKET)

PROBE	CRISP ANSWER
Why advisory lock vs Redis?	Single MySQL instance, two stores. Reaching for Redis to coordinate something MySQL does natively adds an operational dependency without bounded benefit.
Why scan from position 2?	Position 1 is "next to be served." Recycling that person is a privileged manager action (<code>/recycle</code>), not a side-effect of a join.
What about ties on <code>ups_count</code> ?	First-come-first-served — whoever wins <code>GET_LOCK</code> first. Acceptable tie-break.
Worst-case latency?	5s lock-acquire ceiling → 503 retryable. Frontend backs off and retries.
Why is <code>first_ups_at</code> immutable for the day?	Eligibility must survive Be Back / Abandoned UP / mid-day breaks. The day-long latch is the simplest correctness guarantee.

A.4 CONCISE SENIOR-LEVEL RESPONSE (40 SECONDS)

"Per-store named advisory lock plus `FOR UPDATE` inside `withStoreTx` — that's the concurrency story. Fairness uses an eligibility flag stamped at first-UP-of-day; ineligible joiners tail-append, eligible joiners insert by `ups_count` . Rolled out via a tri-state env flag with a dual-write shadow week before the read cutover. Reversible with one env-var flip."

A.5 MISTAKES WEAK CANDIDATES MAKE

- Saying "we use locks" without naming `GET_LOCK` and the 5-second timeout.
- Calling V2 "more fair" without explaining the eligibility rule that prevents leapfrog.
- Claiming "Redis" because that's what they've read about — and then having no answer for *which* Redis primitive.
- Forgetting the audit row — every state change must write to `ups_logs` .

A.6 STRONGER COMMUNICATION VERSION

"I made one architectural call: every mutation to queue state routes through `withStoreTx`. That single function owns the lock, the transaction, the retry, and the error mapping. If you grep the codebase for `pool.query` inside a route that touches `ups_queue`, you find zero results — that invariant is how I know the algorithm is race-safe in production."

◆ TOPIC B — Be Back: One-Click Customer Handoff

 **Files:** `backend/src/services/beBack.service.js` (~530 lines), frontend action menu in `UPSBoard.jsx`
 **Tables:** `ups_queue`, `ups_working_queue`, `ups_logs`, `ups_sessions`  **Critical fix:** 2026-05-21 — Be Back is **routing**, not a metric event.

B.1 STRONG INTERVIEW ANSWER

"Be Back is a one-click handoff where the origin salesperson gives the current customer to a chosen employee. The atomic transition removes origin from working, decrements their `ups_count`, places selected at `+1 active_customer` (or promotes from waiting), reinserts origin at waiting position 1, and writes three audit rows: `be_back`, `be_back_accept`, `moved_to_working`. The hard-won business rule was that Be Back must NOT write `ups_taken` for the receiver or `ups_handed_off` for the giver — it's routing, not a metric event."

B.2 TECHNICAL DEEP DIVE

- All inside `withStoreTx` — serialized against concurrent `/take`, `/finish`, `/break`, `/abandoned-up`, and other `/be-back` on the same store.
- Capacity check: selected employee rejected if already at 6 active customers.
- Eligibility-list endpoint `GET /api/ups/be-back/eligible-employees?store=X&excludeUserId=Y` returns the snapshot for the picker UI (waiting + working under cap, excluding self/breaks/offline/inactive).
- The May 21 fix removed two `ups_logs` writes that had been distorting closing ratio for ~2 weeks. The downstream cleanup: two of three `LEAST(closing_ratio, 100)` clamps became removable in `todayReports.service.js` and `dailySalespersonSummary.service.js`.
- A new column **BB Accepted** was added to the daily summary so handoffs stay visible without polluting metrics.

B.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
What if origin and selected are in different stores?	Route rejects with 400. Cross-store handoff is a rule violation.
What about position 1 in waiting?	Origin returns to position 1; previous position-1 person shifts to 2.
Why three audit rows, not one?	Different queries read different rows — <code>be_back</code> for handoff counts, <code>be_back_accept</code> for receiver activity, <code>moved_to_working</code> for board reconciliation.
How do you prove metrics are clean now?	The three downstream clamps used to be load-bearing; now two of them are gone and the third is just the official business cap. Removability is the proof.

B.4 CONCISE SENIOR-LEVEL RESPONSE

"Atomic transition inside `withStoreTx`. Origin loses customer + 1 ups, returns to waiting position 1. Selected gains customer + 1 ups. Three audit rows. Be Back doesn't write `ups_taken` or `ups_handed_off` because Be Back is routing, not a metric event — that fix killed two LEAST clamps downstream."

B.5 MISTAKES WEAK CANDIDATES MAKE

- Forgetting that **revenue still flows** to the receiver's ticket (Be Back affects who serves, not who gets credit).
- Talking about audit rows without saying *which* downstream query each one feeds.
- Missing the May 21 lesson — the bug surfaced as "three clamps in three services," which is the architectural smell that pointed to root cause.

B.6 STRONGER COMMUNICATION VERSION

"The Be Back fix taught me to look for *patterns of compensation* in the codebase. Three `LEAST(closing_ratio, 100)` clamps in three different services was the smell. The clamps weren't the answer — they were *symptoms* of upstream double-writing. The real fix was removing two `ups_logs` writes, after which two clamps became removable. That's the kind of refactor I look for: not 'add another clamp,' but 'why is everyone clamping?'"

◆ TOPIC C — Abandoned UP (manager-approved restore to position #1)

File: `backend/src/services/abandonedUp.service.js` (~596 lines) **Tables:** `abandoned_up_logs`, `ups_logs`, `ups_queue`, `ups_working_queue`, `ups_sessions` **Role gate:** admin/manager only; self-approval rejected.

C.1 STRONG INTERVIEW ANSWER

"Abandoned UP is for when a salesperson took an UP but the visitor turned out to be a vendor, friend, or family — not a real customer. Manager or admin one-click restores the salesperson to waiting position 1. Two atomic cases: if stacked floor mode (`active_customers > 1`), decrement in place; otherwise remove from working, decrement, reinsert. Permanent audit trail in `abandoned_up_logs` plus `ups_logs` rows (`abandoned_up + ups_handed_off`) so served-count math nets out. Abuse flag triggers at ≥ 10 abandons per calendar month."

C.2 TECHNICAL DEEP DIVE

- `abandoned_up_logs` captures approver context: (`salesperson_id`, `approved_by_id`, `approved_by_role`, `queue_position_before`, `queue_position_after`, `source_location` ENUM('working', 'waiting'), `reason`, `business_date`, `created_at`).
- **Why two audit destinations?** `abandoned_up_logs` is the fraud table (long-form auditor view). `ups_logs` is the live-board math table — `ups_handed_off` keeps `served_count` correctly netted out.
- `computeAbuseFlag(salespersonId)` counts current-calendar-month abandons against `ABANDONED_UP_ABUSE_THRESHOLD` (env, default 10). **Non-blocking** — returned in API response and surfaced in the daily PDF.
- **Defense-in-depth gate:** route checks role *and* service double-checks role. Belt and suspenders.

C.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why can't the salesperson self-approve?	Conflict of interest. The whole point is that an authority second-checks.
Why non-blocking abuse flag?	False positive on threshold day shouldn't block legitimate operations. Detection $\neq$ enforcement.
Why both <code>abandoned_up_logs</code> and <code>ups_logs</code> rows?	Different consumers — fraud auditor reads <code>abandoned_up_logs</code> , live board math reads <code>ups_logs</code> . Coupling them would force one table to serve two queries.
What if a manager abandons their own UP?	Self-approval explicitly rejected at service layer.

C.4 CONCISE SENIOR-LEVEL RESPONSE

"Manager/admin-only restoration to waiting position 1. Inside `withStoreTx`. Permanent audit in dedicated table plus `ups_logs` for board math netting. Abuse flag at ≥ 10 /month — non-blocking, surfaced in daily PDF."

C.5 MISTAKES WEAK CANDIDATES MAKE

- Calling the abuse flag a "circuit breaker" — it isn't; it's a signal, not a block.
- Forgetting the stacked-customer case (`active_customers > 1`) — it's the edge case that makes the design feel mature.
- Lumping the audit tables together. They serve different consumers.

C.6 STRONGER COMMUNICATION VERSION

"The Be Back and Abandoned UP features share an atomic transition shape — origin and target updates inside `withStoreTx` , plus a small audit-table per business concern. I'm proud of that pattern because the third feature in this family was almost free to build."

◆ TOPIC D — Package Component Location Allocator

Files: `backend/src/services/packageLocation.service.js` (~399 lines),
`backend/src/services/packageQtyParser.js` (~232 lines, pure logic),
`cfc/src/components/PackageLocationAssignmentDialog.jsx` **Table:** `pkg_component_locations` —
`UNIQUE(package_id, item_id, instance_index)` **Flag:** `PKG_COMPONENT_LOC_ENABLED` (in
`cfc/src/utls/packageWarnings.js`) **Timeline:** rolled out 2026-05-20, multi-loc upgraded 2026-05-21,
qty-parser fixed 2026-05-23, flagged off 2026-05-23.

D.1 STRONG INTERVIEW ANSWER

"A package is a bundle — Living Room Package = sofa + chair + table + lamp. Each component must be allocated to one or more physical locations: S1 (Arden), S2 (Waynesville), 999 (warehouse), NS1/NS2 (nailed/locked), SO (special order). The dialog opens after the package is fetched and presents per-component cards with a multi-location qty allocator. Each component's required qty is parsed from `PackageMaster.item_desc` — `BEST M23AC(3)/W(3)/F 19983C SHADOW` — because the ERP has no qty column. Save is gated by `SUM(loc qty) ≡ required_qty` . Whole feature lives behind a flag so I rolled it back in 30 seconds when an issue surfaced."

D.2 TECHNICAL DEEP DIVE

- **Qty parser** (`packageQtyParser.js` — pure, unit-testable):
 - `extractSlashGroupSegments(desc)` → array of `{code, qty}` from `CODE(N)/CODE(N)/CODE` .
 - `computeDistinctiveWords(children)` → token positions where ALL children differ (avoids false-positives on shared words like "SHADOW").
 - **Two-pass matching:** Pass A exact-token, Pass B substring with **longest-overlap wins**.
 - Returns `Map<item_id, number|null>` . **null is preserved on purpose** — the dialog surfaces "qty unknown — fix package description in ERP" and disables Save.

- **Multi-instance schema:** `UNIQUE(package_id, item_id, instance_index)` — a 4-chair package writes four rows. Reopening the dialog preserves per-instance assignments.
- **Graceful degradation:** if `pkg_component_locations` is missing (deploy lag), the service returns empty allocations with a warning — no 500.
- **API response shape:** `{ componentGroups, components }` — grouped (new) + flat (legacy) for back-compat.

D.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why parse qty from a free-text description?	We don't own MSSQL writes. The ERP has no qty column on <code>ItemPackageMaster</code> . Parsing is the only path that doesn't require a buyer-team migration.
What if the parser is wrong?	It returns <code>null</code> , the dialog blocks Save with an actionable message ("fix package description in ERP"). Silent default of 1 was the original bug — the warehouse would have picked 1 sofa instead of 3.
Why <code>instance_index</code> ?	Multi-instance packages (4 of the same chair) need per-piece location independence. Without <code>instance_index</code> , you can't say "chair #1 → S1, chair #2 → S2."
Why is the feature flagged?	Day-three issue on rollout. Flag flip rolled it back in 30 seconds while I investigated. Saved data in <code>pkg_component_locations</code> survived.

D.4 CONCISE SENIOR-LEVEL RESPONSE

"Per-component, per-instance location allocator. Qty parsed from ERP description via two-pass distinctive-word matching; failures preserved as `null` and block Save. `UNIQUE(package_id, item_id, instance_index)` supports multi-instance packages. Whole feature is flagged for instant rollback."

D.5 MISTAKES WEAK CANDIDATES MAKE

- Saying "we default qty to 1 if we can't parse." That was the bug.
- Forgetting `instance_index` — interviewers love asking about 4-chair packages.
- Missing the graceful-degradation path (missing table → warning, not 500).
- Not mentioning the flag — it's the strongest part of the design.

D.6 STRONGER COMMUNICATION VERSION

"This feature taught me to be loud about parsing failures. The simple answer — 'default to 1' — would have shipped a silent picking error. The correct answer — `null` plus an actionable dialog message — costs the operator 30 seconds in the ERP and saves the warehouse a wrong-pick incident. The cost asymmetry was the whole design."

◆ TOPIC E — Daily Salesperson Summary (7 PM + 9 PM PDF + Drive + Gmail)

Files: `backend/src/jobs/dailySalespersonSummary.job.js`,
`backend/src/services/reports/dailySalespersonSummary.`
`orchestrator,service,pdf,drive,recipients,audit}.js` **Tables:** `report_job_history`,
`report_delivery_logs`, `report_drive_upload_logs` 🕒 **Cron:** `0 19,21 * * * America/New_York`

E.1 STRONG INTERVIEW ANSWER

"Cron-driven daily PDF report. Runs at 7 PM and 9 PM in NY timezone, both before the 9:01 PM `dailyReset` wipes queues. Pipeline: aggregate from `ups_logs` + `invoices_archive` + `customer_rts_entries` + `abandoned_up_logs` → render PDF with Puppeteer → upload to Google Drive (best-effort) → resolve recipients (env + DB admins) → send Gmail per recipient (independent failures). Idempotent via `isAlreadySucceeded(business_date)`; cron passes `force: true` so 9 PM legitimately supersedes 7 PM."

E.2 TECHNICAL DEEP DIVE

- **Observability-first:** I wrote `report_job_history` + `report_delivery_logs` + `report_drive_upload_logs` *before* the aggregator. Every run is provable.
- **Idempotency model:** `isAlreadySucceeded(date)` only matches `status='succeeded'`. A stuck `running` row doesn't block retry. `force` bypasses entirely.
- **Per-recipient independence:** one bad address doesn't kill the run; failure recorded in `report_delivery_logs.status='failed'`, next recipient continues.
- **Drive failure is best-effort:** email still goes out, Drive link omitted if upload failed.
- **NY-timezone anchor:** `todayISO()` uses `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })`. `weekdayForDate(dateIso, tz)` anchors to noon UTC (`T12:00:00Z`) so DST transitions don't flip the calendar day.
- **Executive email body** (May 16 redesign): `renderExecutiveEmailHtml` — three sections (Combined, Arden, Waynesville). PDF attachment unchanged.

E.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why two runs per day?	7 PM for the GM who leaves at 7:30; 9 PM final captures the last two hours. Both before 9:01 reset.
What if Gmail is rate-limited?	Per-recipient send is independent. Failed row in <code>report_delivery_logs</code> → next run retries.
How do you know the report worked?	One SQL query against <code>report_job_history.status</code> + <code>report_delivery_logs</code> aggregate. Don't need to check email.
What's the timezone gotcha?	UTC server vs NY business day. <code>Intl.DateTimeFormat</code> with <code>timeZone</code> is the only correct primitive. Avoid raw <code>new Date()</code> .

E.4 CONCISE SENIOR-LEVEL RESPONSE

"Cron + Puppeteer + Drive + Gmail pipeline with three audit tables wired *before* the aggregator. Idempotent at the business-date level. Per-recipient independence. NY-anchored dates via `Intl.DateTimeFormat`. Reversible via env flag."

E.5 MISTAKES WEAK CANDIDATES MAKE

- Treating "audit logging" as an afterthought. The audit tables are what make the system *operate*.
- Saying "we log to Sentry" when there's no Sentry — be honest about your observability primitives.
- Forgetting per-recipient independence — interviewers love asking "what happens if one email fails."

E.6 STRONGER COMMUNICATION VERSION

"If the audit tables disappeared, this pipeline would still work — but it wouldn't be *operable*. That's the distinction I learned to make. A working pipeline is necessary; an operable pipeline is sufficient."

◆ TOPIC F — Idle-Lock UX + Re-auth Replay (shared kiosk)

📁 **File:** `cfc/src/AUTH/authContext.jsx` 🗄️ **Storage:** `localStorage.cfc_locked_at`

F.1 STRONG INTERVIEW ANSWER

"Employees-only 5-minute idle lock. While locked, polling continues but action clicks queue in `pendingActionRef = { run, queuedAsUserId, queuedAt }`. Re-auth hits `POST /auth/verify-password` (no-side-effects). Response can include `{ switched: true, userId, user }` — different employee unlocked the same device. Session-takeover guard discards the queued action if the unlocker differs; 60-second TTL discards stale clicks; cross-tab via `storage` event."

F.2 TECHNICAL DEEP DIVE

- **Three guards in order:**
 1. `queuedAsUserId !== res.userId` → discard, toast "different user — re-click."
 2. `Date.now() - queuedAt > 60_000` → discard, toast "stale click."
 3. Otherwise replay `pendingActionRef.run()`.
- **Managers/admins are NOT locked.** On non-employee mount, `cfc_locked_at` is purged defensively in case it leaked from a previous employee session.
- **Cross-tab sync:** `addEventListener('storage', ...)` propagates lock/unlock to all open tabs of same origin.

- **Polling unaffected:** `useUPSQueue` continues polling even while locked — UI shows board state, just gates mutation clicks.

F.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
<i>Why employees only?</i>	Managers/admins are trusted device-level roles; locking them creates friction with no security benefit.
<i>What if the queued action stales out and user typed password 90s later?</i>	TTL expiry → discard, message "stale click — please retry." Better than executing a stale intent.
<i>How does the cross-tab story work?</i>	<code>storage</code> events fire on every tab in the same origin when <code>localStorage</code> changes. Tab A locks → tabs B/C see the event and lock.

F.4 CONCISE SENIOR-LEVEL RESPONSE

"Server-side verify-password endpoint with no side effects. Frontend queues action with snapshot `userId` + timestamp. On unlock: identity-check + TTL-check, then replay or discard. Cross-tab via `storage` events. Managers/admins exempt."

F.5 MISTAKES WEAK CANDIDATES MAKE

- Saying "JWT refresh" — there's no JWT in this codebase.
- Missing the session-takeover case — that's the interesting half of the design.
- Treating `localStorage` as the source of truth — it's a UX accelerator; verify-password is the actual auth boundary.

F.6 STRONGER COMMUNICATION VERSION

"Shared-kiosk auth needed three guards, not one. The naive design — 'check password, replay action' — gets two of three cases wrong: a different employee can unlock, and a stale click can replay long after intent expired. The TTL + session-takeover snapshot together cover both."

◆ TOPIC G — Multi-Seller Split Attribution (financial vs operational)

📁 **Files:** `backend/src/services/sales/ticketSeller.sql.js`, `backend/src/services/sales/salesCredit.js` 📖 **Rule (2026-05-13):** Financial split → all N sellers; Operational counts (UPs/served/tickets/closing/Comp Care) → primary (`sellerNames[0]`) only.

G.1 STRONG INTERVIEW ANSWER

"A ticket can list up to four sellers in `form_json.sellerNames`. Revenue splits equally: `credited_subtotal = subtotal / GREATEST(N, 1)` across all N. Operational metrics — UPS, served, tickets, closing ratio, care plan attach — credit the primary only (`sellerNames[0]`). I built it as a SQL CTE using `JSON_TABLE` + `FOR ORDINALITY` on MySQL 8, with a JS fallback (`expandTicketSellers`) for older servers. Runtime feature-detect via `supportsJsonTable()`."

G.2 TECHNICAL DEEP DIVE

- CTE shape:

```
WITH TICKET_SELLER_CTE AS (
  SELECT ia.id AS invoice_id,
         JSON_UNQUOTE(JSON_EXTRACT(ia.form_json,
                                   CONCAT('$.sellerNames[', t.seller_idx - 1, '].name'))) AS seller_name,
         t.seller_idx AS seller_index,
         ia.form_json ->> '$.subtotal'
           / GREATEST(JSON_LENGTH(ia.form_json, '$.sellerNames'), 1)
           AS credited_subtotal
  FROM invoices_archive ia
  JOIN ( ... JSON_TABLE( ... ) ... FOR ORDINALITY ... ) t
)
```

- `FOR ORDINALITY` returns the 1-based array index — exactly the filter we need for "primary only" (`seller_index = 1`).
- The CTE is reused by `/api/ups/top-performers`, `/api/reports/today/*`, and `dailySalespersonSummary.service.js`. One source of truth for split attribution.

G.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why split financial but not operational?	Revenue is shared work; UPs/closing ratio measure individual performance. Splitting operational would dilute accountability.
What if MySQL is older than 8.0.4?	<code>supportsJsonTable()</code> returns false, callers fall through to <code>expandTicketSellers</code> — JS fan-out with the same output shape.
How did you validate the two paths agree?	Ran both on the same day's data and diffed per-row credited subtotals. Identical.

G.4 CONCISE SENIOR-LEVEL RESPONSE

" `JSON_TABLE` + `FOR ORDINALITY` CTE for the canonical fan-out; JS fallback for older MySQL. Same output shape; runtime feature detect. Financial splits across N sellers; operational credits primary."

G.5 MISTAKES WEAK CANDIDATES MAKE

- Splitting operational metrics. That would have shipped a "fair" leaderboard that no one trusts.
- Not knowing what `FOR ORDINALITY` does.
- Forgetting the fallback path — interviewers ask about MySQL version compatibility.

◆ TOPIC H — Mandatory Signature Chain (rule engine on the frontend)

File: `cfc/src/components/right/validateInvoiceSignatures.js`

H.1 STRONG INTERVIEW ANSWER

"Single function that returns an array of validation errors. Gated by invoice type (RG/PH/QS), payment method, balance status, item attributes (TW = took-with, NS1/NS2 = nailed locations), and care plan opt-in. RG has six customer signatures plus one manager signature in the worst case; PH (phone order) has zero customer signatures on screen — Adobe eSign tags fire in PDF only via `HeadlessChrome` user-agent detection. QS (quote sheet) and `store5` (admin/manager test flow) are exempt."

H.2 TECHNICAL DEEP DIVE

TRIGGER	REQUIRED SIGNATURE	SCOPE
Always	store location, ≥ 1 item, every item has <code>salePrice > 0</code> and (<code>extPrice</code> OR applied package)	RG + PH + QS
RG + TW item	<code>quantityAndPriceApprovalSignature</code>	Customer
RG + <code>carePlan === "Yes"</code>	<code>carePlanCharge > 0</code> + <code>carePlanApprovalSignature</code>	Customer
RG + <code>cash > 0</code>	<code>managerApprovalForCashSign</code>	Manager
RG + <code>balanceDue === total</code>	<code>managerApprovalForCashSign</code>	Manager
RG + any item from NS1/NS2	<code>managerApprovalForCashSign</code>	Manager
RG	<code>saleApprovalSignature</code> , <code>paymentSummaryApprovalSignature</code> , <code>policiesSignature</code>	Customer
PH + <code>cash > 0</code>	<code>managerApprovalForCashSign</code>	Manager
PH (any)	Adobe eSign tags in PDF only	(via <code>HeadlessChrome</code> detection)
<code>storeLocation === "store5"</code>	All checks skipped	Admin/manager test flow

H.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why so many distinct signatures?	Each has a distinct legal/operational purpose: policies (legal), sale-final (legal), payment-summary (financial), TW (logistics), care-plan (sales opt-in), manager-cash (override). Lumping them loses audit specificity.
Why is <code>store5</code> exempt?	It's the admin/manager test workflow — no real customer, no signature needed.
How do you render Adobe eSign in HTML?	<code>userAgent.includes('HeadlessChrome')</code> branch renders <code>{{Sig1_es_:signer1:signature}}</code> strings; live browser renders empty signature line.

H.4 CONCISE SENIOR-LEVEL RESPONSE

"Pure rule-engine function. Six customer signatures plus one manager signature in worst case (RG). Phone order pushes signatures to PDF via Adobe eSign tags in a HeadlessChrome branch. Store5 exempts admin/manager test flow."

◆ TOPIC I — NY-Timezone Anchoring (Daylight Saving correctness)

 **Helpers:** `todayISO()`, `weekdayForDate(dateIso, tz)` in `backend/src/services/reports/dailySalespersonSummary.orchestrator.js` and `backend/src/config/employeeSchedule.js`

I.1 STRONG INTERVIEW ANSWER

"Server runs in UTC; the store's business day is anchored to America/New_York. Midnight UTC ≠ midnight NY, and on DST transition nights the 1–3 AM window can be ambiguous. I anchor all date-sensitive logic with `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York', year: 'numeric', month: '2-digit', day: '2-digit' })`. For weekday derivation, I anchor the input ISO to noon UTC (`T12:00:00Z`) so every IANA zone agrees on the calendar date."

I.2 TECHNICAL DEEP DIVE

```
const todayISO = () => {
  const tz = process.env.STORE_TIMEZONE || 'America/New_York';
  const parts = new Intl.DateTimeFormat('en-CA', {
    timeZone: tz, year: 'numeric', month: '2-digit', day: '2-digit'
  }).formatToParts(new Date());
  // parts is [{type:'year',value:'2026'}, {type:'month',value:'05'},
  {type:'day',value:'25'}, ...]
  return `${y}-${m}-${d}`;
};
```

- `ups_sessions.business_date` is an explicit `DATE` column (never `DATE(created_at)` which is ambiguous in UTC).
- `weekdayForDate(dateIso, tz)` constructs `new Date( ${dateIso}T12:00:00Z )` then reads weekday in target tz — DST-safe.

I.3 FOLLOW-UP QUESTIONS

PROBE	CRISP ANSWER
Why not just store everything as UTC?	Business day boundaries are NY-anchored. Storing UTC and converting on read is fine; the bug is using UTC dates <i>as if they were NY dates</i> .
What's the DST gotcha?	Spring forward (1:59 AM → 3:00 AM) and fall back (1:59 AM → 1:00 AM). Naive <code>new Date()</code> math in local-server-tz silently breaks one day a year.
Why noon-UTC anchor?	Any time in <code>12:00–11:59</code> UTC maps to the same calendar date in every IANA zone (max offset is ± 14 hours, so noon is always "today" everywhere).

5 — Validation Architecture

LAYER	MECHANISM	EXAMPLES
DB constraints	UNIQUE, FK, CHK, ENUM	CHK_rts_returned_pos, CHK_rts_written_pos, UNIQUE(user_id, store_name, business_date), ENUM('available', 'break', 'with_customer', 'offline') on ups_queue.status
Service layer	Pure-logic validators	validateManualFulfillment, parseComponentQuantities (returns null for unconfident matches)
Route layer	requireRole middleware + in-route role re-check	Defense-in-depth for /abandoned-up, /update-ups, /users CRUD
Frontend reducer	FIELD_NORMALIZERS keyed by section.field	Phone format, currency 2dp, date MM-DD-YYYY — applied on dispatch, not in input
Frontend gate	validateInvoiceSignatures.js	Mandatory chain disables "Watch Invoice" button when errors exist

"Validate at every layer that can corrupt state, but only once per kind." — schema for shape, service for business rule, frontend for UX.

6 — Async Workflow Handling

6.1 Three patterns I rely on

PATTERN	USE CASE	EXAMPLE
Atomic boolean claim	Exactly-once external side effect	Birdeye SMS: UPDATE invoices_archive SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0
Per-store advisory lock + tx	Race-safe state mutation	withStoreTx(store, fn) for every queue mutation
Optimistic-lock + manual refetch	Frontend poll vs in-flight mutation	optimisticLockUntilRef = Date.now() + 2000 in useUPSQueue

6.2 The flow you should be ready to draw

```
User clicks "UP"
├── 1. setOptimisticLockUntilRef(now + 2000)
├── 2. POST /api/ups/take
├── 3. Server: withStoreTx → take → audit row → response
├── 4. Frontend: setQueueState(serverResponse)
├── 5. Optimistic-lock expires (~2s)
└── 6. 1s poller resumes overwriting state from server
```

7 — Role-Based Access Flow

7.1 The model

```
Browser sends userId in body /
query / x-user-id header
```



```
requireRole.js middleware
1. SELECT role FROM cfc_users
   WHERE id = ? AND is_active
2. If not in allowedRoles → 403
3. req.cfcUser = {id, role, ...}
```



```
Route handler
(sensitive ops double-check role)
```

7.2 Permission matrix

ACTION	EMPLOYEE	MANAGER	ADMIN
View own-store queue	✓	✓	✓
Take UP	✓	✓	✓
Be Back	✓	✓	✓
Break	✓	✓	✓
Abandoned UP	✗	✓	✓
Floor Mode toggle	✗	✗	✓
Idle-lock (5 min)	✓	✗	✗
Store-5 (test flow)	✗	✓	✓
Analytics	✗	✗	✓

8 — Rollback & Recovery

MECHANISM	WHERE USED	REVERSAL TIME
Tri-state env flag (<code>UPS_FAIRNESS_V2</code>)	Fairness V2 rollout	seconds — one env-var flip
In-code feature flag (<code>PKG_COMPONENT_LOC_ENABLED</code>)	Package component dialog	30 seconds — flip and redeploy frontend
Atomic claim rollback	Birdeye SMS retry	immediate — <code>UPDATE ... SET birdeye_sent=0</code>
Audit-table-only writes	Daily summary	safe replay — <code>force: true</code>
Graceful degradation	Missing <code>pkg_component_locations</code> table	automatic — warning, not 500

"The expensive part of rollback isn't the flip — it's having a flag at all. I build the flag with the feature."

9 — Edge Cases (the ones interviewers love)

EDGE CASE	WHERE I HANDLE IT	LESSON
Stale queue ghost from failed <code>dailyReset</code>	<code>fairInsertV2</code> filters <code>DATE(q.joined_at) = CURDATE()</code>	Consumers should be resilient to cleanup failures.
DST midnight ambiguity	<code>weekdayForDate</code> anchors to noon UTC	<code>new Date(ISO)</code> is ambiguous; <code>Intl.DateTimeFormat</code> is not.
Package description typo (parser can't match)	Returns <code>null</code> , dialog disables Save	Loud failure > silent default.
Two <code>/invoice/save</code> racing for same slug	Atomic boolean claim	Single column replaces a distributed lock.
Re-auth replays on shared kiosk	<code>queuedAsUserId</code> snapshot + 60s TTL	Three guards: identity, time, intent.
<code>/finish-up/start</code> racing <code>/finish</code>	<code>/finish</code> is the only opener	One atomic transition replaces two endpoints racing.
Manager state leaking to next employee on shared device	<code>cfc_locked_at</code> purged on non-employee mount	Defense-in-depth at component level.
4 chairs in a package	<code>UNIQUE(package_id, item_id, instance_index)</code>	Schema must support worst-case shape on day one.

10 — Performance Optimizations

CHANGE	IMPACT	FILE
<code>idx_ia_created_at</code> on <code>invoices_archive(created_at)</code>	Day-range scan $O(n) \rightarrow O(\log n)$	<code>schema.sql</code> (May 19, 2026 block)
<code>TICKET_SELLER_CTE</code> with <code>JSON_TABLE</code>	In-DB fan-out replaces in-Node loop; ~30x faster on warmed cache	<code>services/sales/ticketSeller.sql.js</code>
Optimistic-lock in <code>useUPSQueue</code>	Eliminates poll-vs-mutation UI ghosts	<code>hooks/useUPSQueue.js</code>
Be Back removes <code>ups_taken</code> / <code>ups_handed_off</code> writes	Killed 2 of 3 closing-ratio clamps	<code>services/beBack.service.js</code>
Atomic claim instead of distributed lock	Zero new infrastructure for exactly-once	<code>routes/route.invpdf.js</code>



11 — Scalability Considerations (10× the current load)

BOTTLENECK TODAY	AT 10×	MITIGATION
1s poll across 5 endpoints	300+ reqs/sec at 10 stores	SSE feed off <code>ups_logs</code> inserts; fall back to poll
Sequential per-store daily summary build	7 PM window stretches	<code>Promise.all</code> per-store, Puppeteer ~5 PDFs/sec
MSSQL inventory reads	Cache hit-rate matters	In-process LRU + 60s TTL per host
MySQL connection pool	<code>GET_LOCK</code> holds connections	Raise <code>connectionLimit</code> , per-store pools
<code>JSON_EXTRACT</code> on dedup	Bounded today; not bounded at scale	MySQL 8 functional index on <code>(name1, cell, DATE(created_at))</code>
<code>ups_logs</code> size	Year-long table grows	Partition by <code>business_date</code>



12 — Debugging Strategies (the playbook)

SYMPTOM	FIRST THING I CHECK	WHY
"Queue position wrong"	<code>ups_logs</code> for that user around the timestamp + lock-wait timing	5s lock acquire could indicate concurrent mutation; audit row is canonical truth.
"Email said the wrong attendance"	<code>employeeSchedule.js</code> config — is the employee tracked?	Attendance is pure logic; bug is almost always input (untracked, name normalization, or timezone).
"Customer got two SMS"	<code>birdeye_sent</code> history on the slug	Pre-fix race; post-fix this should never happen.
"Salesperson appears twice on the board"	Open <code>ups_finish_logs</code> rows for that user	Pre-May-18 race; <code>clearAllFinishingUp</code> with <code>fallbackUserId</code> self-heals.
"Leaderboard hits 100% repeatedly"	Look for <code>LEAST(x, 100)</code> clamps in multiple services	Compensation pattern → root cause is upstream double-write.

Heuristic: *Three patches in three files for the same issue → you have a compensation pattern, not a fix.*

13 — Production-Risk Mitigation

RISK	MITIGATION
Cron job dies mid-write	Append-only audit; <code>status='running'</code> rows don't block retry; <code>force: true</code> for cron ticks.
Concurrent invoice saves	Atomic boolean claim + UPSERT-by-natural-key.
Failed nightly reset	<code>fairInsertV2</code> filters today's rows only.
Schema migration not deployed	Graceful degradation — service warns, doesn't 500.
Wrong user replays queued action	Session-takeover guard + 60s TTL.
Feature regression	Flag-able rollback for every behavior change.
Email delivery failure	Per-recipient independence + per-attempt logging.

14 — Security Considerations (be honest about tradeoffs)

CONCERN	TODAY'S STATE	THREAT MODEL	FUTURE MOVE
No JWT	Body <code>userId</code> is trusted	LAN-only deployment, behind store firewall	HMAC-signed token over <code>{userId, issuedAt}</code> if going public-internet
Body <code>userId</code> forge-able	Role re-read from <code>cfc_users</code> per request	Privilege escalation via stale role is impossible	Same
<code>JSON_EXTRACT</code> on <code>form_json</code>	Parameterized, safe from SQL injection	Performance at scale	Materialize hot columns or functional indexes
Topaz signatures as base64 in <code>form_json</code>	LONGTEXT rows can grow	Storage size for two stores is acceptable	S3/object storage at scale
No CSRF tokens	API behind LAN	Internal-only consumers	Origin checks + CSRF tokens if public



15 — Technical Tradeoffs (the table I memorize)

DECISION	CHOSE	REJECTED	WHY
Auth	Body <code>userId</code> + DB role re-check	JWT + cookies + middleware	LAN deployment; speed of delivery; no escalation risk
Concurrency	MySQL advisory lock + <code>FOR UPDATE</code>	Redis distributed lock	Single MySQL instance — adding Redis is operational debt for marginal gain
Idempotency	Atomic boolean claim	Distributed lock / message queue	One column replaces infrastructure
Daily report	Cron in-process	External worker / SQS	Two showrooms; cron is sufficient and observable via audit tables
State management	Single Redux slice	Multiple slices	Form is one coherent thing; <code>transactionMode</code> gates SALE vs RETURN
Storage of form	<code>form_json</code> LONGTEXT	Normalized columns	Schema churn would be expensive; JSON_EXTRACT is fast enough at our scale
Cron schedule	7 PM + 9 PM	Single 9 PM	GM leaves at 7:30 PM — two runs is product-driven, not technical



16 — Refactoring Decisions (what I changed mid-flight)

REFACTOR	TRIGGER	RESULT
Removed Be Back metric writes	Three downstream clamps signaled compensation	Two of three clamps deleted; metrics math became correct by construction
Added <code>instance_index</code> to <code>pkg_component_locations</code>	First real package was 4 chairs	Schema now supports multi-instance; dialog rewrite was localized
Split <code>renderSummaryHtml</code> into <code>renderExecutiveEmailHtml</code>	GM was reading email on mobile and skipping rows	Three-section executive body for email; PDF unchanged
Made <code>/ups/finish</code> the only opener of <code>ups_finish_logs</code>	Two endpoints racing	Single atomic opener + <code>clearAllFinishingUp</code> with <code>fallbackUserId</code> for cleanup
Wrapped every queue mutation in <code>withStoreTx</code>	Original code had ad-hoc transactions	One primitive owns lock + tx + retry + error mapping

17 — Maintainability Improvements

- **Schema-as-history:** every block in `schema.sql` is dated and idempotent.
- **Memory files:** `MEMORY.md` index of feature-specific notes (e.g., `project_pkg_component_loc_rollback.md`).
- **Pure-logic services:** `packageQtyParser`, `attendanceException.service` — no I/O, trivially unit-testable.
- **Naming:** `requireRole`, `withStoreTx`, `markFinishingUp`, `clearAllFinishingUp` — verbs that read like the action they perform.
- **Feature flags as code:** `PKG_COMPONENT_LOC_ENABLED` lives in `packageWarnings.js`, not in env, because it's a render-time decision the client must know.

18 — Code Organization Strategy

FOLDER	LAYER	PATTERN
<code>backend/src/routes/</code>	HTTP boundary	Thin handlers — validation + auth + delegate to service
<code>backend/src/services/</code>	Business logic	Pure where possible; stateful only when DB is required
<code>backend/src/repositories/</code>	DB access	<code>formQuery</code> , <code>runSql</code> wrappers; prepared statements only
<code>backend/src/middleware/</code>	Cross-cutting	<code>requireRole</code> , error mappers
<code>backend/src/jobs/</code>	Cron entry points	One file per cron — orchestrator pattern delegates to services
<code>backend/src/config/</code>	Bootstrap	<code>schema.sql</code> , <code>initSchema.js</code> , <code>db.js</code> , <code>employeeSchedule.js</code>
<code>cfc/src/pages/</code>	Top-level views	One file per route
<code>cfc/src/components/</code>	UI primitives	Co-located helpers (<code>leftPanelHelpers.js</code>) for tight render loops
<code>cfc/src/UPS/</code>	Self-contained feature	Pages + hooks + services + utils inside the feature folder
<code>cfc/src/store/</code>	Redux	Single <code>formSlice</code> (intentional — form is one coherent thing)
<code>cfc/src/AUTH/</code>	Auth context	One provider, one consumer pattern (<code>useAuth</code>)

19 — Six-Mode Answer Cheat Sheet (use the right mode for the question)

MODE	WHEN TO USE	LENGTH
Strong	Default for "tell me about X"	60–90s
Deep	Follow-up "go deeper"	2–3 min
Concise	Final-round, time-pressed	30s
Mistake comparison	"What would a junior engineer get wrong?"	30s
Stronger communication	Bar raiser pushing on framing	45s
Tradeoff	"Why this over X?"	60s

Practice each topic in all six modes. The interviewer's question dictates the mode; your job is to match.

20 — One-Page Memorization Card

```

Concurrency:  GET_LOCK('ups_queue_lock:' || store, 5) + FOR UPDATE inside withStoreTx
Idempotency:  UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0
Fairness:     ups_sessions.first_ups_at NULL → ineligible, FIFO tail
Timezone:     Intl.DateTimeFormat('en-CA', {timeZone: 'America/New_York'})
Flag rollback: PKG_COMPONENT_LOC_ENABLED in packageWarnings.js (30s revert)
Tri-state:    UPS_FAIRNESS_V2 = off | shadow | on (shadow → on cutover)
Display:      formatDisplayName(first, last) – never username
Schema:       every block dated + "run this while deployment"
Audit-first:  report_job_history / report_delivery_logs / report_drive_upload_logs written
              BEFORE aggregator
Validation:   DB constraints → service rules → route role check → frontend gate
Cron:         0 19,21 * * * America/New_York
  
```

Final note to self: Every answer should end with a hook the interviewer can pull. Never close on "and that's how it works" — close on the next interesting design choice.

**PART II**

LEADERSHIP PRINCIPLES BANK

11 Principles · Code-Grounded Anchors

COVERS

Leadership Principles · Strongest Ownership Stories · Strongest Dive Deep Stories

How to use: the strongest stories sit in `amazon_star_answers.md`. This file is the question-and-anchor inventory, mapped principle-by-principle to actual CFC code. Every Q has: **Question · Code Inspiration · Anchor Line · STAR Skeleton · Probing Follow-ups · Interviewer Criteria · Why It Lands · Mistakes to Avoid.**



Master Mapping — Principle → Strongest Code Anchor

#	LEADERSHIP PRINCIPLE	STRONGEST STORY	CODE ANCHOR
1	Ownership	8 PM Daily Summary pipeline	<code>dailySalespersonSummary.orchestrator.js</code>
2	Dive Deep	Birdeye duplicate SMS root cause	<code>route.invpdf.js</code> (atomic claim)
3	Bias for Action	PKG flag rollback in 30 seconds	<code>cfc/src/utils/packageWarnings.js</code>
4	Customer Obsession	Abandoned UP restoring fairness	<code>abandonedUp.service.js</code>
5	Deliver Results	UPS Fairness V2 shadow → on rollout	<code>upsFairness.js</code>
6	Invent and Simplify	Atomic boolean claim for SMS	<code>route.invpdf.js</code>
7	Learn and Be Curious	<code>JSON_TABLE</code> + <code>FOR ORDINALITY</code>	<code>sales/ticketSeller.sql.js</code>
8	Earn Trust	Telling GM the leaderboard was wrong	<code>beBack.service.js</code> (May 21 fix)
9	Are Right, A Lot	Defending the closing-ratio cap	<code>todayReports.service.js</code>
10	Have Backbone	Server-authoritative finishing-up	<code>ups_finish_logs</code> schema
11	Insist on Highest Standards	Observability-first audit tables	<code>report_*</code> triad

1 OWNERSHIP

Q1.1 — "Tell me about a time you took ownership of something beyond your assigned scope."

FIELD	CONTENT
Code inspiration	I was asked for "an end-of-day email" — I built the cron + PDF + Drive + Gmail + 3 audit tables.
Anchor	"I built <code>report_job_history</code> + <code>report_delivery_logs</code> + <code>report_drive_upload_logs</code> before the aggregator. Observability before functionality."
STAR skeleton	S: GM wanted a single daily email. T: build it reliable enough to stop checking dashboards. A: split into 6 named services; audit tables first; per-recipient independence; idempotent via <code>isAlreadySucceeded(date)</code> . R: two emails/day, every send provable in one SQL query, attendance section flagged 2 real scheduling issues week one.
Probing follow-ups	• Why three audit tables? (<i>Different cardinality + failure semantics</i>) • What if Gmail fails? (<i>Per-recipient log, next run retries</i>) • Why two cron times? (<i>GM leaves 7:30; final 9 PM before reset</i>)
Interviewer criteria	Did you go beyond literal ask? Did you build operability, not just functionality?
Why it lands	Ownership at SDE-1 = "I own end-to-end including the parts no one specced."
Mistakes to avoid	Saying "we" instead of "I"; describing audit as an afterthought; vague results.

Q1.2 — "Tell me about a time you noticed a problem nobody had reported."

FIELD	CONTENT
Code inspiration	Three <code>LEAST(closing_ratio, 100)</code> clamps in three services.
Anchor	<i>"Three clamps in three services screamed 'compensation pattern.' That's the architectural smell I learned to chase to root cause."</i>
STAR skeleton	S: closing-ratio math was capped at three layers. T: was this load-bearing math or a real bug? A: traced to <code>beBack.service.js</code> writing <code>ups_taken + ups_handed_off</code> — Be Back was double-counted as a metric event. Removed the writes; two clamps became removable. R: leaderboard reflects reality; only the <i>official</i> business cap remains.
Probing follow-ups	• What's the smell test? • How do you know you didn't break something else? • What's the difference between this cap and the others?
Interviewer criteria	Architectural intuition; root-cause discipline; lesson generalization.
Mistakes to avoid	Saying "I found a bug" — find the <i>pattern</i> , not the bug.

Q1.3 — "Tell me about a time you cleaned up someone else's mess."

FIELD	CONTENT
Code inspiration	Stale queue ghosts from a silently failed <code>dailyReset</code> .
Anchor	<i>"The reset job had a silent fail mode. I made the consumer resilient — <code>fairInsertV2</code> filters today's rows — so future failures of the reset are invisible to fairness correctness."</i>
STAR skeleton	S: morning queue had a ghost row from yesterday. T: stop scoping the bug to the reset job. A: scoped fairness function to <code>DATE(joined_at) = CURDATE()</code> . R: future reset failures are invisible to fairness; cleanup reliability is decoupled from correctness.
Probing follow-ups	• Why not fix the reset job? • Doesn't this hide failures? • What if both fail?
Why it lands	"Make the consumer resilient to producer failures" is a senior-engineering instinct.

Q1.4 — "Tell me about a system you owned that nobody else understood."

FIELD	CONTENT
Code inspiration	<code>upsFairness.js</code> and the tri-state env flag.
Anchor	<i>"I established the schema-as-history convention — every block dated and idempotent — so anyone walking up cold can read the schema and understand the product."</i>
STAR skeleton	S: I was the only one who understood the fairness rules. T: not become a single point of failure. A: dated migration blocks, project memory files, naming conventions (<code>withStoreTx</code> , <code>requireRole</code>), pure-logic services. R: a second engineer could onboard from schema + one route file.
Mistakes to avoid	"I documented it" — anyone can say that. Show the artifacts (dated blocks, naming, memory files).

2 DIVE DEEP

Q2.1 — "Tell me about a bug that took a long time to debug."

FIELD	CONTENT
Code inspiration	Birdeye duplicate SMS — one in fifty saves.
Anchor	<i>"First hypothesis was 'frontend double-click — disable the button.' That would have masked the race. I read the route and saw a non-atomic read-then-write. Fix is two lines of SQL."</i>
STAR skeleton	S: customers sometimes got two SMS. T: find root cause, not the surface fix. A: reproduced by opening invoice in two tabs and clicking Save fast; read <code>route.invpdf.js</code> ; saw read-then-write on <code>birdeye_sent</code> ; replaced with atomic claim <code>UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0</code> . R: zero duplicates; column doubles as support flag.
Probing follow-ups	• Why not <code>FOR UPDATE ?</code> (Holds row lock through HTTP call — multi-hundred ms) • What about rollback on send failure? (<code>UPDATE ... SET birdeye_sent=0</code> so retry can re-claim) • Why not a queue? (One column replaces infra)
Why it lands	"I rejected the surface fix" + concrete reproduction + one-column solution.
Mistakes to avoid	"We added a unique constraint" — wrong fix, not the actual solution.

Q2.2 — "Tell me about a problem where the obvious solution was wrong."

FIELD	CONTENT
Code inspiration	Package qty parser — "default to 1 when you can't parse."
Anchor	<i>"Silent default would have meant the warehouse picked 1 sofa instead of 3 and the customer was furious. The right answer was a louder failure mode, not a quieter one."</i>
STAR skeleton	S: ERP description has typos sometimes. T: pick between silent default and explicit failure. A: parser returns <code>null</code> ; dialog disables Save with "fix package description in ERP." R: zero wrong-pick incidents; operator has actionable error.
Probing follow-ups	• What's the cost of each failure mode? • Why not 50% confidence threshold? (<i>Confidence ≠ correctness — wrong qty silently is worse than no qty loudly</i>)

Q2.3 — "Walk me through a piece of code you wrote that you're most proud of."

FIELD	CONTENT
Code inspiration	<code>withStoreTx</code> — the queue concurrency primitive.
Anchor	<i>"415 lines, three error codes, one transaction primitive. Every queue mutation routes through one function. Grep for <code>pool.query</code> inside a queue-mutating route — zero results."</i>
STAR skeleton	S: queue could be mutated from /take, /finish, /break, /be-back, /abandoned-up. T: prevent any of those from racing on the same store. A: per-store named advisory lock + <code>FOR UPDATE</code> + deadlock retry + structured <code>UpsError</code> codes. R: zero corruption events in production; new queue features inherit safety.
Probing follow-ups	• What's the worst-case lock contention? (<i>5s acquire timeout → 503 retryable</i>) • Why named lock not table lock? (<i>Per-store, doesn't block other stores</i>)

Q2.4 — "Tell me about a time you went deeper than the team needed."

FIELD	CONTENT
Code inspiration	Distinctive-word matching in <code>packageQtyParser.js</code> .
Anchor	<i>"Substring matching was 80% accurate. The two-pass distinctive-word algorithm got to 100%. The 20% I chased was the 20% the warehouse called about."</i>
STAR skeleton	S: package descriptions share tokens like "SHADOW." T: precision matters because wrong qty = wrong picks. A: Pass A exact-token; Pass B substring with longest-overlap-wins. R: zero false-positive parses in production.

3 BIAS FOR ACTION

Q3.1 — "Tell me about a time you made a decision with incomplete information."

FIELD	CONTENT
Code inspiration	2026-05-23 — PKG component location rollback.
Anchor	"I didn't fully understand root cause yet, but I had a flag. Flipped <code>PKG_COMPONENT_LOC_ENABLED</code> to false in 30 seconds. Floor was unaffected within minutes; I had the rest of the day to investigate calmly."
STAR skeleton	S: day-three issue on live floor. T: restore operations fast; investigate later. A: flag flip in <code>packageWarnings.js</code> ; saved data in <code>pkg_component_locations</code> preserved; dialog stops mounting; warehouse reverts to pre-rollout behavior. R: minutes-to-mitigation; full investigation calmly that afternoon.
Probing follow-ups	• Wasn't that a workaround? (<i>Workaround now, fix later — the asymmetry of cost was the decision</i>) • How do you decide flag-off vs forward-fix? (<i>Containable + no flag → forward; stuck floor + flag → flag off</i>)
Why it lands	Bias for Action = preparation. The flag exists because I built the flag.

Q3.2 — "Tell me about a time you shipped something faster than expected."

FIELD	CONTENT
Code inspiration	Abandoned UP, 2026-05-15 — same-day request to production.
Anchor	"One service file, one route, one schema block, one report section. I cut scope by reusing the Be Back transition pattern and made manager approval explicit."
STAR skeleton	S: top performers being penalized for greeting non-customers. T: same-day fix. A: <code>abandonedUp.service.js</code> reused <code>withStoreTx</code> ; <code>abandoned_up_logs</code> table + <code>ups_logs</code> rows for dual-purpose audit; role gate at route + service. R: same-day deploy; first abuse flag surfaced in week one.

Q3.3 — "Tell me about a time you didn't wait for permission."

FIELD	CONTENT
Code inspiration	<code>idx_ia_created_at</code> — May 19, 2026.
Anchor	"The new endpoint would have full-scanned the archive. I bundled the index migration with the code release and documented the deploy step. No meeting required."
STAR skeleton	S: internal read API needed day-range scans. T: avoid full table scan. A: added migration block to <code>schema.sql</code> with date + "run this while deployment"; coordinated with ops via deploy notes. R: query went from $O(n)$ to $O(\log n)$ on day one.

Q3.4 — "When did you choose to act fast over acting correctly?"

FIELD	CONTENT
Code inspiration	The first executive email body (pre-May 16).
Anchor	<i>"I shipped the table format knowing it wasn't ideal — wanted the GM to actually read it for a week before redesigning. Real usage feedback beat my pre-design taste."</i>
STAR skeleton	S: needed daily summary in email body fast. T: ship v1 even if imperfect. A: single-table format; logged GM's actual reading behavior on mobile. R: redesign on May 16 was data-driven (three-section Combined/Arden/Waynesville executive body).

4 CUSTOMER OBSESSION

Q4.1 — "Tell me about a feature you built specifically because of a customer."

FIELD	CONTENT
Code inspiration	Abandoned UP — top performers being penalized for greeting vendors/friends.
Anchor	<i>"The salespeople were losing position for being polite. Manager one-click puts them back. It's a UX feature for them, but a fairness feature for the customer queue."</i>
STAR skeleton	S: top performers reluctant to greet ambiguous visitors. T: restore fairness without enabling abuse. A: manager-gated, audited, abuse-flagged. R: first abuse flag surfaced legitimately in week one — feature <i>and</i> safeguard both worked.

Q4.2 — "Tell me about a customer frustration that changed your design."

FIELD	CONTENT
Code inspiration	Phone Order (PH) signature flow.
Anchor	<i>"Customers were being asked to sign twice — once in store, once in the PDF. I gated on-screen sigs by invoice type and let Adobe eSign handle remote signing in PDF."</i>
STAR skeleton	S: duplicate signature requests on phone orders. T: collapse to one. A: <code>validateInvoiceSignatures.js</code> skips customer sigs for PH ; <code>InvoicePreview.jsx</code> HeadlessChrome branch renders Adobe eSign tags only in PDF. R: zero duplicate-sig complaints since deploy.

Q4.3 — "Tell me about two customers with conflicting needs."

FIELD	CONTENT
Code inspiration	Warehouse vs salesperson on the package dialog.
Anchor	<i>"Same data, two render targets. <code>/internal-invoice/:slug</code> shows per-component LOC for warehouse; <code>/invoice/:slug</code> shows one clean package line for the customer."</i>

Q4.4 — "Tell me about a feature you killed because customers didn't need it."

FIELD	CONTENT
Code inspiration	The legacy flat <code>components</code> list in <code>/api/package/:packageId/components</code> .
Anchor	<i>"I shipped a dual response shape — <code>componentGroups</code> (new) + <code>components</code> (legacy). The legacy list became dead code. I'd remove it next quiet sprint."</i>
Why it lands	Honest acknowledgment of over-engineering — bar raisers reward self-criticism.

5 DELIVER RESULTS

Q5.1 — "Tell me about a project where you owned the outcome."

FIELD	CONTENT
Code inspiration	UPS Fairness V2 — shadow → on rollout.
Anchor	<i>"Eight days from <code>shadow</code> to <code>on</code> . Whole rollout was reversible at any moment because the read mode was a single env var."</i>
STAR skeleton	S: late joiners leapfrogging in V1. T: ship V2 without breaking the live floor. A: tri-state env flag; dual-write to <code>ups_sessions</code> during shadow week; comparison script; cutover via single env-var flip. R: behavior changed live; zero downtime; revert path proven.

Q5.2 — "Tell me about a time you delivered under pressure."

FIELD	CONTENT
Code inspiration	Daily summary deadline before fiscal close.
Anchor	<i>"I wrote the audit tables before the aggregator. Observability before functionality — so I could ship and iterate."</i>
STAR skeleton	S: GM needed daily summary by fiscal week-close. T: ship reliable, not just working. A: <code>report_job_history</code> + <code>report_delivery_logs</code> + <code>report_drive_upload_logs</code> first; then orchestrator; then renderer. R: every run provable; transient Gmail rate-limit recovered automatically via per-recipient logs.

Q5.3 — "Tell me about a goal you missed."

FIELD	CONTENT
Code inspiration	PKG component allocation v1 — single instance per <code>item_id</code> .
Anchor	<i>"I shipped a 1-instance version when the data needed 4. Lesson: schema should support worst-case shape on day one."</i>
STAR skeleton	S: dialog deployed May 20; first real package was 4 chairs. T: rewrite without disrupting active use. A: schema migration to add <code>instance_index</code> ; dialog rewrite to per-instance cards. R: multi-instance support in production by end of week.

6 INVENT AND SIMPLIFY

Q6.1 — "Tell me about a clever solution you came up with."

FIELD	CONTENT
Code inspiration	Atomic boolean claim for Birdeye.
Anchor	<i>"Two lines of SQL did the work of a distributed lock. <code>UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0</code>; if <code>affectedRows == 1</code>, you won — go send."</i>
STAR skeleton	S: concurrent saves sent duplicate SMS. T: exactly-once without new infra. A: atomic compare-and-swap via SQL UPDATE; rollback on send failure. R: zero duplicates; column doubles as support flag.

Q6.2 — "Tell me about a time you simplified an over-engineered design."

FIELD	CONTENT
Code inspiration	Closing-ratio clamps in three services.
Anchor	<i>"Three clamps in three files. One root cause. Removing the root cause removed two clamps."</i>

Q6.3 — "Tell me about a reusable abstraction you built."

FIELD	CONTENT
Code inspiration	<code>withStoreTx + safeAuditLog + formatDisplayName + locAllocation.js</code> .
Anchor	<i>"locAllocation.js is shared between the item allocator and the package-component dialog. LeftPanel keeps inline copies on purpose — hot render path shouldn't pay import cost."</i>

7 LEARN AND BE CURIOUS

Q7.1 — "Tell me about a new technology you picked up."

FIELD	CONTENT
Code inspiration	MySQL <code>JSON_TABLE</code> + <code>FOR ORDINALITY</code> .
Anchor	<i>"FOR ORDINALITY returns the 1-based array index of each JSON element. That's exactly the seller_index = 1 filter I needed for primary-only operational metrics."</i>
STAR skeleton	S: needed per-seller revenue with split attribution. T: do it in DB or in Node? A: learned <code>JSON_TABLE</code> ; built <code>TICKET_SELLER_CTE</code> ; added <code>supportsJsonTable()</code> runtime check + JS fallback. R: ~30x faster than in-Node fan-out; one source of truth for all reporting endpoints.

Q7.2 — "Tell me about a time you taught yourself something to unblock."

FIELD	CONTENT
Code inspiration	Puppeteer + Gmail OAuth + Drive API for the daily report.
Anchor	<i>"Three Google APIs in two days. I built each with a small smoke-test endpoint first so integration came up incrementally."</i>

Q7.3 — "Tell me about a time you went out of your way to understand a system you didn't own."

FIELD	CONTENT
Code inspiration	<code>/api/audit/rv-schema</code> — discovery endpoint for the legacy ERP.
Anchor	<i>"I built <code>/api/audit/rv-schema</code> as my discovery tool so anyone after me could see what fields existed without reading the legacy ERP docs."</i>

8 EARN TRUST

Q8.1 — "Tell me about a time you delivered bad news."

FIELD	CONTENT
Code inspiration	Telling the GM the leaderboard was wrong (Be Back metric drift).
Anchor	<i>"I led with the corrected numbers, not the diagnosis. 'The leaderboard you used Monday is wrong, here's what it should be, here's the fix.' Trust came from delivering the number, not the apology."</i>
STAR skeleton	S: closing-ratio biased for ~2 weeks. T: tell GM before fixing. A: in-person; corrected numbers first; then root cause; then fix; then BB Accepted column. R: GM had right numbers in 24 hours and re-published prior week.

Q8.2 — "Tell me about a time you admitted a mistake."

FIELD	CONTENT
Code inspiration	<code>/finish-up/start</code> race I shipped before the May 18 fix.
Anchor	<i>"Two endpoints opened the same <code>ups_finish_logs</code> row. I told the team, made <code>/finish</code> the only opener, and added <code>fallbackUserId</code> so old open rows self-heal."</i>

Q8.3 — "Tell me about a time you changed your mind."

FIELD	CONTENT
Code inspiration	Package dialog block-on-save → surface error inline.
Anchor	<i>"My first design blocked save with no path forward. Teammate argued operators would be stuck if ERP description was wrong. I changed to inline-error + disabled Save with actionable message."</i>

9 ARE RIGHT, A LOT

Q9.1 — "Tell me about a judgment call you got right."

FIELD	CONTENT
Code inspiration	Picking MySQL advisory lock over Redis for the queue.
Anchor	<i>"Single MySQL instance, two showrooms. Reaching for Redis adds operational debt for marginal benefit."</i>

Q9.2 — "Tell me about a judgment call you got wrong."

FIELD	CONTENT
Code inspiration	First PKG schema — one row per <code>(package_id, item_id)</code> .
Anchor	<i>"I modeled for the easy case. First real package was 4 chairs. Schema should support worst-case data shape on day one."</i>

Q9.3 — "Tell me about a time you trusted data over opinion."

FIELD	CONTENT
Code inspiration	Closing-ratio 100% cap — kept after debate.
Anchor	<i>"People wanted the cap removed. Data showed unbounded values were artifacts of double-counting Be Back. I removed the double-counting; the cap stayed because the business defines it that way."</i>

10 HAVE BACKBONE; DISAGREE AND COMMIT

Q10.1 — "Tell me about a time you disagreed with a teammate."

FIELD	CONTENT
Code inspiration	localStorage-only finishing-up vs server-authoritative <code>ups_finish_logs</code> .
Anchor	<i>"localStorage gives you a fast badge. But two devices show different truths. I put the source of truth on the server and used localStorage as a same-tab elapsed-timer only."</i>
STAR skeleton	S: teammate proposed localStorage-only finish-up timer. T: argue for hybrid. A: made case with concrete cross-device scenario; landed on <code>ups_finish_logs</code> truth + localStorage accelerator; May 18 atomic-opener fix eliminated race class. R: cross-device consistency; same-tab snappiness.

Q10.2 — "Tell me about a design choice you fought for."

FIELD	CONTENT
Code inspiration	Surfacing parser <code>null</code> instead of defaulting qty to 1.
Anchor	<i>"Silent default would have shipped picking errors. Loud failure is the right shape — operator can fix the ERP description; warehouse can't fix a wrong pick after the fact."</i>

Q10.3 — "Tell me about a time you disagreed but committed."

FIELD	CONTENT
Code inspiration	Body <code>userId</code> auth model.
Anchor	<i>"I'd have built JWT. Team's call was to ship faster on LAN deployment. I committed by hardening <code>requireRole.js</code> to re-read role from DB on every request — privilege escalation via stale role is impossible even without JWT."</i>

1 1 INSIST ON THE HIGHEST STANDARDS

Q11.1 — "Tell me about a time you raised the quality bar."

FIELD	CONTENT
Code inspiration	Three audit tables before the aggregator.
Anchor	<i>"Operations was going to forward emails as proof of delivery. I built three audit tables so we'd know if delivery succeeded — even if no human eye saw the email."</i>

Q11.2 — "Tell me about a time you refused to take a shortcut."

FIELD	CONTENT
Code inspiration	Two-pass distinctive-word qty parser.
Anchor	<i>"Substring was a shortcut. I wrote the two-pass matcher because the warehouse can't recover from a silently wrong qty — but they can recover from a loud one."</i>

Q11.3 — "Tell me about a time you held the line on engineering quality."

FIELD	CONTENT
Code inspiration	Schema migration convention — dated + idempotent.
Anchor	<i>"Every block in <code>schema.sql</code> is dated and idempotent. The convention isn't documented in a README — it's enforced by code review. That's how it survives."</i>

12 — Quick-Fire Anchor Bank (one-liners ready to drop)

PRINCIPLE	ONE-LINER ANCHOR
Ownership	"I built the audit tables before the aggregator. Observability before functionality."
Dive Deep	"Three clamps in three files. One root cause. Removing the cause removed two clamps."
Bias for Action	"30-second rollback because I built the flag with the feature."
Customer Obsession	"Salespeople were losing position for being polite. Manager one-click restores them."
Deliver Results	"Eight days from shadow to on. One env-var flip. Zero downtime."
Invent and Simplify	"Two lines of SQL replaced what a lot of teams reach for a queue to solve."
Learn and Be Curious	" FOR ORDINALITY does in SQL what I'd been doing with in-process counters for years."
Earn Trust	"I led with the corrected numbers, not the diagnosis."
Are Right, A Lot	"People wanted the cap removed. Data showed unbounded was an artifact of double-counting."
Have Backbone	"localStorage gives you a fast badge. The server gives you the truth."
Highest Standards	"A working pipeline is necessary. An operable pipeline is sufficient."

13 — Interviewer Evaluation Rubric

For each STAR you deliver, the interviewer is silently scoring:

DIMENSION	WEAK (1)	STRONG (5)
Specificity	"We solved a bug"	"Atomic UPDATE on <code>birdeye_sent</code> column; <code>affectedRows</code> <code>=== 1</code> signals winner"
First-person ownership	"We" / "the team"	"I made this call; here's why"
Tradeoff awareness	"It was the right choice"	"I rejected Redis; the cost was operational debt for marginal benefit"
Result orientation	"Users were happy"	"Two clamps became removable; closing ratio is correct by construction"
Generalization	"We fixed it"	"The pattern (consumer resilient to producer failure) is now how I think about audit-vs-state tables"
Backbone	Always agreed	"I made the case; we landed on a hybrid; here's how I committed"
Self-awareness	"I have no weaknesses"	"I undertest. Pure-logic services would be where I'd add Jest first"

Aim for 4–5 on every dimension for every STAR.

**PART III**

STAR BEHAVIORAL ANSWERS

12 Full Scripts · Probing Chains

COVERS

STAR Behavioral Answers

Format per story: Principle → Situation → Task → Action → Result → Probing follow-ups (with crisp answers) → Why it lands → Mistakes to avoid → Stronger-phrasing alternative. **Delivery time:** target 2 min for the STAR; expect 3–6 follow-ups after.

★ STAR-01 — The 8 PM Pipeline (Ownership · Highest Standards)

Anchor: "I built `report_job_history` + `report_delivery_logs` + `report_drive_upload_logs` BEFORE the aggregator. Observability before functionality."

Situation

Carolina Furniture Concepts runs two showrooms — Arden and Waynesville. The GM was Slack-screenshotting numbers to himself every night because nobody had consolidated UPS counts, revenue, closing ratios, RTS activity, abandoned UPs, and attendance into one place. The ask was "can we get a daily email."

Task

Build the pipeline reliable enough that the GM could stop manually compiling numbers, runs in NY timezone on a UTC server, and **survives partial failures** — one bad recipient address can't kill the run, a Drive outage can't block the email.

Action

- **Schedule:** `0 19,21 * * * in America/New_York`. 7 PM is the mid-evening snapshot; 9 PM is the revised final — both *before* the 9:01 PM `dailyReset` wipes queues.
- **Split orchestrator** into six named services so each step was independently testable:
 - `dailySalespersonSummary.orchestrator.js` — top-level flow + `force` flag for cron ticks.
 - `dailySalespersonSummary.service.js` — aggregator over `ups_logs`, `invoices_archive`, `customer_rts_entries`, `abandoned_up_logs`.
 - `dailySalespersonSummary.pdf.js` — three renderers (`renderSummaryPdf`, `renderExecutiveEmailHtml`, `renderSummaryCsv`).
 - `dailySalespersonSummary.drive.js` — best-effort Drive upload.
 - `dailySalespersonSummary.recipients.js` — env `REPORT_RECIPIENTS` ∪ DB admin emails, gated by `REPORT_INCLUDE_DB_RECIPIENTS`.
 - `dailySalespersonSummary.audit.js` — `startJobRun`, `finishJobRun`, `logDelivery`, `logDriveUpload`, `isAlreadySucceeded`.

- **Three audit tables before any feature code** — that was the deliberate ordering.
- **Idempotency:** `isAlreadySucceeded(business_date)` only returns true on `status='succeeded'`. A stuck `running` row doesn't block retry. Cron ticks pass `force: true` so 9 PM legitimately supersedes 7 PM.
- **Per-recipient independence:** each Gmail send is its own try/catch + audit row. One bad address fails its own row; the next recipient continues.
- **Drive is best-effort.** Email still goes out; Drive link injected only on successful upload.
- **NY-timezone anchor:** `todayISO()` via `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })` — never raw `new Date()`.

Result

- GM gets two emails per day with PDF + Drive archive link.
- Every run provable via one SQL query against `report_job_history`.
- First week: a transient Gmail rate-limit failed one recipient — audit row showed which recipient, which error, which attempt. Next run sent successfully. **Zero false-fail re-sends.**
- Attendance section caught two real scheduling issues in the first week — feature *and* safeguard working.

Probing Follow-ups

Q	A
Why three audit tables and not one?	Different cardinality + failure semantics. Job-level is 1 row per attempt; recipient-level is N rows per attempt; Drive-level is independent of email. Coupling them would force one table to serve three queries.
Why two runs per day?	7 PM mid-evening for the GM who leaves at 7:30. 9 PM final for the last two hours. Both before the 9:01 reset.
How do you know an email was actually delivered?	I know Gmail accepted it. <code>status='sent'</code> + <code>provider_message_id</code> is "Gmail accepted." Bounce-handling is a future integration.
What if the job dies between Drive upload and Gmail send?	Drive log says <code>status='succeeded'</code> with <code>file_id</code> ; Gmail isn't logged. Next run sees <code>report_job_history.status='running'</code> from the dead one; <code>isAlreadySucceeded</code> returns false; runs fresh.
What's the rollback path if the report format is wrong?	Each renderer is independent; I can ship a fix to just <code>renderExecutiveEmailHtml</code> without touching the PDF or the aggregator.

Why It Lands

SDE-1 ownership = "I built this end-to-end and I can defend every piece." This story includes cron + retries + idempotency + observability + 3rd-party API integration + a real timezone gotcha — breadth without sacrificing depth.

Mistakes to Avoid

- Saying "we" — the audit tables were a deliberate solo design decision.
- "Vague results" — name a number, a table, or an observable behavior.
- Forgetting the timezone story — bar raisers love DST follow-ups.

Stronger-Phrasing Alternative

"If the audit tables disappeared, this pipeline would still work — but it wouldn't be operable. A working pipeline is necessary; an operable pipeline is sufficient. That's the distinction this project taught me."

★ STAR-02 — Birdeye Duplicate SMS Root Cause (Dive Deep · Are Right A Lot)

Anchor: *"Two lines of SQL did the work of a distributed lock."*

Situation

Around April 14, 2026, the team reported customers occasionally receiving two Birdeye review-request SMS for the same invoice. The pattern was intermittent — roughly one in fifty saves. First hypothesis: a frontend double-click. Proposed fix: disable Save on click.

Task

Find root cause. I rejected the surface fix because disabling a button masks a race instead of solving it.

Action

1. Read `route.invpdf.js /invoice/save` end-to-end. Two effectful operations on save:
 - UPSERT into `invoices_archive` (dedup by `(name1, cell, DATE(created_at))`).
 - Birdeye SMS send branch.
2. The SMS branch had a classic non-atomic check-then-act:

```
const row = await getInvoice(slug);
if (row.birdeye_sent === 0) {
  await sendBirdeyeSMS(...); // network call
  await formQuery('UPDATE ... SET birdeye_sent=1 WHERE slug=?', [slug]);
}
```

3. Reproduced: opened the same invoice in two tabs, clicked Save in both within a second → two SMS, every time.
4. Replaced with **atomic claim**:

```
UPDATE invoices_archive SET birdeye_sent = 1
WHERE slug = ? AND birdeye_sent = 0
```

Only the request whose `affectedRows` $\equiv$ 1 actually sends. On SMS-send failure, roll back: `UPDATE ... SET birdeye_sent = 0` so a retry can re-claim.

5. Deployed. Re-ran the two-tab reproduction → one SMS.

Result

- **Zero duplicate SMS** in production since deploy.
- The `birdeye_sent` column doubles as a support flag — "did this customer receive the SMS?" is one SELECT.
- No new infrastructure (no Redis, no queue, no distributed lock library).
- Pattern adopted for any future exactly-once external call.

Probing Follow-ups

Q	A
Why not <code>SELECT ... FOR UPDATE</code> ?	It works but holds a row lock for the duration of the multi-hundred-ms Birdeye HTTP call. Atomic UPDATE releases the lock instantly; HTTP call happens lock-free.
What about rollback if SMS fails?	<code>UPDATE ... SET birdeye_sent=0 WHERE slug=?</code> so the next retry can re-claim. The boolean is not a tombstone.
Could you have added a unique constraint?	Unique on what? <code>slug</code> is already unique. The bug was a non-atomic read-then-write, not a duplicate row.
What if the process dies between claim and successful send?	Claim says 1, no SMS went. We accept this as "lost send" because Birdeye dedup on their side handles repeated requests cleanly. For stronger semantics I'd add <code>birdeye_claimed_at</code> and a sweeper that rolls back claims older than N seconds without a corresponding success log.
Why not use Birdeye's idempotency key?	If they exposed one, I would. The atomic claim is what I can do without external dependency on Birdeye's API contract.

Why It Lands

Canonical Dive Deep arc: rejected the surface fix → traced to root cause → solved with primitive simpler than the alleged solution. Bonus points for naming the pattern (CAS) without being pedantic.

Mistakes to Avoid

- "We added a try/catch" — that's not a fix.

- Confusing this with the dedup path. Two different problems in the same route.
- Claiming the unique constraint was the fix.

Stronger-Phrasing Alternative

"Compare-and-swap implemented in SQL. The database is the lock. The `affectedRows` count is the success signal. No application-level lock needed."

★ STAR-03 — The 30-Second Rollback (Bias for Action)

Anchor: *"I had the flag because I built the flag in advance. Bias for Action is partly preparation."*

Situation

2026-05-23, three days after rolling out the package-component location dialog. An issue surfaced that impacted the live floor. I didn't have full root cause yet — investigation vs immediate mitigation was the decision.

Task

Restore floor operations fast. Investigate calmly afterward.

Action

- Flipped `PKG_COMPONENT_LOC_ENABLED` to `false` in `cfc/src/utils/packageWarnings.js`.
- No other code changes. No schema rollback. Saved rows in `pkg_component_locations` stayed in place for re-enablement.
- Dialog stopped mounting. Invoice rows stopped rendering child components. Warehouse reverted to the pre-May 20 single-row-per-package experience.
- Documented in my project-memory file `project_pkg_component_loc_rollback.md` so anyone walking up cold knows "flip the flag back to re-enable."

Result

- Floor unaffected within minutes.
- Full afternoon to investigate calmly without operational pressure.
- Saved data survived for re-enablement — no data loss, no migration needed to revert.

Probing Follow-ups

Q	A
<i>Wasn't this a workaround, not a fix?</i>	Workaround now, fix later. The asymmetry of cost was the decision — broken floor costs revenue; an unnecessary flag-off costs an hour.
<i>How do you decide flag-off vs forward-fix?</i>	Containable + no flag → forward-fix fast. Stuck floor + flag exists → flag off, investigate calmly.
<i>Was the flag added pre-emptively or after a fire drill?</i>	Pre-emptively. Every behavior change ships with a flag. That's the cost-of-doing-business I committed to after the first painless rollback.
<i>What about the saved data in <code>pkg_component_locations</code>?</i>	Untouched. Schema migrations are forward-compatible (new columns nullable, new tables additive). Re-enabling the flag turns the feature back on with all historical assignments intact.

Why It Lands

SDE-1 bias for action = "I can make a small reversible decision quickly." The critical part is *preparation* — the flag existed because I designed it that way.

Mistakes to Avoid

- Saying "I disabled the feature" — name the flag, name the file.
- Forgetting the data-preservation angle. Flags + idempotent schemas = clean revert.

★ STAR-04 — Be Back Metric Drift (Earn Trust · Dive Deep)

Anchor: *"I led with the corrected numbers, not the diagnosis."*

Situation

Around May 20, 2026, I noticed three independent services were each clamping closing ratio at 100% — `ups.routes.js /top-performers`, `todayReports.service.js`, and `dailySalespersonSummary.service.js`. Three clamps in three services is a compensation pattern. I traced root cause to `beBack.service.js` writing two `ups_logs` rows for every Be Back handoff: `ups_taken` (receiver) and `ups_handed_off` (giver). For ~2 weeks, the closing-ratio leaderboard had been systematically biased.

Task

Tell the GM. He had used those numbers in Monday's all-hands meeting.

Action

1. Drafted the message with three things in this order: (1) the leaderboard you used Monday is wrong, (2) here's the corrected number for each affected salesperson, (3) here's the root cause and the fix.
2. Delivered in person, not Slack — the conversation needed to be a conversation.
3. Same day, shipped the fix in `beBack.service.js`: removed `ups_taken` and `ups_handed_off` writes. Be Back is routing, not a metric event.
4. Added the **BB Accepted** column to the daily summary so handoffs stay visible without distorting metrics.
5. Removed two of the three downstream `LEAST(closing_ratio, 100)` clamps. The third stayed because the cap-at-100% is the official business definition (not a math bug).

Result

- GM had the right numbers within 24 hours. He re-published the prior week's leaderboard with the correction noted.
- The fix shape (no metric writes for Be Back, visible BB Accepted column) became the model for the Abandoned UP feature the following week.
- The clamps that remained are now *intentional*, not compensating.

Probing Follow-ups

Q	A
<i>Why not silently fix it?</i>	He used the numbers in a meeting. Silent fix leaves him with a wrong memory of who's on top. Trust comes from leading with the bad news.
<i>Could you have caught this earlier?</i>	Yes — a metrics-integrity test asserting <code>closing_ratio</code> doesn't need a clamp under clean data would have flagged it. I added that to my "next time" list.
<i>How do you know removing the writes is root cause, not a symptom?</i>	Removing the writes made two clamps removable. If I'd only removed one write or one clamp, the others would still be needed. Removability is the proof of root cause.
<i>What if the GM didn't agree to republish?</i>	I'd have offered the choice: republish corrected, or stop using those numbers. Quietly suppressing the bad numbers while still trusting them wasn't an option.

Why It Lands

Earn Trust at SDE-1 = "I deliver bad news fast with the diagnosis and the fix in the same breath." Concrete numbers + re-published leaderboard + reused pattern in next feature.

Mistakes to Avoid

- "We told stakeholders" — name the person, name the meeting, name the artifact (the leaderboard).
- Apologizing as the lead. The corrected numbers are the lead.

Stronger-Phrasing Alternative

"Three clamps in three services taught me to look for patterns of compensation. Clamps are symptoms; the cure is upstream. The leaderboard was wrong because we were double-counting, not because the math couldn't be capped."

★ STAR-05 — UPS Fairness V2 Shadow → On (Deliver Results · Earn Trust)

Anchor: *"Eight days from shadow to on. The whole rollout was reversible at any moment because the read mode was a single env var."*

Situation

The original UPS queue (V1) treated everyone the same way — a late joiner at 12:30 PM with zero UPs that day could land ahead of the crew that had been on the floor since 9 AM. The team wanted a fairness rule: ineligible new joiners go to the tail; eligible incumbents go where their `ups_count` says.

Task

Roll out the new rule without breaking the live floor.

Action

- **Tri-state env flag** `UPS_FAIRNESS_V2 = off | shadow | on`.
- **Shadow week (May 1–7):** V1 reads unchanged, dual-write to a new `ups_sessions` table. The crew observed the new table populating with no behavior change. I wrote a comparison script that compared what V2 *would* have done against what V1 *actually* did.
- **`upsFairness.js` (~415 lines):**
 - `withStoreTx(store, async (conn) => ...)` — per-store named advisory lock + transaction.
 - `getOrCreateSession(conn, userId, store)` — idempotent open/re-open of today's session.
 - `fairInsertV2(conn, userId, store, session)` — eligibility-gated insertion.
- **Eligibility rule:** an employee earns fairness only after `first_ups_at` is stamped (first `/ups/take` of the day). Until then they FIFO-append to the tail.
- **Cutover (May 8):** flipped `UPS_FAIRNESS_V2 = on`. Reads switched. No data migration. No downtime.
- **Retry semantics:** deadlock + lock-wait-timeout retried twice with fresh connection; `LOCK_TIMEOUT` returned as 503 retryable.

Result

- Behavior changed live with **one env-var flip**.
- Shadow week proved the algorithm against production data.
- Zero rollback needed; revert path was the same single flip.
- The new `ups_sessions` table now powers the closing-ratio numerator and the BB Accepted column too — single source of truth.

Probing Follow-ups

Q	A
What if shadow showed a divergence?	Flip back to <code>off</code> , debug, redeploy. The risk is bounded — no read-path changes during shadow.
What's the deadlock story?	<code>withStoreTx</code> retries twice with a fresh connection. Structured <code>UpsError</code> codes map to HTTP status + retryable flag for frontend backoff.
Why a per-store lock, not a global lock?	Single MySQL instance, two stores. Per-store sharding means adding stores is linear. Global lock would serialize all stores.
What about position 1?	Position 1 is locked — only manager <code>recycle</code> can move it. Joining at fair position never overwrites position 1 because the scan starts from 2.

Why It Lands

Textbook Deliver Results + Earn Trust: shadow-rollout pattern, single-toggle revert, observable behavior change.

Stronger-Phrasing Alternative

"I shipped a behavioral change to a live system using a shadow-rollout pattern with a single-toggle revert. The pattern itself was the deliverable — the next behavioral change (package component dialog) used the same shape."

★ STAR-06 — Server-Authoritative Finishing Up (Have Backbone · Disagree and Commit)

Anchor: *"localStorage gives you a fast badge. The server gives you the truth. The hybrid is both."*

Situation

The first version of the finishing-up paperwork timer was proposed as localStorage-only — "it's just a badge, no need for a server table." I disagreed because of cross-device behavior: a salesperson can start paperwork at the front counter on device A and finish at the manager's desk on device B; the badge would be wrong on device B.

Task

Argue for a server-authoritative `ups_finish_logs` table without losing the snappiness of the localStorage badge.

Action

- Proposed the hybrid: `ups_finish_logs` is the truth (the queue UI on every device reads from `/ups/finish-up/active`); localStorage is a same-tab elapsed-timer accelerator with a 30-min TTL.
- May 18 atomic fix: `/ups/finish` became the *only* opener of an `ups_finish_logs` row. Pre-fix, both `/ups/finish` and `/finish-up/start` could open a row, leaving two open rows for the same user.
- Added `clearAllFinishingUp(workflowType, fallbackUserId)` — if localStorage is empty (private browser, second device, cache wipe), the call still closes the open row by `(userId, ended_at IS NULL)` lookup.

Result

- Cross-device finishing-up state is consistent.
- localStorage still makes the badge feel instant.
- The May 18 atomic fix eliminated the duplicate-row class of bug.
- The pattern (server-authoritative + same-tab accelerator) became the model for every subsequent feature.

Probing Follow-ups

Q	A
How do you decide localStorage vs server?	Server-authoritative for anything multi-device. localStorage for same-tab UX accelerators only. The litmus test: "if someone signs in on a second device, do they see the same state?"
What about the lag of polling vs localStorage?	Board polls every second. localStorage is "this tab's truth"; server is "everyone's truth." Lag is 1s for cross-device, 0s for same-tab — acceptable.
What if you'd lost the argument?	I'd have committed by adding cross-tab <code>storage</code> event listeners and writing 'this is a same-tab badge only' into the file header. The bug would have surfaced anyway.

Why It Lands

"I disagreed with a simpler design, made the case, got us to a hybrid that's both correct and fast." Concrete code anchors (`ups_finish_logs`, `fallbackUserId`, `/finish-up/active`) make it credible.

★ STAR-07 — Atomic Boolean Claim (Invent and Simplify)

Anchor: "One column replaced what teams reach for a queue to solve."

(See STAR-02 for the technical detail; this story is the same fix viewed through the Invent and Simplify lens.)

Situation

Concurrent invoice saves were sending duplicate Birdeye SMS.

Task

Fix without introducing new infrastructure (no Redis, no queue, no library).

Action

- One column: `birdeye_sent`.
- One query pattern: `UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0`. `affectedRows` `== 1` is the success signal.
- On send failure: `UPDATE ... SET birdeye_sent=0 WHERE slug=?` so retry can re-claim.

Result

- Zero duplicate SMS.
- The column doubles as a support flag and feeds the daily report aggregator.

Why It Lands

"Solved a real concurrency problem with one column and one query." Bar raisers love clean primitives. Bonus: name the pattern (CAS) without being pedantic.

Stronger-Phrasing Alternative

"The database is the lock. The `affectedRows` count is the success signal. The simplest form of the right answer."

★ STAR-08 — Closing-Ratio Cap (Are Right, A Lot)

Anchor: "People wanted the cap removed. Data showed unbounded was an artifact of double-counting, not a math bug."

Situation

During the Be Back metric drift post-mortem, someone proposed removing the `LEAST(closing_ratio, 100)` cap entirely. The argument was: "the math should just work; if numbers exceed 100, the math is wrong."

Task

Decide whether the cap was a bug or a definition.

Action

1. Pulled the data. In the legitimate post-fix flow, a Be Back receiver who closes the sale gets `primary_tickets = 1` against `ups_taken_raw = 0`. The raw ratio is "undefined" or infinity.
2. Considered three options:
 - Remove the cap, accept undefined/infinity (broken UI).
 - Change the formula to a different denominator.
 - Keep the cap; it's the business definition, not a math bug.
3. Made the case with three concrete example rows showing what each produced.
4. Chose option 3. The business defines closing ratio as "primary tickets over UPs you personally took, capped at 100% by definition."

Result

- Cap stayed in `dailySalespersonSummary.service.js`.
- Be Back metric writes were removed (the real bug).
- Two of three downstream clamps deleted; the surviving one is the official business cap.

Probing Follow-ups

Q	A
How do you distinguish math bug from definition?	A math bug breaks reproducibility — same data, different answers. A definition is "this is what the business calls it." If removing the cap would change the business's mental model of the KPI, it's a definition.
How did you make the case?	Three example rows. Walked through each: with cap, without cap. The without-cap version had a 500% closing ratio that was meaningless.

Stronger-Phrasing Alternative

"I trusted the data over the opinion that 'math should just work.' The unbounded ratios were artifacts of upstream double-counting, not a math problem the cap was solving."

★ STAR-09 — JSON_TABLE + FOR ORDINALITY (Learn and Be Curious)

Anchor: "FOR ORDINALITY does in SQL what I'd been doing with in-process counters for years."

Situation

Per-salesperson revenue needed multi-seller split — a ticket with 3 sellers credits `subtotal / 3` to each. Daily summary aggregates ~hundreds of tickets per day; interactive endpoints (`/top-performers` , `/today`) are latency-sensitive.

Task

Decide between (a) fanning out in Node and aggregating in-process, or (b) doing it in SQL.

Action

1. Recognized that the same query was used by three endpoints. SQL CTE means one source of truth.
2. Spent an hour learning `JSON_TABLE` in MySQL 8. The killer feature was `FOR ORDINALITY` — the 1-based array index of each element. Exactly the `seller_index = 1` filter I needed for "primary only" operational metrics.
3. Built `TICKET_SELLER_CTE` in `backend/src/services/sales/ticketSeller.sql.js`:

```
WITH TICKET_SELLER_CTE AS (
  SELECT ia.id AS invoice_id,
         JSON_UNQUOTE(JSON_EXTRACT(ia.form_json,
                                     CONCAT('$.sellerNames[', t.seller_idx - 1, '].name'))) AS seller_name,
         t.seller_idx AS seller_index,
         ia.form_json ->> '$.subtotal'
           / GREATEST(JSON_LENGTH(ia.form_json, '$.sellerNames'), 1)
         AS credited_subtotal
  FROM invoices_archive ia
  JOIN (... JSON_TABLE(...) ... FOR ORDINALITY ...) t
)
```

4. Added `supportsJsonTable()` runtime check + JS fallback `expandTicketSellers` for older MySQL.

- Validated parity by running both paths on same day's data and diffing per-row credited subtotals — identical.

Result

- ~30x faster than in-Node fan-out on warmed-cache range queries.
- One CTE feeds three endpoints — single source of truth.
- Fallback path means older MySQL deployments still work.

Probing Follow-ups

Q	A
What surprised you?	<code>FOR ORDINALITY</code> — I'd been writing in-process counters to do exactly this for years.
How do you know parity?	Diffed identical per-row credited subtotals + primary-only filtering across both code paths on the same data.
Why not require MySQL 8?	Older deployments exist. Runtime feature detect + fallback is operational maturity, not over-engineering.

Why It Lands

"I learned something new because the simple answer wasn't good enough" + a graceful-degradation fallback shows operational discipline.

★ STAR-10 — Schema Wide Enough on Day One (Failure → Lesson)

Anchor: "Schema should support the worst-case data shape on day one, even if v1's UI only exposes the simple case."

Situation

First version of the package-component location dialog (May 20) supported one instance per item per package. First real package I tested in staging was a 4-chair set — `ItemPackageMaster` had four rows for the same chair `item_id`, one per piece. My schema had `UNIQUE(package_id, item_id)` — couldn't represent "chair #1 → S1, chair #2 → S2."

Task

Rewrite without disrupting active use.

Action

- Added `instance_index INT` column.
- New unique key: `UNIQUE(package_id, item_id, instance_index)`.
- Dialog rewrite: per-instance card with its own location picker.
- Qty parser still returned `required_qty = 4` for the chair component; validator checked `SUM(loc qty across all 4 instances) == 4`.

Result

- Multi-instance support in production by end of week.
- Pattern generalized: every schema design now starts with "what's the worst-case shape?"

Probing Follow-ups

Q	A
How did you discover the gap?	First real package I tested. <code>ItemPackageMaster</code> had four rows for the same chair — that's how the ERP encodes multi-instance.
Could you have known earlier?	Yes — one query against <code>ItemPackageMaster ORDER BY (item_id, count(*)) DESC</code> would have surfaced multi-instance packages on day one. I added that as a "first query before schema" rule.

Why It Lands

Failure story with a generalizable lesson + concrete artifact (the schema change). Bar raisers reward self-awareness.

★ STAR-11 — Frugality (Reusing the Daily Summary Pipeline for RTS)

Anchor: "The 8 PM email already knew how to render and deliver. I just gave it a new section."

Situation

When we added the RTS interaction-capture feature, the ask included "we need it visible in the daily report."

Task

Don't build a separate pipeline.

Action

- Added one schema block: `customer_rts_entries(salesperson_id, username, store_name, returned_amount, written_sale, interaction_id, created_at)`.
- Added one service: `rts.service.js` with `createRtsEntry` + `fetchRtsEntriesForDay`.
- Added one section to the existing `dailySalespersonSummary.service.js` aggregator. The PDF renderer picked it up automatically (iterates over named sections).
- Linked to `customer_acquisition` via `interactionId` so each RTS shows which acquisition source led to the return.

Result

- RTS section in daily PDF + executive email with zero new pipeline.
- Net new code: one table, one service, one section.

Why It Lands

"Notice the right pipeline already exists; don't build a new one." Concrete reuse + minimal net new surface area.

★ STAR-12 — Observability Before Functionality (Highest Standards)

Anchor: *"From day one, I could answer 'did the 9 PM run actually go out?' in one SQL query."*

(Same project as STAR-01; this is the same work viewed through the "highest standards" lens.)

Situation

Daily summary email job — could ship happy-path-first and add logging when something broke, or build the audit tables first.

Task

Pick the order.

Action

- Wrote `report_job_history` first — one row per attempt with `status/attempt_no/duration_ms/error_message`.
- Wrote `report_delivery_logs` second — one row per recipient per attempt.
- Wrote `report_drive_upload_logs` third — one row per file upload.
- Wrote the orchestrator that emits rows into all three at the right boundaries.

- Then wrote the aggregator and renderer.

Result

- First cron tick: every row in every audit table was already wired.
- Could prove the job ran without checking email.
- One transient Gmail rate-limit in week one: audit row told me which recipient, which error, which attempt. Retry logic handled it.

Stronger-Phrasing Alternative

"A working pipeline is necessary. An operable pipeline is sufficient. Observability isn't a feature you add later — it's the feature that makes everything else operable."



STAR Quick-Reference Card

#	STORY	PRIMARY LP	SECONDARY LP	ANCHOR
01	8 PM Pipeline	Ownership	Highest Standards	Audit tables before aggregator
02	Birdeye Dup SMS	Dive Deep	Are Right A Lot	Two lines of SQL replaced distributed lock
03	PKG Flag Rollback	Bias for Action	—	30-second revert because I built the flag
04	Be Back Metric Drift	Earn Trust	Dive Deep	Led with corrected numbers, not diagnosis
05	UPS Fairness V2	Deliver Results	Earn Trust	One env-var flip; shadow week proved algo
06	Server-Auth Finishing	Have Backbone	—	localStorage gives badge; server gives truth
07	Atomic Claim	Invent & Simplify	—	One column replaced infrastructure
08	Closing-Ratio Cap	Are Right A Lot	—	Data over opinion; cap is definition not bug
09	JSON_TABLE	Learn & Be Curious	Frugality	FOR ORDINALITY in SQL; 30x faster
10	PKG Schema	Failure → Lesson	Are Right A Lot	Schema for worst-case shape day one
11	RTS Pipeline Reuse	Frugality	—	Existing pipeline absorbed new section
12	Audit-First	Highest Standards	Ownership	Operability is sufficient; working is necessary

STAR Delivery Templates (use these stems on the fly)

- **Situation stem:** *"At Carolina Furniture Concepts, our two-showroom operation needed [specific operational need]..."*
- **Task stem:** *"My job was to [specific outcome], without [specific failure mode]."*
- **Action stem (3+ bullets always):** the file, the function, the table, and one tricky decision.
- **Result stem:** *"After we shipped, [observable behavior or named artifact]."* Avoid vague results — name a table, a column, or a behavior.

End every STAR on a hook the interviewer can pull. Never close on "and that's how we did it" — close on the next interesting design choice.

**PART IV**

BAR RAISER SIMULATION

Ambiguity · Conflict · Failure · Curveballs

COVERS

Amazon Bar Raiser Questions

Bar Raiser ≠ technical screen. They test *judgment under uncertainty*. They will probe until you flinch, then probe one more time. **Format per question:** Question · What they're really probing · Strong answer · The trap · Probing chain (4–6 deep) · Stronger phrasing.

0 — Pre-Round Mindset

BAR RAISER IS LOOKING FOR	ANTIDOTE (WHAT TO DEMONSTRATE)
Vague results	Name a table, a column, a number, a re-published artifact.
"We" instead of "I"	Stay first-person all the way through.
No tradeoff	Every decision has a cost — name yours.
Failure theater ("I learned humility")	Name the concrete artifact of the failure (wrong schema, wrong shape, wrong default).
Always-agreed	Surface a disagreement that ended in a hybrid or a commit.
Hindsight bias ("the right answer was obvious")	Acknowledge what you didn't know at the time.

"Brief is good — silent is not. Vague is worse than wrong."

1 AMBIGUITY HANDLING

Q1.1 — "Walk me through a project where requirements were unclear. What did you do?"

They're probing: Do you wait for spec or produce a draft and iterate?

STRONG ANSWER

"The package-component location feature started with one sentence — 'salespeople need to assign each component to a location.' No spec.

I started with the schema, not the UI. Schema forces decisions. `pkg_component_locations(package_id, item_id, instance_index, location_id, saved_by_user_id)` — two questions answered themselves: multi-instance support (yes, `instance_index`), allowed locations (S1, S2, 999, NS1, NS2, SO, codified as `ALLOWED_ALLOCATION_LOCATIONS`).

Then I drafted the API response shape (`componentGroups` + `components` for back-compat) and built the dialog against it. We iterated on UX after seeing it work end-to-end — that's where the multi-location qty allocator came from on May 21 and the qty-unknown error on May 23."

THE TRAP: Saying "I asked questions until I had clarity." Real PMs don't have answers either — produce a draft.

Probing chain:

Q	A
<i>How did you decide what to defer?</i>	I deferred multi-instance to v1.1 mentally — then realized day-one that real packages had it (4 chairs in <code>ItemPackageMaster</code>). Rewrote schema same week.
<i>How did you know your draft schema was right?</i>	I didn't. The <code>UNIQUE(package_id, item_id, instance_index)</code> was a guess that paid off. If the worst-case shape had been <code>(location, qty)</code> -keyed instead, I'd have rewritten.
<i>Wasn't that wasteful?</i>	Less wasteful than over-specifying for a year. Two-week feedback cycle on a real schema beats four-month spec on a hypothetical one.
<i>How do you decide schema-first vs UI-first?</i>	Schema first when the data model is non-obvious. UI first when the data is well-understood and the question is "how do users interact." For PKG, data was non-obvious.

STRONGER PHRASING

"Schema forces decisions. If I can write the unique key, I understand the domain. If I can't, the domain is ambiguous."

Q1.2 — "You're handed a feature with no acceptance criteria. How do you define 'done'?"

STRONG ANSWER

"Three observable conditions:

1. Happy-path user completes the flow without help.
2. Every failure mode I designed for surfaces an actionable error, not a 500.
3. The audit trail proves the system did what it claimed.

Concretely on the daily summary: done was 'audit tables `report_job_history`, `report_delivery_logs`, `report_drive_upload_logs` all show consistent state after a real 7 PM run, recipients receive a PDF, the Drive link works.' If any of those wasn't true, it wasn't done."

THE TRAP: Vague "done = working." Define falsifiable conditions.

Probing chain:

Q	A
<i>What if happy-path passes but you don't have an audit trail?</i>	Not done. Audit trail is the evidence that lets ops trust the pipeline.
<i>How do you handle a 500 you didn't design for?</i>	New audit row with the unexpected error. Then a follow-up to make it actionable.
<i>Is this overkill?</i>	For a daily money-adjacent report read by the GM? No. For a debug endpoint? Yes — match rigor to consequence.

2 CONFLICT QUESTIONS

Q2.1 — "Tell me about a time you disagreed with a teammate's technical approach."

They're probing: Do you have backbone AND can you commit?

STRONG ANSWER

"localStorage-only finishing-up timer. A teammate argued simpler + snappier; I argued cross-device behavior would break. I made the case with a concrete user scenario: salesperson starts at front counter, finishes at manager's desk on a different device — badge would be wrong on the second device.

We agreed on the hybrid: server-authoritative `ups_finish_logs` for truth, localStorage as same-tab elapsed-timer accelerator. I committed by building the May 18 atomic fix — `/ups/finish` became the only opener — eliminating the duplicate-row race that came from the original design."

THE TRAP: "I won and now everyone uses my design." Bar raisers reward collaboration + commit.

Probing chain:

Q	A
What if you'd lost?	I'd have committed by adding cross-tab <code>storage</code> event listeners and writing 'this is a same-tab badge only' into the file header. The cross-device bug would have surfaced anyway and we'd have rebuilt then.
How did you bring the data?	I asked them to walk through the flow on two physical devices. The first device's badge stayed alive; the second's was wrong. The flow was the argument.
Was the hybrid actually better?	Yes — and the proof is the May 18 fix landed cleanly because the server side was already authoritative. If we'd been localStorage-only, the duplicate-row class of bug would have required a much bigger rewrite.

STRONGER PHRASING

"I disagreed loudly during design, committed completely during build. Backbone is a design behavior; commit is an execution behavior. They're not contradictory."

Q2.2 — "Tell me about a time stakeholders had conflicting priorities."**STRONG ANSWER**

"Warehouse wanted per-component LOC on every invoice line; salespeople wanted the invoice clean. Both customers, both valid.

I solved it with two render targets reading from the same data: `/invoice/:slug` is the customer-facing invoice with one clean package row; `/internal-invoice/:slug` is the warehouse view with per-component LOC rows expanded. Same `form_json`, different `*Invoice.jsx` files (`InvoicePreview.jsx` vs `InternalInvoice.jsx`)."

Probing chain:

Q	A
Why not just two checkboxes on one page?	Each audience has a different mental model. Single page with toggle puts cognitive load on the user. Two URLs makes the affordance physical — print this one, send that one.
What if requirements diverge further?	The data layer stays one; the render layer can split as many ways as needed. Adding a third audience is a third file, not a third data shape.

Q2.3 — "Tell me about a disagreement with a non-engineer."

STRONG ANSWER

"The proposal to remove the closing-ratio 100% cap. Argument was 'math should just work.' I pulled three example rows showing what each looked like with cap vs without. One had a 500% closing ratio that was meaningless. We landed on: keep the cap (it's the business definition), but remove the upstream double-counting (the real bug). The cap stopped being load-bearing and started being intentional."

Probing chain:

Q	A
<i>How did you persuade?</i>	Three example rows. Not principles, not best-practices — three rows you could read off a screen.
<i>What if they still didn't agree?</i>	I'd have asked the GM to choose. Engineering disagreement with a stakeholder is escalated to the customer of the decision, which is the GM.

3 FAILURE QUESTIONS

Q3.1 — "Tell me about a time you failed. What did you learn?"

They're probing: Self-awareness + second-order lesson.

STRONG ANSWER

"First version of the package-component dialog (May 20) supported one instance per item per package. Then I tested with a real 4-chair package — `ItemPackageMaster` had four rows for the same chair `item_id`, one per piece. My schema couldn't represent 'chair #1 → S1, chair #2 → S2.'"

I rewrote with `instance_index`. The lesson isn't 'look at the data first' — that's the obvious lesson. The lesson is: when I'm modeling a new domain, the schema should be as wide as the worst-case data shape on day one, even if v1's UI only exposes the simple case. Cheap to widen the schema on day one; expensive to migrate later."

THE TRAP: Fake failures ("I took on too much"). Real failures have an artifact.

Probing chain:

Q	A
How did you discover the gap?	First real package I tested. <code>ItemPackageMaster</code> had four rows for the same chair.
Could you have known earlier?	Yes. One query: <code>SELECT item_id, COUNT(*) FROM ItemPackageMaster GROUP BY item_id ORDER BY 2 DESC</code> would have flagged multi-instance packages on day one.
Was anyone affected?	Internal — I caught it in staging before any user saw it. But the schema migration the next week was avoidable cost.
What did the team learn?	"First query before schema" became my rule. I added it to my project memory.
What if you had shipped the broken version?	Warehouse would have picked the wrong number of pieces. Recovery would have required reconciling with customers post-delivery — much more expensive than a staging-stage catch.

STRONGER PHRASING

"Failure with a generalizable lesson and a concrete artifact. 'Run the worst-case query before designing the schema' is my new rule."

Q3.2 — "Tell me about a bug you shipped to production."

STRONG ANSWER

"The `/finish-up/start` race. Both `/ups/finish` and `/finish-up/start` could open an `ups_finish_logs` row, leaving two open rows for the same user. Showed up as a duplicate salesperson in the 'Finishing Up' section of the board.

I traced it by reading the route handlers side-by-side. Fix was making `/ups/finish` the only atomic opener. `/finish-up/start` became a fallback used only when `localStorage` state had drifted. Also added `clearAllFinishingUp(workflowType, fallbackUserId)` so even with an empty `localStorage` map, the end call still closes the right row by `(userId, ended_at IS NULL)` lookup."

Probing chain:

Q	A
Why did you have two endpoints opening the same row?	Original design used <code>/finish-up/start</code> for the dialog open; later refactor moved opening into <code>/finish</code> atomically. The orphaned start endpoint wasn't removed.
What was the blast radius?	Visible duplicates on the board but no data corruption. Hours of UX confusion.
How did you catch it?	Manager called out the dup on a Tuesday. I had the fix that afternoon.

Q3.3 — "Tell me about a time you missed a deadline."

STRONG ANSWER

"First version of the executive email body before the May 16 redesign. I shipped a single-table format thinking it would be readable. The GM was reading on mobile and skipping rows — effectively missing the deadline-of-being-useful even though the email itself shipped on time.

I split into `renderExecutiveEmailHtml` with three sections (Combined, Arden, Waynesville) and KPI cards at the top. PDF stayed unchanged. Lesson: I designed for the data, not the consumption channel."

Probing chain:

Q	A
<i>Why didn't you ask before designing?</i>	I should have. I assumed "table" was the right shape because that's how the GM verbally described it. He meant 'I want to see the numbers,' not 'I want a table.'

4 TRADEOFF QUESTIONS

Q4.1 — "Walk me through a hard tradeoff."

STRONG ANSWER

"Auth model. Body `userId` is trusted, no JWT, no cookies, no session middleware. Sacrificed defense-in-depth against `userId` forgery for simplicity and speed on a LAN-only deployment.

To compensate, `requireRole.js` re-reads the role from `cfc_users` on every request. Forged `userIds` can still impersonate, but they cannot escalate to admin/manager via a stale local role. Sensitive operations (`/abandoned-up` , `/update-ups` , user CRUD) double-check role in the route.

If this were public-internet, I'd reach for HMAC-signed tokens. Refactor is small because every route already goes through `req.cfcUser`."

THE TRAP: Defending as "the right answer." Tradeoffs have downsides — name yours.

Probing chain:

Q	A
What would convince you to change?	Going public-internet, regulated industry, or a single incident of internal misuse.
What's the threat model that's uncovered?	An authenticated employee could forge another employee's userId and act as them. Detection would come from <code>ups_logs.username</code> not matching the device's localStorage user.
Is the detection real?	Yes — <code>ups_logs</code> writes username separately from action data. Cross-check is one query.

Q4.2 — "Why MySQL + MSSQL? Why not consolidate?"

STRONG ANSWER

"Two databases on purpose. MSSQL is the legacy ERP — owns inventory (`ItemMaster` , `PackageMaster` , `ItemPackageMaster` , `InvMasterReport`). We don't own writes. Migrating it would require a yearlong project to replace the buyer/inventory team's tools.

MySQL is operational data we own end-to-end. The cost of two databases is cross-DB joins at the app level (`runSql` for MSSQL, `formQuery` for MySQL, join in Node). The hot read paths are single-row lookups, so latency is fine."

Probing chain:

Q	A
How do you maintain consistency?	For PKG components: source-of-truth for components is MSSQL (<code>ItemPackageMaster</code>); source-of-truth for assignments is MySQL (<code>pkg_component_locations</code>). One-way relationship — if MSSQL removes a component, the MySQL row is orphaned but never returned (lookup starts from MSSQL).
What if MSSQL goes down?	Inventory and package fetches fail. Operational features (queue, audit, reports) continue. Acceptable degradation.

5 PRIORITIZATION QUESTIONS

Q5.1 — "You have three high-priority tasks. How do you sequence?"

STRONG ANSWER

"Concrete from May 20 week: PKG component location dialog, Abandoned UP, daily summary refinements. I sequenced by **blast radius and reversibility**:

1. PKG dialog first — biggest user impact (warehouse), longest UX iteration lead time, protected by `PKG_COMPONENT_LOC_ENABLED` so reversible.
2. Abandoned UP second — clear scope, short build time, reuses the Be Back transition pattern.
3. Daily summary tweaks last — incremental, low-risk, ships when ready.

Spend uncertainty budget on the highest-impact / hardest-to-reverse work first, while you still have the calendar to fix mistakes."

THE TRAP: Pick by "what's hardest" or "what's easiest." Bar raisers want a *framework*.

Probing chain:

Q	A
What if all three were urgent?	Negotiate scope. Each gets a "v0.5" version that ships within the same window. Reversibility says PKG must have the flag from day one.
Who decides priority if you can't?	The GM. Engineering's job is to surface the tradeoff with concrete options.

Q5.2 — "When do you refactor vs patch?"

STRONG ANSWER

"I refactor when the patch *count* tells me the model is wrong. Three `LEAST(closing_ratio, 100)` clamps in three services was three patches compensating for one upstream bug — the signal to refactor (remove Be Back metric writes), not patch (add a fourth clamp).

I patch when the model is right and one specific case slipped. Adding NS1/NS2 nailed-location handling to the manager-approval check was a patch; the model (manager-approval reasons in `systemNotes`) was already right."

Probing chain:

Q	A
Always refactor when there are multiple patches?	No — sometimes the patches are independent. The heuristic is "are they compensating for the same upstream cause?" If yes, refactor.

6 TECHNICAL TRADEOFFS

Q6.1 — "Why advisory lock vs Redis vs application-level mutex?"

STRONG ANSWER

"Single MySQL instance, two stores. MySQL's `GET_LOCK` is per-store, gives us mutual exclusion across processes, holds the connection for the transaction lifetime, and releases atomically with commit/rollback. Redis would add an operational dependency (a new piece of infrastructure to monitor and back up) for marginal benefit at our scale. Application-level mutex would only work if all writes went through one Node process — which isn't guaranteed."

Probing chain:

Q	A
What if MySQL has multiple primaries (Galera, etc)?	<code>GET_LOCK</code> is per-connection-per-node. Wouldn't work cross-node. We'd reach for Redis or a coordination service.
What's the cost of <code>GET_LOCK</code> holding a connection?	Connection pool pressure. With 2 stores and burst contention it's negligible; at 10 stores I'd raise <code>connectionLimit</code> proportionally.

7 PRODUCTION INCIDENTS

Q7.1 — "Walk me through a production-like issue you handled."

STRONG ANSWER

"May 23, 2026. Day three after PKG component location rollout. An issue surfaced on the live floor. I had two options: investigate first, or pull the feature flag.

Flipped `PKG_COMPONENT_LOC_ENABLED = false` in `cfc/src/utls/packageWarnings.js`. 30 seconds. No schema rollback. Saved data in `pkg_component_locations` preserved. Dialog stopped mounting; invoice rows reverted to pre-May-20 behavior. Floor was unaffected within minutes.

Then I had the rest of the day to investigate calmly. The critical preparation was building the flag *with* the feature, not after the fire."

Probing chain:

Q	A
<i>Wasn't this a workaround?</i>	Yes — workaround now, fix later. Asymmetry of cost: stuck floor costs revenue; unnecessary flag-off costs an hour.
<i>How do you balance flag-off vs forward-fix?</i>	Stuck floor + flag exists → flag off. Containable + no flag → forward fix. The flag has to exist for the choice to exist.
<i>Did you tell anyone before flipping?</i>	The on-floor manager via Slack as I flipped — not asking permission, informing. Reversibility makes "ask forgiveness" the right move; irreversibility makes "ask permission" the right move.

8 DEBUGGING PRESSURE

Q8.1 — "You're paged at 11 PM. Daily report didn't go out. Walk me through your first hour."

STRONG ANSWER

"First five minutes — read `report_job_history` for tonight's date. Three possibilities:

1. No row → cron didn't fire. Check cron logs. Likely process didn't start.
2. Row with `status='running'` → process started but didn't finish. Read `error_message`. Likely a Puppeteer crash or DB connectivity blip.
3. Row with `status='succeeded'` but recipients didn't get email → check `report_delivery_logs.status`. Likely Gmail rate limit on bulk send.

Next ten minutes — depending on which branch, take the targeted action:

- For (1): manually invoke `runDailyReport({ triggerSource: 'manual', force: true })`.
- For (2): same as (1), but check for the underlying cause first.
- For (3): retry just the failed recipients; the PDF and Drive upload already succeeded.

The whole story is in three append-only audit tables. I don't have to grep logs."

Probing chain:

Q	A
What if the audit tables themselves are corrupt?	They're append-only. Corruption is unlikely. If they're gone, schema migration is incomplete — re-run <code>applyStartupMigrations()</code> .
What if Puppeteer is the issue every night?	Move PDF rendering to a long-running side process; cron triggers it via HTTP. Decouples the renderer's stability from the cron.
No alarms — would you add them?	Yes: <code>report_job_history.status='failed'</code> count > 0 in last 24h, recipient-failure rate > 30% in last week. CloudWatch / Datadog if available; Slack webhook fallback if not.

9 INCOMPLETE REQUIREMENTS

Q9.1 — "PM gives you 'we need fraud detection on this feature.' What do you do?"

STRONG ANSWER

"First, define fraud specifically. 'Self-approval' is one form (manager approving their own UP). 'Threshold abuse' is another (10+ abandons per month). 'Pattern abuse' is a third (always abandoning on Saturday afternoons).

For Abandoned UP I built two: self-approval blocked at service level (explicit rejection); threshold abuse flagged non-blocking via `computeAbuseFlag(salespersonId)` against env `ABANDONED_UP_ABUSE_THRESHOLD` . Surfaced in API response and daily PDF.

Pattern abuse I deferred — needs longer history and a clear signal. Added to the backlog with a note."

Probing chain:

Q	A
Why non-blocking on threshold?	False positive on threshold day shouldn't block legitimate operations. Detection ≠ enforcement.
What if pattern abuse becomes real?	The <code>abandoned_up_logs</code> table already has <code>business_date</code> , <code>approved_by_id</code> , <code>reason</code> . The data shape supports any future detection.

10 CUSTOMER-IMPACT QUESTIONS

Q10.1 — "How did you measure the impact of something you shipped?"

STRONG ANSWER

"Abandoned UP: counted abuse flags. `computeAbuseFlag(salespersonId)` counts current-calendar-month abandons; threshold default 10. Two flags surfaced in the first two weeks — both legitimate, both surfaced in the daily PDF, both addressed by management. The feature was working *and* the safeguard was working.

The fairness fix on the queue was harder to measure quantitatively — I leaned on user feedback. The day after the fix, no salesperson complained about being penalized for greeting non-customers. Absence-of-complaint is a weak signal but a real one."

Probing chain:

Q	A
<i>Is absence-of-complaint enough?</i>	No — but it's data. I'd combine it with <code>abandoned_up_logs</code> volume (decreased? increased?) and qualitative GM feedback.
<i>What metric would you build if you could?</i>	"Time-spent-on-floor / UP" — a productivity metric that decouples from queue position.

Q10.2 — "Tell me about a feature that didn't have the impact you expected."

STRONG ANSWER

"First executive email body (pre-May 16). Single-table format. The GM was on mobile, skipping rows. I assumed table = readable; data said no. Redesign on May 16: three sections (Combined / Arden / Waynesville) with KPI cards at the top. PDF unchanged because the GM uses email body in meetings and PDF for archiving."

11 SCALING PRESSURE

Q11.1 — "What breaks first at 10× current load?"

STRONG ANSWER

"Three predictable failure modes:

1. **The 1-second `useUPSQueue` poll.** Five endpoints per poll, ~6 devices per store. At 10 stores → 300 reqs/sec for state that's mostly unchanged. Mitigation: SSE feed off `ups_logs` inserts; fall back to poll on disconnect.
2. **MSSQL inventory reads.** Same item lookups hit on every package fetch and item search. Mitigation: in-process LRU + 60s TTL per host.
3. **MySQL pool exhaustion.** `GET_LOCK` holds a connection for the transaction. At burst contention across 10 stores, the pool can run out. Mitigation: per-store connection pool sizing + raise `connectionLimit`."

Probing chain:

Q	A
What doesn't change at 10x?	The fairness algorithm. Per-store advisory lock is sharded by store, so adding stores is linear.
What's the single biggest engineering investment?	SSE/WebSocket. The 1-second poll wastes the most resources at scale.

1 2 DEADLINE TRADEOFFS

Q12.1 — "You have one week. Fix tech debt or ship a feature. What do you do?"

STRONG ANSWER

"Depends on the debt and the feature. The deciding question is 'what's the cost of *not* doing this?'

If the debt is making every PR slower (three closing-ratio clamps compensating for an upstream bug), the cost of ignoring is compounding — fix it. If the debt is a typo'd filename (`InternalInvoice.jsx`), the cost of ignoring is zero — ship the feature.

Most weeks the answer is 'a little of both' — bundle a small debt fix into the feature PR. Last week of fiscal quarter? Feature wins by default."

1 3 AGGRESSIVE PROBING TEMPLATES (rehearse these answers)

PROBE	TEMPLATE ANSWER
Why did you choose that approach?	"I considered [other] and rejected it because [concrete cost]. The chosen approach has [benefit] at the cost of [downside], which was acceptable because [context]."
What would you do differently?	"On reflection, [specific thing I'd do earlier]. The reason I didn't was [honest reason]. The cost was [observable cost]."
What metrics improved?	"Before: [observable behavior]. After: [observable behavior]. Mechanism: [code change]."
How did you know it was root cause?	"Removing the root cause removed downstream compensation. If I'd fixed a symptom, the others would have remained needed."
How did you handle disagreement?	"Brought [concrete artifact: data, example, scenario]. Landed on [resolution]. If I'd lost, I'd have [specific commit action]."
Biggest risk?	"Worst case: [scenario]. Protected by [specific mechanism]."
What was your biggest mistake?	"[Concrete artifact of the failure]. Lesson: [generalizable rule]. Application: [where I use it now]."
Why was this production safe?	"Reversible because [flag/env/schema decision]. Observable because [audit row]. Idempotent because [pattern]."

1 4 CURVEBALL QUESTIONS

Q14.1 — "If we deleted your most-loved feature tomorrow, what would the system look like?"

STRONG ANSWER

"Delete the package-component location dialog. Saved rows in `pkg_component_locations` stay (re-enable is one flag flip). The invoice goes back to one row per package, exactly the pre-May 20 experience. Warehouse loses per-component picking info; they revert to reading the package description.

I'd be sad but the system wouldn't break. That's actually the test I aim for when I build a feature — can it be deleted cleanly? If no, the design coupled too tightly."

Q14.2 — "Tell me about the worst code you wrote in this project."

STRONG ANSWER

" InvoicePreview.jsx has too many // 🔥 flow markers and the page-break-driven PDF layout is intricate. It's the right shape for the use case (one source of truth for screen + print + Adobe eSign tags via HeadlessChrome detection), but a junior reader spends a while figuring out which branch fires when.

I'd refactor by extracting the Adobe eSign tag branch into a dedicated render helper, but I haven't because the file isn't changing weekly anymore."

Q14.3 — "Walk me through what your skip-level would say about you."

STRONG ANSWER

"He'd say I sometimes optimize for the small team I'm currently in. I've built audit tables, idempotency patterns, and feature flags that are right for two showrooms and one engineer. He'd push me to design for what the team will look like in two years — more engineers, more review, more documentation overhead.

What I'm doing about it: I write durable project memories that survive my own absence. Naming conventions, dated migration blocks, rollback flags — that's the 'second engineer onboarding' I'd need if the team grew tomorrow."

1 5 Bar Raiser Anti-Patterns (do not do these)

ANTI-PATTERN	WHY IT FAILS
Vague results ("users loved it")	Bar raisers will dig for a number.
"We" instead of "I"	They're hiring you, not your team.
No tradeoff	Every decision has a downside. Naming yours = maturity.
Failure theater ("I learned humility")	Real failures have an artifact.
Always-agreed	Pick one disagreement that resolved cleanly.
Hindsight bias ("the answer was obvious")	Acknowledge what you didn't know at the time.
Apologizing as the lead	Lead with the diagnosis or the corrected number.
Defensive when probed	Treat probes as curiosity. Expand depth.



1

6

Pre-loaded depth questions (have answers ready for each STAR)

For every STAR you deliver, the Bar Raiser may dig into any of these:

1. What did you reject and why?
2. What was the most surprising thing you learned?
3. What would you build differently with hindsight?
4. What was the metric you cared about?
5. Who depended on this and how did you confirm they were unblocked?
6. What was the failure mode you didn't design for?
7. What's the next thing you'd ship on this?
8. Did you talk to anyone before deciding? Why or why not?
9. If your manager said "stop building this," what would you say?
10. What was the riskiest 30 minutes of this project?

Rule of thumb: if you have to think for more than 3 seconds, you don't know the story well enough. Rehearse.

**PART V**

TECHNICAL CROSS- QUESTIONING

31 Deep Probes · Excellent vs Weak Answers

COVERS

Technical Deep Dive Questions

Audience: an SDE-3 / staff-engineer interviewer who will probe one technical decision at a time. **Format per question:** Question · Excellent Answer · Weak Answer (so you know what NOT to say) · Concise Senior Version · What the Interviewer Expects.



Master question index

#	TOPIC	QUESTION
A1	Architecture	Why two databases (MySQL + MSSQL)?
A2	Architecture	Walk through the UPS schema.
A3	Architecture	How does <code>fairInsertV2</code> work?
B1	Concurrency	How is Be Back race-safe?
B2	Concurrency	How does Birdeye guarantee exactly-once?
B3	Concurrency	How does idle-lock re-auth handle races?
C1	Data Model	Why is <code>form_json</code> a LONGTEXT JSON blob?
C2	Data Model	How does <code>instance_index</code> handle a 4-chair package?
C3	Data Model	Walk through invoice dedup logic.
D1	Validation	Walk through the mandatory signature chain.
D2	Validation	How do you handle partial failures in the daily job?
D3	Validation	What's your error model?
E1	Performance	Where are the bottlenecks?
E2	Performance	How would you scale to 10 stores?
E3	Performance	Tell me about an optimization you made.
F1	Frontend	How does Redux fit into the app?
F2	Frontend	Walk me through Focus Flow.
F3	Frontend	How does the package-location dialog open?
G1	Operations	How do you test without dedicated QA?
G2	Operations	What's your observability story?
G3	Operations	How do you deploy?
H1	Specific Code	Show me where <code>withStoreTx</code> is implemented.
H2	Specific Code	How does the seller-split CTE fall back?
H3	Specific Code	What's the 12-employee schedule?
H4	Specific Code	What's the schema migration convention?
H5	Specific Code	What are the package allocator locations?

#	TOPIC	QUESTION
I1	System Design	Design a queue for 10 stores, 1000 visits/day.
I2	System Design	Design observability for this system.
J1	Code Quality	Show me code you regret.
J2	Code Quality	Where's the most over-engineered code?
J3	Code Quality	Where's the worst naming?

A ARCHITECTURE & DATABASE DESIGN

A1 — "Why two databases (MySQL + MSSQL)? Why not consolidate?"

EXCELLENT ANSWER

"MSSQL is the legacy ERP. It owns inventory — `ItemMaster`, `PackageMaster`, `ItemPackageMaster`, `InvMasterReport`. We don't own writes. Migrating it would mean a multi-quarter project to replace the buyer/inventory team's tools.

MySQL is the operational system I own end-to-end — `ups_queue`, `ups_logs`, `ups_sessions`, `invoices_archive`, `customer_acquisition`, `customer_leads`, `customer_rts_entries`, `abandoned_up_logs`, `reminders`, `pkg_component_locations`, `manager_store_sessions`, `report_*_triad`, `audit_results`.

The cost of two databases is real: cross-DB joins are app-level. Two pools (`cfcFormPool`, `mssqlPool`) with helpers (`formQuery`, `runSql`). Hot read paths are single-row lookups so latency is fine."

WEAK ANSWER

"We use MySQL for the app and MSSQL for inventory."

(No reasoning, no ownership context, no cost discussion.)

CONCISE SENIOR VERSION

"Constraint, not a choice. ERP is MSSQL and read-only. We own MySQL for operational state. Cross-DB joins are app-level — acceptable because hot reads are single-row."

WHAT THE INTERVIEWER EXPECTS

- You acknowledge it's a constraint, not a design.
- You can name what each DB owns.
- You can name the integration cost.
- You can describe a future migration path (don't claim to do it today).

A2 — "Walk me through the UPS schema."

EXCELLENT ANSWER

"Five tables in the queue layer:

1. `ups_queue` — waiting line. One row per (`user_id`, `store_name`), UNIQUE.
2. `ups_working_queue` — currently with customer. `active_customers` supports stacked floor mode.
3. `ups_logs` — append-only audit. 20+ `action_type` ENUMs (`ups_taken`, `queue_join`, `break_start`, `break_end`, `be_back`, `be_back_accept`, `moved_to_working`, `abandoned_up`, `ups_handed_off`, `queue_leave`). `reason` JSON. `idx_ups_logs_store_action_date` for day aggregations.
4. `ups_sessions` — per-day fairness state. UNIQUE on (`user_id`, `store_name`, `business_date`). `first_ups_at` is the eligibility flag — NULL until first take, then immutable for the day.
5. `ups_finish_logs` — paperwork timer. (`user_id`, `started_at`, `ended_at`, `duration_sec`, `workflow_type`).

The design separates **transient state** (`ups_queue`, `ups_working_queue` — wiped nightly by `dailyReset`) from **immutable audit** (`ups_logs`, `ups_sessions`, `ups_finish_logs` — never reset). Reports read from immutable audit; live board reads from transient. The 9 PM reset is safe by construction."

WEAK ANSWER

"There's a queue table and a log table."

CONCISE SENIOR VERSION

"Transient state in `ups_queue` + `ups_working_queue` (nightly reset). Immutable audit in `ups_logs` + `ups_sessions` + `ups_finish_logs`. Reports query audit; board queries state."

WHAT THE INTERVIEWER EXPECTS

- You can name every table without thinking.
- You explain *why* state vs audit are separated.
- You can describe the UNIQUE constraints and the role of `first_ups_at`.

A3 — "Walk through `fairInsertV2` step by step."

EXCELLENT ANSWER

"Inside `withStoreTx(store, async (conn) => ...)`, so we hold `GET_LOCK('ups_queue_lock:' || store, 5)` and are in a transaction.

1. Load today's queue: `SELECT user_id, queue_position, ups_count FROM ups_queue WHERE store_name=? AND DATE(joined_at)=CURDATE() ORDER BY queue_position FOR UPDATE`. The DATE filter is the stale-queue-ghost defense.
2. Load joiner's session: `SELECT first_ups_at, ups_count FROM ups_sessions WHERE user_id=? AND store_name=? AND business_date=CURDATE() FOR UPDATE`.
3. Empty queue → joiner at position 1.
4. `first_ups_at IS NULL` → append at tail (ineligible).
5. Eligible → scan from position 2 forward. Insert before first incumbent whose `ups_count > joiner.ups_count`.
6. Position 1 never overwritten — `recycle` is the only way to move it.
7. Compress positions 1..N.
8. Write `ups_logs.action_type='queue_join'`.

The eligibility gate is the key insight. It separates 'I'm new today' from 'I belong in rotation by ups count.'"

WEAK ANSWER

"We sort by `ups_count`."

CONCISE SENIOR VERSION

"Lock + FOR UPDATE on today's rows. Empty → pos 1. Ineligible (`first_ups_at IS NULL`) → tail. Eligible → insert before first incumbent with higher `ups_count`. Compress and audit."

WHAT THE INTERVIEWER EXPECTS

- You explain the eligibility rule and why it exists (anti-leapfrog).
- You can defend the `DATE(joined_at) = CURDATE()` filter (stale-ghost defense).

- You acknowledge position 1 is privileged (manager recycle only).

B CONCURRENCY, RACES, IDEMPOTENCY

B1 — "Walk through how Be Back is race-safe."

EXCELLENT ANSWER

"Single transaction in `executeOneClickHandoff` :

1. Verify origin is in `ups_working_queue` (`FOR UPDATE`).
2. Verify selected is in waiting or working with `active_customers < 6` (`FOR UPDATE`).
3. Remove origin from working; decrement `ups_count` .
4. If selected in working → +1 `active_customers`, +1 `ups_count`. If in waiting → delete from waiting, insert into working, +1 `ups_count`.
5. Reinsert origin at position 1 of waiting.
6. Compress waiting positions.
7. Write three audit rows: `be_back` , `be_back_accept` , `moved_to_working` .

Inside `withStoreTx` so it's serialized against concurrent `/take` , `/finish` , `/break` , `/abandoned-up` , other `/be-back` on the same store. Selected employee can only be in one state at a time — guaranteed by `FOR UPDATE` reads."

WEAK ANSWER

"We use a transaction."

CONCISE SENIOR VERSION

"Per-store advisory lock + `FOR UPDATE` on both origin and selected rows. Atomic transition. Three audit rows. Does NOT write `ups_taken` / `ups_handed_off` since 2026-05-21 — Be Back is routing, not a metric event."

WHAT THE INTERVIEWER EXPECTS

- You can name the lock primitive and the `FOR UPDATE` requirement.
- You know the May 21 metric rule (Be Back doesn't write `ups_taken`).
- You acknowledge cross-store handoff is rejected at the route.

B2 — "How does Birdeye SMS guarantee exactly-once?"

EXCELLENT ANSWER

"Atomic boolean claim:

```
UPDATE invoices_archive SET birdeye_sent = 1
WHERE slug = ? AND birdeye_sent = 0
```

Check `affectedRows`. Only `affectedRows === 1` (the winner) sends the SMS. On send failure, roll back:
`UPDATE ... SET birdeye_sent = 0` so a retry can re-claim.

This is compare-and-swap implemented in SQL. The database is the lock; `affectedRows` is the success signal. No application lock, no Redis, no message queue. The `birdeye_sent` column doubles as a support flag."

WEAK ANSWER

"We use a transaction with SELECT FOR UPDATE."

(Works but holds a row lock across the multi-hundred-ms HTTP call — the atomic UPDATE pattern releases the lock instantly.)

CONCISE SENIOR VERSION

"CAS-in-SQL. Atomic UPDATE with predicate; `affectedRows === 1` is the winner. Roll back on SMS failure. No external infra."

WHAT THE INTERVIEWER EXPECTS

- You name the pattern (compare-and-swap).
- You explain why this beats `SELECT FOR UPDATE` (no row-lock held across HTTP).
- You handle the failure case (rollback so retry can re-claim).
- You acknowledge that exactly-once is a *cooperative* property — Birdeye also dedups on their side.

B3 — "How does the 5-minute idle lock handle a re-auth race?"

EXCELLENT ANSWER

"Three guards in `cfc/src/AUTH/authContext.jsx` :

1. **Queued action snapshot.** When an action is taken while locked, we store `pendingActionRef = { run, queuedAsUserId: user?.id, queuedAt }`.
2. **Session-takeover guard.** On unlock, the response can be `{ switched: true, userId, user }` — a different employee unlocked the same device. If `pending.queuedAsUserId !== res.userId`, the action is discarded with a toast.
3. **TTL expiry.** `Date.now() - pending.queuedAt > 60_000` → discard. If the user walked away and someone unlocked 90s later, the queued action is stale.

Cross-tab sync via `storage` event listener on `cfc_locked_at`. Managers and admins are not locked — `cfc_locked_at` is purged defensively on non-employee mount."

WEAK ANSWER

"We re-check the password."

CONCISE SENIOR VERSION

"Identity snapshot + 60s TTL + cross-tab `storage` event. Verify-password endpoint has no side effects. Different unlocker → discard queued action."

WHAT THE INTERVIEWER EXPECTS

- You name all three guards (identity, TTL, storage event).
- You explain why verify-password has no side effects (no commit, just identity check).
- You acknowledge the shared-kiosk threat model.

DATA MODELING & API DESIGN

C1 — "Why is `form_json` a LONGTEXT JSON blob instead of normalized columns?"

EXCELLENT ANSWER

"Tradeoff between flexibility and queryability. Chose flexibility because:

1. The invoice has ~80 fields including 6 customer signatures (base64), nested items with package components and multi-location allocations, payments object with 8 methods, delivery block, RTS subtree.
2. The shape changes often (we added care plan, then `taxOverride`, then package components, then RTS). Schema migrations for each would be expensive.
3. The customer-facing invoice is rendered from this JSON directly — keeping it together preserves the 'what the customer signed' guarantee.

Where queryability matters, we use `JSON_EXTRACT`. The dedup query is `JSON_UNQUOTE(JSON_EXTRACT(form_json, '$.billTo.name1'))`. Slower than a column but bounded to today's rows."

WEAK ANSWER

"JSON is easier to work with."

CONCISE SENIOR VERSION

"Form shape churns every sprint. JSON survives churn. `JSON_EXTRACT` on dedup is bounded to today's rows. Functional indexes would help if we grew 100×."

WHAT THE INTERVIEWER EXPECTS

- You acknowledge the tradeoff (queryability vs flexibility).
- You name the rescue pattern (functional indexes, MySQL 8) if scale demands it.
- You preserve the integrity argument (what was signed = what's stored).

C2 — "How does `instance_index` handle a 4-chair package?"

EXCELLENT ANSWER

"Schema:

```
CREATE TABLE pkg_component_locations (  
  package_id INT,  
  item_id VARCHAR(...),  
  instance_index INT,          -- 0..N-1 for N pieces  
  location_id VARCHAR(...),    -- S1/S2/999/NS1/NS2/SO  
  saved_by_user_id INT,  
  UNIQUE KEY (package_id, item_id, instance_index)  
);
```

For a 4-chair package, four rows: (pkg, chair, 0, S1), (pkg, chair, 1, S1), (pkg, chair, 2, S2), (pkg, chair, 3, 999).

The dialog shows 4 separate rows. The qty parser still returns `required_qty = 4` for the chair component; the validator checks `SUM(loc qty across all 4 instances) == 4`.

WEAK ANSWER

"We have a row per chair."

CONCISE SENIOR VERSION

"`UNIQUE(package_id, item_id, instance_index)` . Four rows for four chairs, each with independent location. UPDATE is row-wise; concurrency clean."

WHAT THE INTERVIEWER EXPECTS

- You can name the unique key.
- You can defend per-instance rows vs JSON-array-of-locations (UPDATE concurrency).

C3 — "Walk through invoice dedup on `/invoice/save`."

EXCELLENT ANSWER

```

""`js const [existing] = await formQuery( SELECT id FROM invoices_archive WHERE
JSON_UNQUOTE(JSON_EXTRACT(form_json, '$.billTo.name1')) = ? AND
JSON_UNQUOTE(JSON_EXTRACT(form_json, '$.billTo.cell')) = ? AND DATE(created_at) = CURDATE()
LIMIT 1 , [custName, custCell]);

```

```

if (existing) { await formQuery( UPDATE invoices_archive SET form_json=?, updated_at=NOW() WHERE
id=?, [json, existing.id]); } else { await formQuery( INSERT INTO invoices_archive (slug, ...,
form_json, ...) VALUES (?, ..., ?, ...), [...]); }

```

Natural key for 'ticket today' is `(customer\_name, customer\_cell, business\_date)`.
 JSON_EXTRACT instead of materialized columns because the natural key matured over time
 and the JSON has always had `billTo.cell`."

WEAK ANSWER

"We do an upsert."

CONCISE SENIOR VERSION

"UPSERT-by-natural-key via JSON_EXTRACT. Bounded to today's rows. Two simultaneous saves for a brand-new customer could both INSERT — hardened version uses a MySQL 8 functional index on the extracted fields."

WHAT THE INTERVIEWER EXPECTS

- You can name the natural key.
- You name the race (two simultaneous INSERTs for new customer) and the future hardening (functional index).
- You catch the NY-timezone bug (DATE(created_at) uses server time — should be CONVERT_TZ).

📌 VALIDATION, ERROR HANDLING

D1 — "Walk through the mandatory signature chain."

EXCELLENT ANSWER

" `cfc/src/components/right/validateInvoiceSignatures.js` returns an error array. Rules in order:

TRIGGER	REQUIRED SIGNATURE
Always	store location, ≥ 1 item, every item <code>salePrice > 0</code> + (<code>extPrice</code> OR applied package)
RG + any TW item	<code>quantityAndPriceApprovalSignature</code> (customer)
RG + <code>carePlan="Yes"</code>	<code>carePlanCharge > 0</code> , <code>carePlanApprovalSignature</code> (customer)
RG + <code>cash > 0</code>	<code>managerApprovalForCashSign</code> (manager)
RG + <code>balanceDue === total</code>	<code>managerApprovalForCashSign</code> (manager)
RG + items from NS1/NS2	<code>managerApprovalForCashSign</code> (manager)
RG	<code>saleApprovalSignature</code> , <code>paymentSummaryApprovalSignature</code> , <code>policiesSignature</code> (all customer)
PH + <code>cash > 0</code>	<code>managerApprovalForCashSign</code> (manager). No customer sigs on screen — Adobe eSign tags in PDF only via HeadlessChrome detection.
QS	none
<code>storeLocation === 'store5'</code>	exempt

Returns `{ valid, errors[] }` . 'Watch Invoice' button disabled when `errors.length > 0` ; toast shows first error."

WEAK ANSWER

"We check that signatures exist."

CONCISE SENIOR VERSION

"Pure rule-engine function. Returns errors array. Gated by invoice type, payment, balance, item attrs (TW, NS1/NS2), care plan opt-in, role. PH pushes sigs to PDF via Adobe eSign tags."

WHAT THE INTERVIEWER EXPECTS

- You can recite at least 4 distinct signature triggers.
- You explain why so many (legal/financial/logistics specificity).

- You name the store5 exemption.

D2 — "How does the daily job handle partial failures?"

EXCELLENT ANSWER

"Per-recipient sends are independent. Orchestrator builds the PDF and Drive upload once, then iterates recipients:

```
for (const email of recipients) {  
  try {  
    const result = await gmailMailer.sendEmail({ to: email, ... });  
    await logDelivery({ recipient_email: email, status: 'sent', provider_message_id:  
result.id });  
  } catch (err) {  
    await logDelivery({ recipient_email: email, status: 'failed', error: err.message });  
  }  
}
```

Drive failure is best-effort — email goes out, Drive link omitted. Up to 3 attempts at the job level; per-recipient retries happen on the next run. `report_job_history.status='running'` from a dead process doesn't block retry — `isAlreadySucceeded` only matches `succeeded`."

WEAK ANSWER

"We try/catch each email."

CONCISE SENIOR VERSION

"Per-recipient independence. Drive best-effort. Job-level retries with `force: true`. Idempotency via `isAlreadySucceeded(business_date)` matching only `succeeded`."

D3 — "What's your error model on the backend?"

EXCELLENT ANSWER

"Two layers:

- **Structured UpsError** for UPS routes with machine-readable codes: `LOCK_TIMEOUT`, `DEADLOCK`, `NOT_FOUND`, `BAD_INPUT`, `DB_ERROR`. `sendUpsError(res, err)` maps to HTTP status + `retryable: bool`. Frontend honors retryable with backoff.
- **Generic try/catch on other routes** returning JSON `{ ok: false, error: 'message' }` with 4xx/5xx. Less rigorous but consistent.

For the daily job, errors are written into `report_job_history.error_message` so post-mortems don't require log file access."

WHAT THE INTERVIEWER EXPECTS

- You acknowledge the inconsistency (two layers) honestly.
- You name what's missing (correlation id per request).

E PERFORMANCE, SCALABILITY

E1 — "Where are the bottlenecks today?"

EXCELLENT ANSWER

"Three I can name:

1. **1s useUPSQueue poll**. Five endpoints per poll, ~6 devices per store → ~30 reqs/sec per store. Wasteful but manageable. Replacement: SSE off `ups_logs` inserts.
2. **MSSQL inventory reads**. `InvMasterReport` lookups on every package fetch and item search. In-process cache 60s would help; read replica or Redis would be the next step.
3. **JSON_EXTRACT on invoices_archive for dedup**. Bounded to today's rows so cheap today. Functional indexes on `(name1, cell, DATE(created_at))` would help at 100x."

WEAK ANSWER

"We don't have any."

CONCISE SENIOR VERSION

"Polling overhead (replace with SSE), MSSQL inventory cache miss (in-process LRU), JSON_EXTRACT dedup (functional index at scale)."

E2 — "How would you scale to 10 stores?"

EXCELLENT ANSWER

"The hot path is per-store advisory locks — queue layer scales horizontally already. What changes:

1. **MySQL pool tuning.** `GET_LOCK` holds a connection; raise `connectionLimit` to $\sim 10\times$ minimum, or per-store pools.
2. **Daily summary fanout.** Sequential per-store today; parallelize with `Promise.all`. Puppeteer ceiling ~ 5 PDFs/sec.
3. **WebSocket / SSE** to replace 1s poll. 60 devices $\times$ 5 endpoints $\times$ 1 Hz across 10 stores = 3000 reqs/sec for state mostly unchanged.
4. **MSSQL caching.** Same item lookups happen across stores. Per-host cache layer.
5. **Composite indexes.** `ups_logs(store_name, business_date, action_type)` and `invoices_archive(store_name, created_at)` for store-scoped reports.
6. **Partition `ups_logs` by `business_date`** to keep day-range scans bounded.

What doesn't change: fairness algorithm. Single-store shard means adding stores is linear."

E3 — "Tell me about an optimization you made."

EXCELLENT ANSWER

"Two real ones:

1. **`idx_ia_created_at`** (May 19, 2026) — day-range list scan $O(n) \rightarrow O(\log n)$ for the internal read API. Migration block in `schema.sql` with deploy comment.
2. **Optimistic lock in `useUPSQueue`** — 1s poll was overwriting in-flight mutation results, causing brief UI ghosts. Added `optimisticLockUntilRef = Date.now() + 2000` set just before mutation; poller's `setState` noops until expired. Explicit refetch after mutation resolves."

F FRONTEND STATE, REACT PATTERNS

F1 — "How does Redux fit into the app?"

EXCELLENT ANSWER

"Single slice (`formSlice.js`) holds the invoice + RTS form state. `redux-persist` to `localStorage` so a Topaz disconnect or a refresh during a multi-page invoice doesn't lose customer input.

`updateField({ section, field, value })` is the central reducer — routes through `FIELD_NORMALIZERS` keyed by `section.field`. Phone formats on dispatch, not in input component. Currency rounds at the reducer level.

UPS data is **not in Redux** — `useUPSQueue` is a hook with its own state. `AuthContext` owns user + idle-lock + active-store. Rule I follow: Redux for *form state* (user input surviving routes); local hook state for *server-fetched live data*."

WEAK ANSWER

"We use Redux for state."

F2 — "Walk me through Focus Flow."

EXCELLENT ANSWER

" `cfc/src/components/left/comp/ui/` — `FocusFlowContext`, `FocusFlowProvider`, `FocusField`, `useFocusFlow`. The provider builds a tab order across registered field IDs; fields register on mount and unregister on unmount.

```
<FocusFlowProvider deps={[form.deliveryOption]}>
  <FocusField fieldId="salesperson" ref={spRef} />
  <FocusField fieldId="customer_name" ref={cnRef} />
</FocusFlowProvider>
```

`deps` invalidates the order when conditional fields appear/disappear (e.g., 'delivery_special' reveals a special-date input). `useFocusFlow()` exposes `focusNext(currentId)` and `focusPrev(currentId)`.

Why it exists: native tab order is DOM-based and doesn't skip conditionally-hidden fields cleanly. Plus we want Enter to advance like Tab in specific contexts (autocompletes)."

F3 — "How does the package-location dialog open?"

EXCELLENT ANSWER

"Flow in `LeftPanel.jsx` :

1. User selects a package.
2. `LeftPanel` calls `packageLocationApi.getPackageComponents(packageId)` → hits `GET /api/package/:packageId/components`.
3. Backend returns `{ packageId, package, components, componentGroups }`.
4. Frontend prefers `componentGroups`. If `required_qty` is `null` → 'qty unknown' state, disable Save with `g.qty_unknown_reason`.
5. `PackageLocationAssignmentDialog` mounts. Per-component card with multi-loc qty allocator. `validateManualFulfillment(required_qty, allocations)` on every keystroke.
6. Save → `POST /api/package/component-location` with `{ packageId, allocations }`. Backend writes per `(package_id, item_id, instance_index)` row.

Feature flag `PKG_COMPONENT_LOC_ENABLED` (in `cfc/src/utils/packageWarnings.js`) gates dialog mount and the parent/child row rendering on invoices."

🔗 TESTING, OBSERVABILITY, OPERATIONS

G1 — "How do you test something this complex without dedicated QA?"

EXCELLENT ANSWER

"Three lines of defense:

1. **Schema as test scaffolding.** `UNIQUE / FK / CHK / ENUM` constraints catch malformed writes. `CHK_rts_returned_pos`, `CHK_rts_written_pos` reject negatives at DB. `ENUM` on `ups_queue.status` rejects typos.
2. **Idempotency by design.** `isAlreadySucceeded`, atomic boolean claim — re-running is safe while testing.
3. **Audit tables as proof.** `report_job_history.duration_ms` + `report_delivery_logs.status` tell me the system worked when no one was looking.

What I don't have: unit test suite for routes. Honest gap. Pure-logic services (`packageQtyParser`, `attendanceException`, `employeeSchedule`) are pure functions — easy to test, where I'd add Jest first."

WEAK ANSWER

"We do manual testing."

WHAT THE INTERVIEWER EXPECTS

- You acknowledge the gap (no automated route tests).
- You name what mitigates it (schema constraints, idempotency, audit tables).
- You name what you'd add at Amazon (Jest on pure services first).

G2 — "What's your observability story?"

EXCELLENT ANSWER

"Append-only audit tables:

- `ups_logs` — every user action with reason JSON.
- `report_job_history` + `report_delivery_logs` + `report_drive_upload_logs` — daily job audit.
- `manager_store_sessions` — manager store picks per day.
- `abandoned_up_logs` — fraud trail with approver context.
- `audit_results` — ticket-vs-RV reconciliation comparisons.

Monitoring: I rely on the daily PDF itself — if email arrives, pipeline worked. If not, check `report_job_history`. No APM yet (would add at Amazon).

What I'd add first: request-id correlation across `ups_logs` rows for the same atomic operation. `be_back`, `be_back_accept`, `moved_to_working` are written by the same transaction but currently uncorrelated. One column."

G3 — "How do you deploy?"

EXCELLENT ANSWER

"Backend has `applyStartupMigrations()` in `config/initSchema.js` for idempotent DDL on boot. Bigger/risky migrations (like `idx_ia_created_at`) are dated blocks in `schema.sql` with 'run this while deployment' — ops applies them manually before code release. `backend/admin-deploy/` has the deploy scripts. Frontend deploys are static-built with Vite.

Rollback strategy:

- Feature-flag-able changes → env var or in-code flag (e.g., `PKG_COMPONENT_LOC_ENABLED`).
- Behavior changes → tri-state env (`UPS_FAIRNESS_V2 = off | shadow | on`) — revert without code change.
- Schema additions: forward-compatible (new tables, new columns nullable, new indexes additive). Schema removals: never."

▣ SPECIFIC CODE PROBES

H1 — "Show me where `withStoreTx` is implemented."

`backend/src/services/upsFairness.js` lines 86–151. Acquires `GET_LOCK(name, 5)`, opens transaction on the same connection, runs the callback, commits or rolls back, releases the lock, returns the connection. Deadlock retry up to 2 attempts with a fresh connection.

H2 — "How does the seller-split CTE fall back?"

`backend/src/services/sales/ticketSeller.sql.js` exports `supportsJsonTable()` — feature-detect at startup. If false, callers fall through to `backend/src/services/sales/salesCredit.js` `expandTicketSellers(tickets)` — JS fan-out + division. Same output shape; callers don't care which path.

H3 — "What's the 12-employee schedule?"

`backend/src/config/employeeSchedule.js`, `RAW_OFF_DAYS` :

NAME	OFF
Chris Butler	Tue
Justin Reeves	Wed
Billy Terry	Tue, Thu
Evey Young	Mon, Wed
Reis McCall	Wed, Fri
Carol Terry	Tue, Thu
Missy Garvitte	Tue, Fri
Jon Weidenmiller	Tue, Fri
Joe Cole	Mon, Wed
Amy Heatherly	Thu, Sun
Donna Owle	Mon, Wed
Jeff Reece	Fri, Sat

API: `isEmployeeTracked`, `getScheduledOffDays`, `isEmployeeOffDay`, `weekdayForDate(dateIso, tz)`, `normalizeWeekday`, `normalizeEmployeeKey`.

H4 — "What's the schema migration convention?"

Every block in `backend/src/config/schema.sql` is dated and prefixed `"run this while deployment"`. Idempotent (`CREATE TABLE IF NOT EXISTS` / `CREATE INDEX IF NOT EXISTS`). This is in my project memory because I established it as a hard rule.

H5 — "Allowed package allocator locations?"

`ALLOWED_ALLOCATION_LOCATIONS = ['S1', 'S2', '999', 'NS1', 'NS2', 'SO']`. S1 = Arden, S2 = Waynesville, 999 = warehouse, NS1/NS2 = nailed (require manager approval), SO = Special Order.

SYSTEM DESIGN PROBES

I1 — "Design a queue for 10 stores, 60 salespeople/store, 1000 visits/day."

EXCELLENT ANSWER

- **Storage.** Keep MySQL for state. Partition `ups_logs` by `business_date`. Per-store advisory locks survive — already sharded by store.
- **Real-time.** Drop 1s poll. SSE feed per store, server-pushes deltas from `ups_logs` inserts (one event per insert). Fall back to poll on disconnect.
- **Fairness.** Unchanged — single-store shard already handles 60 salespeople. Fair-insert is $O(n)$ over today's queue, bounded by salespeople-on-floor.
- **Daily summary.** Parallel per-store builds. PDF rendering is the bottleneck; Puppeteer does ~5 PDFs/sec.
- **Read replicas.** MSSQL reads to a replica. MySQL reads (`/top-performers` , `/today`) to a replica with master-write.
- **Frontend.** Virtualize the queue list (60 rows is small but at 100+ I'd want windowing).

I2 — "Design observability for this system."

EXCELLENT ANSWER

- Per-request correlation id propagated through `ups_logs.request_id` and `report_*` audit columns.
- Structured logging (JSON to stdout) with `event` , `userId` , `store` , `requestId` , `durationMs` .
- APM (CloudWatch / Datadog) for HTTP and DB query timings.
- Daily KPI dashboard from `report_job_history` + `report_delivery_logs` (uptime SLO for the report).
- Alarms: any `report_job_history.status='failed'` , Birdeye claim rollbacks $> N/\text{hour}$, `withStoreTx` lock-timeout rate $> X\%$.

CODE QUALITY PROBES

J1 — "Show me a piece of code you regret."

EXCELLENT ANSWER

" `InternalInvoice.jsx` is doing too much in one file. Renders the warehouse invoice with per-component LOC rows, package summary rows, multi-instance qty multipliers like `(qty/pkg × qty_pkgs = total)` . It's correct, but cognitive load to read is high. I'd refactor by extracting the package-component row renderer into its own component — but I haven't because the file isn't changing weekly."

J2 — "Where's your most over-engineered code?"

EXCELLENT ANSWER

"Dual `componentGroups` + `components` response shape from `/api/package/:packageId/components`. I kept the legacy flat `components` list for back-compat with code that might still consume it; the new grouped shape was what the dialog actually needed. The flat list is now unused. I'd remove it."

J3 — "Where's the worst naming?"

EXCELLENT ANSWER

"`cfc/src/pages/InternalInvoice.jsx` — typo in the filename. 'Invoice' missing an 'i.' I left it because renaming is a route change and the page is rendered at `/internal-invoice/:slug` regardless of filename. Worth fixing in a quiet week."



Universal evaluation rubric

For each technical answer, the interviewer is silently scoring:

DIMENSION	WEAK (1)	STRONG (5)
Specificity	"We use locks"	"Per-store named advisory lock via <code>`GET_LOCK('ups_queue_lock:'</code> "
Tradeoff	"It's the right choice"	"Rejected Redis because [cost]; chose advisory lock because [benefit] at the cost of [downside]"
Failure modes	"It just works"	"Lock-timeout → 503 retryable; deadlock retried twice with fresh connection"
Generalization	"I fixed this bug"	"The pattern (resilient consumer, append-only audit, atomic CAS in SQL) generalizes to [other use cases]"
Self-awareness	"All my code is clean"	"I have a typo'd filename and a dual-shape response. Both deliberate compromises with known refactor paths"

Aim for 4–5 across the board on every answer. The interviewer is looking for the candidate who can defend AND criticize their own work.

**PART VI**

HIRING MANAGER ROUND

Resume Walkthrough · Polished Q&A

COVERS

Resume Deep Dive · Hiring Manager Questions

The hiring manager round = "can you talk about your work in a way that holds up to scrutiny." They will hand you the rope; your job is not to hang yourself with it. Below are the 11 questions that **will** be asked, in the order they're usually asked, with polished answers + probing follow-ups + delivery notes.



0 — Round Posture Cheat Sheet

BEHAVIOR	WHY
First person, always	They're hiring you, not your team.
Lead each answer with a one-line thesis	Sets the frame, lets them interrupt if curious.
End each answer with a hook	"And the most interesting part was..." gives them somewhere to go.
Numbers > adjectives	"Two emails a day, every send provable in one SQL query" beats "reliable."
Acknowledge cost	Every decision has a downside. Naming yours = maturity.
Stay on cfc/ + backend/	They want to know what <i>you</i> built.

1 "Tell me about yourself / your project." (90 seconds)

POLISHED ANSWER

"I own a MERN platform for Carolina Furniture Concepts — a two-showroom retailer in North Carolina, stores in Arden and Waynesville. The frontend is React 18 + Vite + Redux Toolkit; the backend is Node and Express on MySQL for operational data and a read-only MSSQL legacy ERP for inventory.

The platform runs three workflows: a real-time customer-opportunity queue called the UPS Board with a fairness algorithm, digital sales invoices including a returns-to-stock path, and an automated 7 PM / 9 PM PDF daily report that emails the GM and archives to Google Drive.

The technical bits I'm proudest of: a race-safe per-store advisory-lock transaction primitive called `withStoreTx`, a multi-location quantity allocator whose required-qty has to be parsed out of a free-text ERP description, NY-timezone-anchored business dates across a UTC server, and an idle-lock UX for shared kiosks with re-auth that replays a queued action only if the same employee unlocks within 60 seconds.

I implemented every feature in those folders personally — schema design, API routes, services, frontend components, Redux state, cron jobs, Gmail and Drive OAuth integrations, and the observability tables that monitor the pipeline."

DELIVERY NOTES

- Time it. **90 seconds, not 120.** Anything longer and they're tuning out.
- Drop one hook in the last sentence — "race-safe primitive" or "atomic boolean claim" — that they'll follow up on.
- **Pause after the last sentence.** Let them pick the thread.

PROBING FOLLOW-UPS YOU'LL GET

LIKELY FOLLOW-UP	ONE-LINE ANSWER READY IN CHAMBER
Tell me more about the fairness algorithm	Eligibility gated on first-UP-of-day; ineligible joiners FIFO-tail-append; eligible joiners insert by <code>ups_count</code> ; tri-state env flag for rollout.
What's the timezone gotcha?	UTC server, NY business day; <code>Intl.DateTimeFormat</code> with <code>timeZone</code> is the only correct primitive.
Tell me about the kiosk UX	Idle-lock for employees only; queued action stores user-id snapshot + 60s TTL; cross-tab via <code>storage</code> event.

2 "What are you most proud of?"

POLISHED ANSWER

"The UPS Fairness V2 rollout. I designed a tri-state environment flag — `off`, `shadow`, `on` — and shipped the new fairness algorithm in shadow mode for a week, dual-writing to a new `ups_sessions` table while V1 reads stayed unchanged. That let me validate the algorithm's decisions against live production data before any user-visible behavior changed. The cutover was a single env-var flip with zero downtime and a clear path to revert.

The reason I'm proud of it isn't that it worked — it's that the pattern worked. The next month, when I rolled out the package-component location dialog, I used the same shape: feature flag (`PKG_COMPONENT_LOC_ENABLED`), graceful degradation when the new schema isn't deployed yet, instant rollback when an issue surfaced on day three."

PROBING FOLLOW-UPS

Q	A
How did the cutover go?	Quiet — which is what I want from infrastructure changes. Shadow week had already proven the algorithm.
What if shadow had shown a divergence?	Flip back to <code>off</code> , debug, redeploy. Risk is bounded — no read-path changes during shadow.
Did you have to coordinate with anyone?	Heads-up to the on-floor manager, no permission needed because the flag preserves invariants.

3 "What was the hardest bug?"

POLISHED ANSWER

"The Be Back metric drift. The symptom was three different services each clamping closing ratio at 100% — `ups.routes.js /top-performers`, `todayReports.service.js`, and `dailySalespersonSummary.service.js`. Three clamps in three services is a giant red flag that they're each compensating for an upstream bug.

I traced it to `beBack.service.js` writing two `ups_logs` rows for every Be Back handoff: `ups_taken` for the receiver and `ups_handed_off` for the giver. The intent was 'Be Back is a metric event.' But the business rule was actually 'Be Back is routing, not a metric event.' The dual write was inflating one side and deflating the other.

The fix on May 21, 2026: stop writing those two `ups_logs` rows. Add a 'BB Accepted' column to the daily summary so handoffs stay visible without distorting metrics. Two of the three clamps came out because the upstream data no longer breached the cap.

The hardest part wasn't the code — it was telling the GM the leaderboard he used in Monday's meeting was wrong. I led with the corrected numbers, not the diagnosis."

PROBING FOLLOW-UPS

Q	A
How long?	A day for the fix, a week for cleaning up downstream clamps and re-laying-out the daily PDF.
How did you know it was root cause?	Two of three clamps became removable. If I'd only fixed a symptom, all three would still be needed.
What's the lesson?	Three patches in three files for the same issue is a compensation pattern, not a fix.

4 "Tell me about a production-like issue."

POLISHED ANSWER

"May 23, 2026 — three days after rolling out the package-component location dialog. An issue surfaced that impacted the live floor. I didn't have full root cause yet, but I had `PKG_COMPONENT_LOC_ENABLED` — a feature flag in `cfc/src/utis/packageWarnings.js`. I flipped it to false in about 30 seconds.

No other code changes. No schema rollback. Saved rows in `pkg_component_locations` stayed in place for re-enablement. The dialog stopped mounting, the invoice rows stopped rendering child components, and the warehouse reverted to the pre-May-20 experience.

Then I had the rest of the day to investigate calmly. The key lesson: I had the flag because I built the flag in advance. Bias for Action is partly preparation."

PROBING FOLLOW-UPS

Q	A
Wasn't that a workaround?	Workaround now, fix later. The asymmetry of cost was the decision.
How do you decide?	Stuck floor + flag exists → flag off. Containable + no flag → forward fix.
Did you escalate first?	Informed, didn't ask. Reversibility makes "ask forgiveness" the right move.

5 "Tell me about a failure."

POLISHED ANSWER

"The first version of the package-component location feature on May 20 supported only one instance per item per package. Then I tested with a real 4-chair package — `ItemPackageMaster` had four rows for the same chair `item_id`, one per piece. My schema only had one row per `(package_id, item_id)`.

The lesson isn't 'check the data first' — that's the obvious lesson. The real lesson is: when I'm modeling a new domain, the schema should be as wide as the worst-case data shape, even if v1's UI only exposes the simple case. Cheap to widen the schema on day one; expensive to migrate later.

The fix is in production now: `UNIQUE(package_id, item_id, instance_index)`. The dialog shows each instance as a separate card with its own location picker."

PROBING FOLLOW-UPS

Q	A
Who was affected?	Internal — caught it in staging. But the schema migration the next week was avoidable cost.
Could you have known earlier?	Yes — one query against <code>ItemPackageMaster</code> grouped by <code>item_id</code> would have surfaced multi-instance packages. I added that as a "first query before schema" rule.
Generalize the lesson	Schema should support worst-case data on day one. Cheap to widen up-front; expensive to migrate after.

6 "Tell me about a disagreement."

POLISHED ANSWER

"The first version of the finishing-up paperwork timer. A teammate wanted `localStorage`-only — simpler, snappier, no server round trip. I disagreed because of cross-device behavior: a salesperson can start paperwork at the front counter and finish at the manager's desk on a different device, and the badge would be wrong on the second device.

I made the case with a concrete user scenario, not abstract principles. We landed on the hybrid: server-authoritative `ups_finish_logs` table is the truth, `localStorage` is a same-tab elapsed-timer accelerator with a 30-minute TTL.

Then on May 18 I shipped the atomic fix where `/ups/finish` is the only opener of an `ups_finish_logs` row — that eliminated a race condition between `/finish` and `/finish-up/start` that had been leaving duplicate open rows. I also added a `fallbackUserId` to `clearAllFinishingUp` so cleared-cache and second-device cases still close the row by `(userId, ended_at IS NULL)` lookup."

PROBING FOLLOW-UPS

Q	A
What if you'd lost?	I'd have committed by adding cross-tab <code>storage</code> listeners and writing 'this is a same-tab badge only' into the file header. The cross-device bug would have surfaced anyway.
What changed your teammate's mind?	The concrete scenario walked through on two physical devices. The flow was the argument.

7 "How did you improve an existing system?"

POLISHED ANSWER

"Three I'd highlight:

1. `idx_id_created_at` on `invoices_archive(created_at)` — turned a day-range list scan from $O(n)$ over the full archive to $O(\log n)$ for the internal read API.
2. **Be Back as routing, not metric event** — removed two `ups_logs` writes that had been distorting closing ratio for ~2 weeks. Killed two of three downstream clamps. The math became boring, which is what you want.
3. **Server-authoritative finishing-up** — converted a `localStorage`-only badge into a server-of-truth + `localStorage`-accelerator pattern. Cross-device behavior went from broken to correct."

8 "How would you scale this system?"

POLISHED ANSWER

"Three changes for 10 stores at the current per-store activity:

1. **WebSocket / SSE feed** to replace the 1-second `useUPSQueue` poll. At ~5 endpoints per poll and ~6 devices per store, the API is doing 30 reqs/sec per store today. At 10 stores that's 300 reqs/sec mostly returning unchanged data. SSE pushes deltas from `ups_logs` inserts.
2. **Parallel daily-summary builds** — currently sequential per store. With `Promise.all` and Puppeteer's ~5 PDFs/sec ceiling, the 7 PM / 9 PM window is fine for 10 stores.
3. **MSSQL inventory cache** — single in-memory cache layer per backend host. `ItemMaster` lookups are largely static and hit on every package fetch.

The thing that doesn't change is the fairness algorithm — it's sharded by store from day one via the per-store named advisory lock."

PROBING FOLLOW-UPS

Q	A
Bottleneck at 100 stores?	MySQL connection pool — <code>GET_LOCK</code> holds a connection. Per-store pools + horizontal Node deployment + partitioned <code>ups_logs</code> by <code>business_date</code> .
Where would I add caching first?	MSSQL inventory reads. Same item lookups hit on every package fetch across stores.

9 "How would you onboard a new engineer?"

POLISHED ANSWER

"Three artifacts, in order: `schema.sql`, then `backend/src/routes/ups.routes.js`, then `cfc/src/AUTH/authContext.jsx`.

Schema first because the schema tells the product's story. Every table block has a date prefix and a one-line purpose. Reading the schema chronologically is reading the product's history.

`ups.routes.js` is the central nervous system — over 2,400 lines covering every queue action and every reporting query. If you understand this file you understand 70% of the backend.

`authContext.jsx` next because the idle-lock and re-auth-replay UX is non-obvious. If you don't read this file first, you'll spend two days wondering why employees get logged out and managers don't.

After those, I'd pair them on adding a new section to the daily summary — that touches a service, a renderer, an audit table, and the cron orchestrator. They get a full vertical slice on day three."

10 "What technical debt exists?"

POLISHED ANSWER

"Three honest items:

1. **No unit-test suite for routes.** Pure-logic services (`packageQtyParser` , `attendanceException`) are pure functions and would be easy to add Jest tests for first. Schema constraints carry most of the validation load today.
2. **Dual response shape on `/api/package/:packageId/components`** — `componentGroups` (new) + `components` (legacy). The flat list is unused; should be removed.
3. **`InternalInvoice.jsx` filename typo.** Missing an 'i' in 'Invoice.' Left because renaming requires a route change. Worth fixing in a quiet sprint.

None of these are blockers; all are known and tracked in project memory. The discipline I've learned is 'refactor when the patch count tells me the model is wrong.' These don't meet that bar — they meet the 'fix in a quiet sprint' bar."

1 1 "What would you redesign if you could?"

POLISHED ANSWER

"The single Redux slice for both SALE and RTS form state. Today every reducer checks `transactionMode` . It works because the form is one coherent thing, but it bloats the slice. If I were starting over I'd have two slices and a selector that produces the unified preview.

Honestly though, the bigger redesign would be auth. Body `userId` is trusted everywhere, with role re-read from `cfc_users` for defense-in-depth. It's the right call for a LAN deployment, but if we went public-internet, I'd want HMAC-signed tokens over `{userId, issuedAt}` . The refactor would be small because every route already goes through `req.cfcUser` ."

12 — "Why Amazon? Why this team?" (customize per JD)

POLISHED ANSWER TEMPLATE

"Three reasons.

First, the scale. The work I've done is at one company, two showrooms — but the patterns I've used (per-store sharding, race-safe transactions, audit-driven observability, feature flags, idempotent jobs) are the same patterns you use at Amazon scale. I want to apply them to systems that 100× what I've worked on.

Second, the customer-obsession is not lip service in the kind of work I do. Building the abandoned-UP feature because top performers were being penalized for greeting non-customers — that's the same instinct as 'work backwards from the customer.'

Third, the bar. The audit tables I built before shipping the daily summary are a small example, but they're the kind of standard I want to be held to and to hold others to. Amazon's bar-raiser model is the system-level version of that instinct."

"Why SDE-1, not SDE-2 / not internship?"

"SDE-1 matches my experience — full ownership of features end-to-end, multi-system integration, but not yet leading multi-engineer projects. I'm hungry for the next step but honest about where I am."

13 — Questions YOU ask back

Pick 2–3 per round based on context:

QUESTION	WHY
What does success for an SDE-1 look like at 6 months? At 12?	Concrete expectations
Which part of the codebase has the most accumulated complexity?	Shows you'll engage with debt, not just features
How does code review work — checklist or cultural?	Maps to your conscientiousness
What's the deploy cadence and feature-flagging story?	Aligns with your rollout instincts
What's the most surprising thing about working here?	Open-ended; lets them say something real
What would make you regret hiring me?	Bold but works in last 5 min of round

NEVER ask

- Comp, perks, working hours, remote policy. Save for the recruiter.
- "Do you have a free lunch?"
- Anything you could Google.



14 — Polished one-liner answers (memorize these)

Q	A
Why Node not Python/Java?	"JS end-to-end — frontend devs can write backend, and form state lives in JSON anyway."
Why MySQL not Postgres?	"Legacy ERP is MSSQL; ops experience is MySQL. Postgres would be new operational surface area for marginal benefit."
Most-used lib?	"Redux Toolkit + redux-persist on frontend, <code>mysql2/promise</code> on backend."
Most-regretted dep?	"Topaz signature SDK — vendor-locked and fragile. Worth it because no real alternative exists at price point."
Code style?	"Functional core, imperative shell. Pure services (<code>packageQtyParser</code> , <code>attendanceException</code>), I/O-driven routes."
Testing discipline?	"Honest gap. Pure-logic files first. Schema constraints carry the load today."
First thing if you had a free week?	"Request-id correlation across <code>ups_logs</code> and <code>report_*</code> tables. Cheap to add, huge debugging payoff."
Most surprising thing you learned?	" <code>FOR ORDINALITY</code> in <code>JSON_TABLE</code> . SQL had been doing what I'd been doing in Node for years."

15 — Don't-say list

PHRASE	WHY IT FAILS
"It was easy"	Bar raisers will probe.
"We" (when you mean "I")	First-person ownership.
"I just..."	Downplays the work.
"I don't have a great example"	Pick a smaller story; never refuse.
"All my code is clean"	Lack of self-awareness.
"Looking back, the right answer was obvious"	Hindsight bias.
"I have no weaknesses"	Self-aware-ectomy.
"I work too hard"	The fakest weakness on Earth.

16 — Resume walkthrough rhythm

1. Name · current role at Carolina Furniture Concepts · location
2. Primary project – one sentence:
 "I own a MERN platform for a two-showroom retailer.
 React + Vite + Redux on top of Node + Express + MySQL + MSSQL."
3. Three named achievements (highest-impact first):
 - UPS Fairness V2 – per-store advisory-lock + shadow→on rollout
 - Daily Summary pipeline – 7 PM/9 PM PDF + Drive + Gmail + 3 audit tables
 - Birdeye duplicate-SMS fix – atomic boolean claim CAS in SQL
4. Stack list (one breath):
 React 18, Redux Toolkit, redux-persist, Vite, Tailwind,
 Node, Express, MySQL 8, MSSQL, Puppeteer, Gmail API, Drive API,
 cron, MySQL JSON_TABLE + CTE.
5. Education (one line).

Time it. **Under 75 seconds.** Practice the rhythm of pauses — every comma is breath.

17 — Final pre-round mental checklist

- ☐ Pitch (90s) memorized and timed.
- ☐ 3 STARs per principle ready (especially Ownership, Dive Deep, Earn Trust).

- ☐ One "I failed" story without flinching.
- ☐ One "I disagreed and committed" story with a concrete commit action.
- ☐ One "I went deep" story with a root-cause trace.
- ☐ Scaling answer ("10 stores") rehearsed.
- ☐ Schema list memorized.
- ☐ Concurrency primitive memorized: `GET_LOCK + FOR UPDATE` inside `withStoreTx`.
- ☐ Idempotency primitive memorized: atomic boolean claim.
- ☐ Fairness eligibility rule memorized: `first_ups_at IS NULL → tail`.
- ☐ NY-tz primitive memorized: `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })`.
- ☐ Two questions to ask back per round.
- ☐ First-person language audit.
- ☐ Five-second pause rule rehearsed (never rush into the next sentence).

Walk in calm. Lead with numbers. Close on hooks.

**PART VII**

FULL MOCK INTERVIEW LOOP

5 Rounds · Weak vs Strong vs Stronger

COVERS

Mock Interviews · Common Weak Answers vs Strong Answers

Five rounds. End-to-end simulation. Each question is followed by:

-  **WEAK** answer (what NOT to say)
-  **STRONG** answer (the version to deliver)
-  **WHAT THE INTERVIEWER EVALUATES**
-  **COMMON MISTAKES**
-  **STRONGER PHRASING** (an even tighter delivery)

Read it through once. Then rehearse aloud from memory. Then time yourself.

Loop Overview

ROUND	INTERVIEWER PERSONA	DURATION	THEME
1	Recruiter screen (or SDE-2 phone screen)	30 min	Pitch + project hook
2	Bar Raiser	45 min	Leadership Principles
3	Senior SDE	60 min	Project deep dive
4	Hiring Manager	45 min	Fit + working style
5	Cross-team Bar Raiser	45 min	Judgment + curveballs

ROUND 1 — Phone Screen / Introduction (30 min)

Interviewer M: "Hi, thanks for taking time. I'm M, an SDE-2 on Customer Behavior. We'll spend a few minutes on intros, then walk through one of your projects in detail. Sound good?"

Q1.1 — "Tell me about yourself."

WEAK

"I'm a MERN developer. I've worked on a few projects, including one for a furniture company. I like building features and solving problems. I'm interested in Amazon because of the scale."

Why weak: Generic, no specifics, says "MERN" without proving depth, no hook for follow-up.

■ STRONG (90 SECONDS, MEMORIZED)

"I own a MERN platform for Carolina Furniture Concepts — a two-showroom retailer in North Carolina. The frontend is React + Vite + Redux Toolkit; the backend is Node and Express on MySQL for operational data and a read-only MSSQL legacy ERP for inventory. The platform runs three workflows: a real-time customer-opportunity queue called the UPS Board, digital sales invoices with a returns-to-stock path, and an automated 7 PM / 9 PM PDF daily report that emails the GM and archives to Google Drive.

Technically the most interesting parts have been a race-safe per-store advisory-lock fairness algorithm, a multi-location quantity allocator whose required-qty has to be parsed out of a free-text ERP description, NY-timezone-anchored business dates across a UTC server, and an idle-lock UX for shared kiosks with re-auth-replay. I implemented every feature in those folders personally — schema design, API routes, services, frontend components, Redux state, cron jobs, Gmail and Drive OAuth, and the observability tables that monitor the pipeline."

🎯 EVALUATION

- Did they speak in first person?
- Did they give specifics over slogans?
- Did they hand you a hook to pull on?

⚠️ COMMON MISTAKES

- "I love coding" / "I'm a quick learner" — adjective soup.
- 2+ minutes — they're tuning out.
- No hook in the closing sentence.

✨ STRONGER PHRASING

Replace "platform for Carolina Furniture Concepts" with **"operational platform a two-showroom retailer uses every day to onboard customers, write invoices, and run the floor."** Concrete + customer-grounded.

Q1.2 — "Pick the most interesting bit and go deep."

Suggested pick: UPS Fairness V2 + shadow rollout. Has algorithm + concurrency + rollout pattern + clean revert.

■ STRONG (~5 MIN IF UNINTERRUPTED)

"The UPS Board is a real-time queue across two stores. Salespeople join the waiting line, take customers, go on break, get added back. The original V1 was strictly first-come-first-served — a late joiner with zero UPs would land at the bottom.

The new rule (V2) is eligibility-gated. An employee earns fairness only after their first UP of the day. Until that first UP, they're ineligible and FIFO-append to the tail. Once eligible — `ups_sessions.first_ups_at` stamped — they get inserted at the first incumbent with a higher `ups_count`. This prevents a 12:30 PM joiner with zero UPs from leapfrogging the crew that's been on the floor since open.

I built it as a tri-state environment flag: `UPS_FAIRNESS_V2 = off | shadow | on`. In shadow mode, V1 reads stay unchanged but I dual-write to a new `ups_sessions` table. That let me run a comparison script for a week — V2's decisions against V1's actual outcomes — without changing user-visible behavior. The cutover was a single env-var flip.

The concurrency primitive is a per-store named advisory lock — `GET_LOCK('ups_queue_lock:' || store, 5)` — plus `FOR UPDATE` on queue rows inside a transaction. Every queue mutation goes through one function called `withStoreTx`. Lock timeout returns 503 with a retryable flag; deadlocks are retried twice with a fresh connection.

The schema decision I'm proud of: `ups_sessions(user_id, store_name, business_date) UNIQUE`. Once `first_ups_at` is stamped, it stays stamped for the day — so a salesperson who handed off a Be Back customer doesn't lose their fairness eligibility.

The reason I'm proud of this isn't the algorithm — it's the pattern. Shadow → on rollout with one revert path is what I now use for every behavior change. Same shape was used for the package-component location feature later."

🎯 EVALUATION

- Stayed in first person?
- Coherent technical story (problem → solution → why it matters)?
- Offered hooks for follow-up?

⚠️ COMMON MISTAKES

- Walking through code line-by-line without naming the concept.
- Saying "race conditions" without naming the primitive (`GET_LOCK`).
- Forgetting the rollout story — interviewer wants the engineering judgment.

✦ STRONGER PHRASING

End on: "And the most interesting test of the design was the comparison script during shadow week — V2's would-be decisions against V1's actual ones, day by day. The diffs were all legitimate eligibility events, which told me the algorithm was right before any user noticed."

Q1.3 — "What was the trickiest edge case?"

■ WEAK

"Race conditions. We solved them with locks."

■ STRONG

"The 'stale queue ghost' from a silently failed `dailyReset`. The 9 PM reset wipes `ups_queue` and `ups_working_queue`. If the reset failed quietly, a row from yesterday would persist. The next morning the first joiner would see `queue.length > 0` — a ghost row — and fair-insert would treat them as a tail-append candidate even though the queue was effectively empty.

The fix was scoping the fair-insert function to today's rows: `WHERE DATE(q.joined_at) = CURDATE()`. The reset's reliability becomes irrelevant to fairness correctness — the function self-heals against missed resets.

The lesson: when there's a transient state table and a cleanup job, the consumers should be resilient to cleanup failures, not just the cleanup itself."

✦ STRONGER PHRASING

Close: "Consumer resilience to producer failure is now my default design instinct — `fairInsertV2`, `clearAllFinishingUp` with `fallbackUserId`, graceful degradation when `pkg_component_locations` is missing — same pattern."

Q1.4 — "What would you do differently?"

■ WEAK

"I'd write more tests."

STRONG

"Two things. First, I'd write a metrics-integrity test before adding the closing-ratio clamps. The clamps were patches; the test would have flagged that they were compensating for an upstream bug instead of being the answer. I wouldn't have spent two weeks publishing biased leaderboards.

Second, I'd model `pkg_component_locations` with `instance_index` on day one. The simple shape — one row per `(package_id, item_id)` — was a 5-minute optimization that cost a full rewrite when real packages with 4 of the same chair appeared."

ROUND 2 — Behavioral (45 min)

Interviewer K: "I'm K, a bar raiser. Four leadership principles, specific examples with numbers and artifacts. First person. Let's start with Ownership."

Q2.1 (Ownership) — "Tell me about taking responsibility outside your scope."

WEAK

"I once stayed late to finish a bug fix that wasn't mine."

STRONG

S: GM at Carolina Furniture wanted an end-of-day summary email with revenue, closing ratios, RTS activity, and attendance exceptions. Ask was a single email. I scoped it bigger.

T: build the pipeline reliable enough that the GM could stop Slack-screenshotting numbers, and that I could prove it worked even when nobody read the email.

A: Schedule `0 19,21 * * *` in NY timezone — two runs before the 9:01 PM reset. I built three audit tables — `report_job_history`, `report_delivery_logs`, `report_drive_upload_logs` — before I built the aggregator. Observability before functionality. Per-recipient sends are independent so one bad address doesn't kill the run. Idempotency via `isAlreadySucceeded(date)` — process crash and restart doesn't re-send.

R: GM gets two emails a day plus a Drive archive link. In the first week, one transient Gmail rate-limit failed one recipient — audit row told me which recipient, which error, which attempt. Retry logic took it from there. The pattern (audit tables before functionality) became my default for every subsequent feature."

 EVALUATION

- Did they exceed the literal ask?
- Generalizable behavior emerged?
- Numbers / named artifacts present?

 PROBING FOLLOW-UP

K: "Why three audit tables instead of one?"

Answer: "Different cardinality and failure semantics. `report_job_history` is one row per attempt — coarse-grained for 'did this run.' `report_delivery_logs` is one row per recipient per attempt — that's where the rate-limit failure lives. `report_drive_upload_logs` is one row per file uploaded — Drive is best-effort and might fail without the email failing. Separating them means a Drive outage doesn't cause a delivery row to be confused."

 COMMON MISTAKES

- "We" instead of "I."
- Vague R section ("users were happy").
- Forgetting the per-recipient independence design.

Q2.2 (Dive Deep) — "Tell me about a hard bug."

 STRONG

S: Around April 14, 2026, the team reported customers occasionally receiving two Birdeye SMS for the same invoice. ~1 in 50 saves.

T: Find root cause. First hypothesis was a frontend double-click; proposed fix was to disable the Save button. I rejected that — masking a race instead of fixing it.

A: Read `route.invpdf.js /invoice/save` carefully. Two things happen on save: UPSERT into `invoices_archive` with dedup on `(billTo.name1, billTo.cell, DATE(created_at))`, and the Birdeye SMS branch. The SMS branch read `birdeye_sent == 0` and then wrote `birdeye_sent = 1`. Read-then-write, not atomic. Two requests could both read 0, both decide to send, both write 1. Reproduced by opening the same invoice in two tabs and clicking Save in both within a second — got two SMS every time.

Fix: atomic claim. `UPDATE invoices_archive SET birdeye_sent = 1 WHERE slug = ? AND birdeye_sent = 0`. Only the request whose `affectedRows == 1` sends. On SMS-send failure, roll back: `UPDATE ... SET birdeye_sent = 0` so a retry can re-claim.

R: Zero duplicate sends since deploy. The `birdeye_sent` column doubles as a support flag — 'did this customer receive the SMS?' is one SELECT."

 PROBING FOLLOW-UP

K: "Why didn't you use `SELECT ... FOR UPDATE ?`"

Answer: "It would have worked, but it holds a row lock through the Birdeye HTTP call — multi-hundred-ms. The atomic UPDATE releases the lock instantly and the HTTP call happens lock-free. The atomic compare-and-swap is the simplest form of the right answer."

Q2.3 (Bias for Action) — "Tell me about acting with incomplete information."

 STRONG

S: May 23, 2026. Three days after rolling out the package-component location dialog, an issue surfaced on the live floor.

T: Restore floor operations fast, then investigate.

A: Flipped `PKG_COMPONENT_LOC_ENABLED` to false in `cfc/src/utls/packageWarnings.js`. No other code change. No schema rollback. The dialog stopped mounting, invoice rows reverted to single-row-per-package, the warehouse went back to the pre-May-20 experience. Saved data in `pkg_component_locations` stayed in place for re-enablement.

R: Floor was unaffected within minutes. I had the rest of the day to investigate calmly. The key prep work was that I'd built the flag in advance — Bias for Action is partly preparation."

 PROBING FOLLOW-UP

K: "How do you decide flag-off vs fix-forward?"

Answer: "Two criteria. Containable + no flag → fix forward fast. Stuck floor + flag exists → flag off, fix calmly. The decision rule is asymmetry of cost: an unnecessary flag-off costs an hour; a stuck floor costs revenue."

Q2.4 (Earn Trust) — "Tell me about delivering bad news."

STRONG

"S: I traced the Be Back metric drift to root cause. The closing-ratio leaderboard had been biased for ~2 weeks. The GM had used those numbers in Monday's all-hands.

T: Tell him before fixing.

A: I went in person, not Slack. Led with the corrected numbers — 'for the past two weeks, your top three on the leaderboard were actually X, Y, Z, not who you mentioned in standup.' Then explained the diagnosis — Be Back writing two `ups_logs` rows when it should have written zero. Then the plan — remove the writes, kill downstream clamps that were compensating, add a BB Accepted column so handoffs stay visible.

R: He had the right numbers within 24 hours. He re-published the prior week's leaderboard with the correction noted. Trust came from leading with the numbers, not the diagnosis."

PROBING FOLLOW-UP

K: "What if he'd pushed back on republishing?"

Answer: "I'd have presented the choice: re-publish corrected, or quietly suppress and never use those numbers again. The middle ground — keep the old numbers visible without correction — wasn't an option I'd have offered."

ROUND 3 — Project Deep Dive (60 min)

Interviewer R: "I'm R, senior SDE. Full hour, one project. Pick whichever you defend in greatest depth."

Suggested pick: UPS Board fairness V2 + the daily reporting pipeline. Covers concurrency, schema design, cron, observability.

Q3.1 — "Walk me through the schema."

STRONG

"Five tables for the queue layer:

1. `ups_queue` — waiting line. UNIQUE on (`user_id`, `store_name`).
2. `ups_working_queue` — currently with customer. `active_customers` supports stacked floor.
3. `ups_logs` — append-only audit. 20+ `action_type` ENUMs (`ups_taken` , `queue_join` , `break_start` , `break_end` , `be_back` , `be_back_accept` , `moved_to_working` , `abandoned_up` , `ups_handed_off` , `queue_leave`). `reason` JSON. `idx_ups_logs_store_action_date` .
4. `ups_sessions` — per-day fairness state. UNIQUE on (`user_id` , `store_name` , `business_date`) . `first_ups_at` immutable for the day once stamped.
5. `ups_finish_logs` — paperwork timer.

Plus three for the daily summary pipeline:

6. `report_job_history` — one row per attempt. `status`, `attempt_no`, `duration_ms`, `error_message`, `context_json`.
7. `report_delivery_logs` — one row per recipient per attempt. FK to `job_history`.
8. `report_drive_upload_logs` — one row per file upload.

Key design decision: separate **transient state** (queue tables, wiped nightly) from **immutable audit** (logs, sessions, finish_logs). Reports read from audit; live board reads from state. The 9 PM reset is safe by construction."

Q3.2 — "Walk through `fairInsertV2` line by line."

STRONG

"Inside `withStoreTx(store, async (conn) => { ... })`, so we hold `GET_LOCK('ups_queue_lock:' || store, 5)` and are in a transaction.

1. Load today's queue: `SELECT user_id, queue_position, ups_count FROM ups_queue WHERE store_name=? AND DATE(joined_at)=CURDATE() ORDER BY queue_position FOR UPDATE`. The DATE filter is the stale-queue-ghost defense.
2. Load joiner's session: `SELECT first_ups_at, ups_count FROM ups_sessions WHERE user_id=? AND store_name=? AND business_date=CURDATE() FOR UPDATE`.
3. Empty queue → joiner at position 1.
4. `first_ups_at IS NULL` → append at tail (ineligible).
5. Eligible → scan from position 2:
 - For each incumbent at position p, compare `incumbent.ups_count` vs `joiner.ups_count`.
 - Insert before first incumbent whose `ups_count > joiner.ups_count`.
 - If no such incumbent → append at tail.
6. Position 1 never overwritten (`recycle` is the only way to move it).
7. Compress positions 1..N.
8. Write `ups_logs.action_type='queue_join'`.

The eligibility test is the key insight. It separates 'I'm new today' from 'I belong in the rotation by ups_count'. The asymmetry — ineligible append, eligible insert — is the fairness rule."

Q3.3 — "What concurrency bug could still happen here?"

STRONG

"Two I can think of.

1. **Cross-store cleanup race.** `dailyReset` at 9:01 PM wipes both queue tables. If a `/take` arrives at 9:00:59 and reset hits at 9:01:00 mid-transaction, `FOR UPDATE` inside `withStoreTx` serializes them — reset waits for `/take` to commit, or vice versa. Worst case: `/take` succeeds, reset wipes it 1ms later, user briefly thinks they took a customer. Audit row in `ups_logs` still exists. Acceptable.
2. **Connection pool exhaustion.** `GET_LOCK` holds a connection. Many concurrent mutations on the same store can run the pool dry. Today `connectionLimit` is tuned high enough; at 10 stores I'd raise it and consider per-store pools."

Q3.4 — "Where would you add SLOs?"

 **STRONG**

"Three SLOs:

1. **Daily summary delivery: 99% of business days, all recipients receive within 15 minutes of cron tick.** Measured by `report_delivery_logs.status='sent'` count vs recipient count with timestamp diff.
2. **UPS take latency: p99 < 2s.** Add a timing column to `ups_logs` for `ups_taken`. Most failures would be `GET_LOCK` contention.
3. **Be Back atomicity: zero crossed audit rows per week.** A 'crossed row' is `be_back_accept` without a matching `be_back` in same store within 1s. Structurally enforced by `withStoreTx`; SLO catches code regressions."*

ROUND 4 — Hiring Manager (45 min)

Interviewer S: "I'm S, hiring manager for the team you'd join. I want to understand how you work and how you'd fit."

Q4.1 — "What kind of work energizes you?"

 **STRONG**

"Work where the right design simplifies the problem instead of layering more code on it. The Be Back metric fix is the canonical example: three services had clamps, and the clamps weren't the answer — removing two `ups_logs` writes was. The math became boring afterward.

The opposite kind of work — adding workaround on workaround — drains me. I'd rather spend an extra day finding root cause than ship a fast patch that I'll be paying interest on for months."

Q4.2 — "What's a weakness you're working on?"

 **WEAK**

"I work too hard."

 STRONG

"I undertest. The CFC codebase has good schema-level constraints — UNIQUE, FK, CHK — that catch bad writes, and the pure-logic services like `packageQtyParser` are easy-to-test pure functions. But I don't have a unit test suite for the routes. Deliberate tradeoff on a small team; a real liability on a larger one.

What I'm doing about it: every new feature, I write Jest tests for the pure-logic helpers first. The route is then mostly glue around tested helpers. Not full coverage, but tests target where bugs would cost the most."

Q4.3 — "Tell me about your relationship with the GM."

 STRONG

"Operational, daily, and direct. He's not technical — he reads the daily PDF and notices things. When I shipped the Be Back metric fix, the conversation that surfaced the bug was him asking 'why does this leaderboard column never go above 100?' That question, more than any code review, told me the design was wrong.

The pattern: ship a feature, watch him use it for a day, ask what he ignored or had to mentally translate. The May 16 executive email redesign came from one of those conversations — the old single-table format didn't survive a mobile read."

Q4.4 — "How do you handle competing priorities?"

 STRONG

*"I sequence by **blast radius and reversibility**. Recent example from May 20 week:*

- 1. Package-component location dialog first — biggest user impact, longest UX iteration lead time, protected by a feature flag so reversible.*
- 2. Abandoned UP second — clear scope, short build time, reuses the Be Back transition pattern.*
- 3. Daily summary tweaks last — incremental, low-risk, ships when ready.*

Spend uncertainty budget on the highest-impact / hardest-to-reverse work first, while you still have the calendar to fix mistakes."

Q4.5 — "What do you want to learn from working at Amazon?"

STRONG

"Three things specifically.

1. **Scale.** *The patterns I've used — per-store sharding, race-safe transactions, idempotent jobs, audit-driven observability — generalize to Amazon-scale, but I've only proven them at one company. I want to apply them where they'll be stress-tested.*
2. **Working in a large engineering organization.** *I've been small-team-and-fast. Code review at scale, design docs, on-call rotations, multi-team integration — I want to be good at all of those.*
3. **The Amazon bar.** *Bar-raiser as a system is a discipline I respect. I want to be held to that bar and to develop the judgment to hold others to it."*

ROUND 5 — Bar Raiser (45 min)

Interviewer L: *"I'm L, bar raiser from outside your team. My job is to look for someone who'd raise the bar of your prospective team. Mix of LPs and a couple of judgment questions."*

Q5.1 (Are Right, A Lot) — "Tell me about a judgment call you got wrong."

WEAK

"I once misjudged how long a project would take."

STRONG

"S: First version of the package-component location dialog. I designed `pkg_component_locations` keyed by `(package_id, item_id)` — one row per item per package. I tested with a 1-item-per-package case and it worked.

T: Ship a working dialog for the warehouse.

A: First real package I tested in staging was a 4-chair set. `ItemPackageMaster` had four rows for the same chair `item_id` — one per piece. My schema couldn't represent 'chair #1 → S1, chair #2 → S2.' I rewrote the schema with `instance_index` as a third column in the unique key, rewrote the dialog state to be per-instance, and redeployed.

R: Current schema handles multi-instance cleanly. Lesson generalized: when modeling a new domain, the schema should be as wide as the worst-case data shape on day one, even if v1's UI only exposes the simple case. Cheap to widen on day one; expensive to migrate later."

 PROBING FOLLOW-UP

L: "How do you find the worst-case shape now?"

Answer: "Before I touch the schema, I run a sample query against the source-of-truth table — in MSSQL or wherever — and ORDER BY count DESC. The first few rows tell me the variability I have to handle."

Q5.2 (Have Backbone) — "Tell me about committing to a decision you disagreed with."**STRONG**

"S: The CFC auth model. No JWT, no cookies — body `userId` is trusted, with role re-read from `cfc_users` on every request via `requireRole.js`.

I argued for HMAC-signed tokens early on. The argument back: we're behind a store firewall on a LAN. Complexity cost of tokens, key rotation, refresh flows, middleware layer wasn't justified for the threat model.

I committed by hardening the role check. Every sensitive operation goes through `requireRole(allowedRoles)` which re-reads from the DB — a forged `userId` can impersonate but cannot escalate. Sensitive routes (`/abandoned-up` , `/update-ups`) also double-check role in the route as defense-in-depth.

If the threat model changes — going public-internet, regulated industry, single internal incident — I have a written upgrade path. The refactor is small because every route already goes through `req.cfcUser`."

 PROBING FOLLOW-UP

L: "Why didn't you keep arguing?"

Answer: *"Because the team's call was reasonable for the scope. Committed-after-disagreement is its own discipline. If I keep arguing every decision I lose, no one trusts me to commit. I save backbone for calls where the cost of being wrong is irreversible — auth on a LAN-only system wasn't one."*

Q5.3 (Curveball) — "If we hired you and a year in you realized the team isn't a good fit, what would you do?"

STRONG

"First, define 'not a good fit' specifically — not enjoying the work, not learning, manager mismatch, or scope misaligned with my goals. The action depends."

If it's scope — work doesn't stretch me — I'd talk to my manager about taking on adjacent work or stretching toward SDE-2 ramp. If it's manager mismatch, same conversation but framed differently, and Amazon has internal transfer processes.

If after honest conversations the fit doesn't improve, I'd look for an internal move before considering external. I'd be transparent with my manager rather than ghosting. The bar I'd hold myself to is the same one I'd hold the team to — direct, fast, honest."

EVALUATION

- Self-awareness about your career?
- Will you raise issues directly?
- Loyalty + transparency?

Q5.4 (Pure judgment) — "Two weeks in, you notice a money-handling service has no test coverage. Manager wants to ship a feature this sprint. What do you do?"

STRONG

"First, get the facts. 'No coverage' could mean three things: zero automated tests of any kind, no tests for this code path, or tests exist but coverage tools say insufficient. Action depends."

If literally zero coverage on money-handling code, that's a real risk. Two-week-tenured me doesn't have the standing to halt a sprint, but I have the standing to surface. I'd write a one-page note: 'Here's what I observed. Here's the concrete risk scenario (refund double-issuance). Here's a proposal — write tests for the specific change in this sprint, add a backlog item for the rest.' Then commit to the manager's call.

If the feature is urgent and the gap predates me, I'd ship — but I'd ship with tests for the new code path, plus the backlog item. The 'halt the sprint' move at week two would be reading too far into my mandate."

Q5.5 — "Last one. What would your previous manager say is your biggest growth area?"

STRONG

"He'd say I sometimes optimize for the small team I'm currently in. I've built audit tables, idempotency patterns, and feature flags that are right for two showrooms and one engineer. He'd push me to design for what the team will look like in two years — more engineers, more code review, more documentation overhead.

What I'm doing about it: I started writing project memories as durable notes that survive my own absence. The auto-memory files I maintain — naming conventions, dated migration blocks, rollback flags — are essentially the 'second engineer onboarding' I'd need if the team grew tomorrow."

COMMON MISTAKES (across all rounds)

MISTAKE	WHY IT COSTS YOU
"We" when you mean "I"	They're hiring you, not your team.
Vague results	"Users were happy" is unfalsifiable. Always name a table, a number, or a behavior.
Refusing to admit weakness	"I work too hard" reads as zero self-awareness.
Defensive under probing	Probes are curiosity. Expand depth, don't push back.
No hook in closing sentence	Each answer ends in dead space — interviewer flails.
Memorized answers without context	Canned reads canned. Adapt to the room.
Not asking questions back	When invited, ask. Reserving for the recruiter is a tell.
Talking past the time	2 min behavioral; 4–5 min deep technical; let them interrupt.
Apologizing for not having a story	Adapt. Pick a smaller story. Never refuse.
Forgetting to confirm understanding	"Did that answer your question, or did you want me to go deeper?" is a strong close.

**STRONGER PHRASING — UNIVERSAL UPGRADES**

VAGUE	SPECIFIC
"I led the project"	"I owned the schema, the orchestrator, and the three audit tables end-to-end."
"Users found it useful"	"GM gets two emails a day; every send is provable in one SQL query."
"We refactored it"	"I removed two <code>ups_logs</code> writes from <code>beBack.service.js</code> ; two of three clamps became removable."
"It scales well"	"Per-store advisory lock means adding stores is linear; no change to the algorithm."
"There was a race condition"	"Two <code>/invoice/save</code> requests for the same slug could both read <code>birdeye_sent=0</code> before either wrote."
"I learned a lot"	"I generalized 'consumer resilient to producer failure' from this fix and applied it to <code>clearAllFinishingUp</code> ."
"We had a bug"	"Three <code>LEAST(closing_ratio, 100)</code> clamps in three services — a compensation pattern that pointed to upstream double-write."

**PRE-INTERVIEW CHECKLIST (final 24 hours)**

- ☐ Pitch (90s) timed three times.
- ☐ Three STARs per principle ready (especially Ownership, Dive Deep, Earn Trust, Bias for Action).
- ☐ One "I failed" story you can defend without flinching.
- ☐ One "I disagreed and committed" with a concrete commit action.
- ☐ One "I went deep" with a root-cause trace.
- ☐ System design ("10 stores") answer rehearsed.
- ☐ Schema list memorized: `cfc_users` · `ups_queue` · `ups_working_queue` · `ups_logs` · `ups_sessions` · `ups_finish_logs` · `invoices_archive` · `customer_acquisition` · `customer_leads` · `customer_rts_entries` · `abandoned_up_logs` · `reminders` · `pkg_component_locations` · `manager_store_sessions` · `report_job_history` · `report_delivery_logs` · `report_drive_upload_logs` · `audit_results` .
- ☐ Concurrency primitive memorized: `GET_LOCK('ups_queue_lock:' || store, 5)` + `FOR UPDATE` + `withStoreTx` .
- ☐ Idempotency primitive memorized: `UPDATE ... SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0` .
- ☐ Fairness eligibility rule memorized: `first_ups_at IS NULL` → ineligible, FIFO tail.
- ☐ NY-tz primitive memorized: `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })` and `weekdayForDate` anchored to `T12:00:00Z` .

- ☐ Two questions to ask per round.
 - ☐ First-person language audit (no "we").
-

CLOSING MINDSET

"Walk in calm. Lead with numbers. Stay in first person. End each answer on a hook. Treat probes as curiosity, not attack. Acknowledge cost. Don't apologize as the lead. The corrected number is the lead."

And the most important rule: *if you have to think more than 3 seconds, you don't know the story well enough. Rehearse until it lives in muscle memory.*



FINAL AMAZON INTERVIEW CHEAT SHEET

The dense, last-mile revision page. Read on the morning of the loop.

The 10 Primitives — Memorize Verbatim

1. **Concurrency:** `GET_LOCK('ups_queue_lock:' || store, 5)` + `FOR UPDATE` inside `withStoreTx`. Every queue mutation routes through one function.
2. **Idempotency:** `UPDATE invoices_archive SET birdeye_sent=1 WHERE slug=? AND birdeye_sent=0`. CAS in SQL; `affectedRows === 1` is the success signal.
3. **Fairness:** `ups_sessions.first_ups_at IS NULL` $\Rightarrow$ ineligible, FIFO tail. After first take, immutable for the day.
4. **Tri-state rollout:** `UPS_FAIRNESS_V2 = off | shadow | on`. Shadow dual-writes; cutover is one env-var flip.
5. **Flag-based revert:** `PKG_COMPONENT_LOC_ENABLED` in `cfc/src/utils/packageWarnings.js`. 30-second revert, no code change.
6. **NY timezone:** `Intl.DateTimeFormat('en-CA', { timeZone: 'America/New_York' })`; weekday anchored to `T12:00:00Z`.
7. **Display rule:** `formatDisplayName(first, last)` — never username.
8. **Schema convention:** every block in `schema.sql` dated + "run this while deployment". Idempotent.
9. **Audit-first:** `report_job_history` + `report_delivery_logs` + `report_drive_upload_logs` written BEFORE the aggregator.
10. **Validation chain:** DB constraints $\rightarrow$ service rules $\rightarrow$ route role re-check $\rightarrow$ frontend gate. Validate once per layer.

Top 10 Talking Points (rank-ordered)

1. **UPS Fairness V2 shadow $\rightarrow$ on rollout** — one env-var flip, dual-write shadow week, zero downtime.
2. **Birdeye duplicate SMS fix** — atomic boolean claim CAS in SQL; replaced what a distributed lock would have done.
3. **Be Back metric drift root cause** — three clamps in three services pointed to upstream double-write; removed the writes; two clamps deleted.

4. **Audit tables before aggregator** — observability is not a feature you add later; it is what makes a feature operable.
5. **Package qty parser surface failures as null** — silent default of 1 would have shipped wrong picks; loud failure preserves correctness.
6. **30-second rollback via feature flag** — Bias for Action is partly preparation; the flag exists because I built it.
7. **Server-authoritative finishing-up + localStorage accelerator** — multi-device truth + same-tab snappiness.
8. **Idle-lock with re-auth replay** — three guards: identity snapshot, 60s TTL, cross-tab via storage event.
9. **Multi-instance schema for 4-chair packages** — schema should support worst-case shape on day one.
10. **Closing-ratio cap is definition, not a bug** — trust data over opinion; cap stays, double-counting goes.

Strongest Ownership Stories (rank-ordered)

1. **8 PM Daily Summary Pipeline** — built cron + PDF + Drive + Gmail + 3 audit tables before the aggregator. Two emails/day; every send provable in one SQL query.
2. **UPS Fairness V2** — designed the tri-state rollout pattern. The pattern became the model for every subsequent behavior change.
3. **Schema-as-history convention** — every block in `schema.sql` dated + idempotent. A second engineer can onboard from schema + one route file.

Strongest Dive Deep Stories (rank-ordered)

1. **Birdeye duplicate SMS** — rejected the surface fix (disable button) and traced to a non-atomic read-then-write. Solved with one SQL pattern.
2. **Be Back metric drift** — three clamps in three services as the smell; root cause was upstream `ups_logs` double-write.
3. **Stale queue ghost** — failed `dailyReset` would leave yesterday rows that broke fairness; fix is consumer-side `DATE(joined_at)=CURDATE()`.

Weak vs Strong — Universal Upgrade Table

VAGUE (WEAK)	SPECIFIC (STRONG)
"I led the project"	"I owned the schema, the orchestrator, and the three audit tables end-to-end."
"Users found it useful"	"GM gets two emails a day; every send is provable in one SQL query."
"We refactored it"	"I removed two <code>ups_logs</code> writes from <code>beBack.service.js</code> ; two of three clamps became removable."
"It scales well"	"Per-store advisory lock means adding stores is linear; no algorithm change."
"There was a race condition"	"Two <code>/invoice/save</code> requests for the same slug could both read <code>birdeye_sent=0</code> before either wrote."
"I learned a lot"	"I generalized 'consumer resilient to producer failure' from this fix; same pattern in <code>clearAllFinishingUp</code> ."
"We had a bug"	"Three <code>LEAST(closing_ratio, 100)</code> clamps in three services — a compensation pattern pointing to upstream double-write."

Final Pre-Loop Mental Checklist

- ☐ 90-second pitch timed three times
- ☐ Three STARs per principle ready
- ☐ One "I failed" story without flinching
- ☐ One "I disagreed and committed" with concrete commit action
- ☐ One "I went deep" with root-cause trace
- ☐ Scaling answer ("10 stores") rehearsed
- ☐ Concurrency primitive memorized
- ☐ Idempotency primitive memorized
- ☐ Fairness eligibility rule memorized
- ☐ NY-tz primitive memorized
- ☐ Two questions to ask per round
- ☐ First-person language audit (no "we")
- ☐ Five-second pause rule rehearsed
- ☐ Closing on hooks rehearsed

CLOSING MINDSET

Walk in calm. Lead with numbers. Stay in first person. End each answer on a hook. Treat probes as curiosity, not attack. Acknowledge cost. Don't apologize as the lead — the corrected number is the lead.

And the most important rule: **if you have to think more than 3 seconds, you don't know the story well enough. Rehearse until it lives in muscle memory.**